Digital Design & Computer Arch.

Lecture 10: Microarchitecture Fundamentals & Design II

Prof. Onur Mutlu

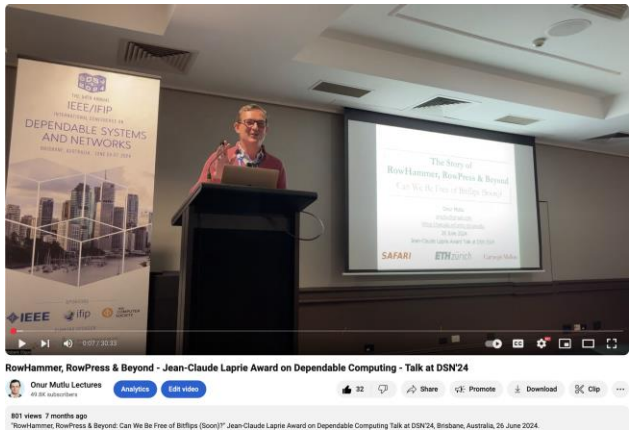
ETH Zürich

Spring 2025

21 March 2025

Extra Credit Assignment: Last Chance

- The Story of RowHammer, RowPress & Beyond
- **Watch and analyze this short lecture (30 minutes)**
  - https://www.youtube.com/watch?v=U1EcqXlclKU (June 2024)



- **Assignment – for 1% extra credit**
  - **Write a good 1-page individualized summary**
    - What are your key takeaways? What did you learn?
    - What surprised you about the content presented? What excited you?
    - What do you think solutions should be like?
    - Submit your summary to Moodle – deadline **March 21**

Extra Assignment 2: Moore's Law (I)

- **Paper review**
- G.E. Moore. "Cramming more components onto integrated circuits," Electronics magazine, 1965

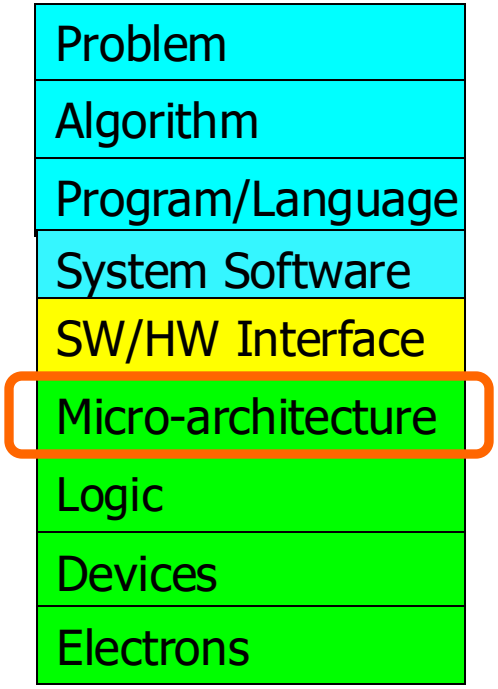
- **Optional Assignment – for 0.5% extra credit**
  - **Write a 1-page individual review**
    - What are your key takeaways? What did you learn?
    - What surprised you about the content presented? What excited you?
    - How do you think Moore's law evolved since the paper was written?
    - What do you think (future) solutions should be like?
    - Submit your summary to Moodle – deadline **April 1**

Cramming More Components onto Integrated Circuits

GORDON E. MOORE, LIFE FELLOW, IEEE

Agenda for Today & Next Few Lectures

- Instruction Set Architectures (ISA): LC-3 and MIPS
- Assembly programming: LC-3 and MIPS
- Microarchitecture (principles & single-cycle uarch)
- Multi-cycle microarchitecture
- Pipelining
- Issues in Pipelining:
  - Control & Data Dependence Handling
  - State Maintenance and Recovery
- Out-of-Order Execution



Readings

- This week
  - Introduction to microarchitecture and single-cycle microarchitecture
    - H&H, Chapter 7.1-7.3
    - P&P, Appendices A and C
  - Multi-cycle microarchitecture
    - H&H, Chapter 7.4
    - P&P, Appendices A and C

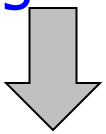
- Next week
  - Pipelining
    - H&H, Chapter 7.5
  - Pipelining Issues
    - H&H, Chapter 7.7, 7.8.1-7.8.3

Implementing the ISA: Microarchitecture Basics

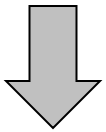
A Very Basic Instruction Processing Engine

- Each instruction takes a single clock cycle to execute
- Only combinational logic is used to implement instruction execution
  - *No intermediate, programmer-invisible state updates*

AS = Architectural (programmer visible) state
at the beginning of a clock cycle



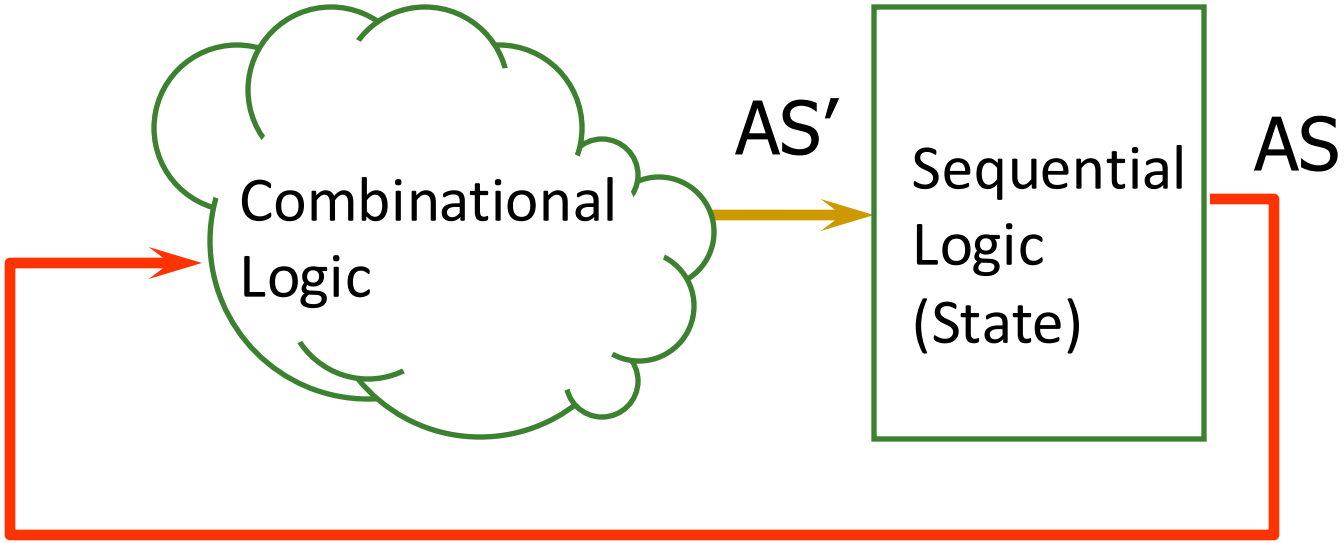
Process instruction in **one clock cycle**



AS' = Architectural (programmer visible) state
at the end of a clock cycle

A Very Basic Instruction Processing Engine

- Single-cycle machine



- What is the *clock cycle time* determined by?
- What is the *critical path* (i.e., longest delay path) of the combinational logic determined by?

AS: Architectural State

Recall: Single-cycle vs. Multi-cycle Machines

- Single-cycle machines
  - Each instruction takes a single clock cycle
  - All state updates made at the end of an instruction's execution
  - Big disadvantage: The slowest instruction determines cycle time → long clock cycle time

- Multi-cycle machines
  - Instruction processing broken into multiple cycles/stages
  - State updates can be made during an instruction's execution
  - Architectural state updates made at the end of an instruction's execution
  - Advantage over single-cycle: The slowest "stage" determines cycle time

- Both single-cycle and multi-cycle machines literally follow the von Neumann model at the microarchitecture level

Instruction Processing "Cycle"

- Instructions are processed under the direction of a "control unit" step by step.
- Instruction cycle: Sequence of steps to process an instruction
- Fundamentally, there are six steps:

- Fetch
- Decode
- Evaluate Address
- Fetch Operands
- Execute
- Store Result

- Not all instructions require all six steps (see P&P Ch. 4)

Recall: The Instruction Processing "Cycle"

- FETCH
- DECODE
- EVALUATE ADDRESS
- FETCH OPERANDS
- EXECUTE
- STORE RESULT

Instruction Processing "Cycle" vs. Machine Clock Cycle

- Single-cycle machine:
  - All six phases of the instruction processing cycle take a *single machine clock cycle* to complete

- Multi-cycle machine:
  - All six phases of the instruction processing cycle can take *multiple machine clock cycles* to complete
  - In fact, each phase can take multiple clock cycles to complete

Instruction Processing Viewed Another Way

- Instructions transform Data (AS) to Data' (AS')
- This transformation is done by functional units
  - Units that "operate" on data
- These units need to be told what to do to the data

- An instruction processing engine consists of two components
  - Datapath: Consists of hardware elements that deal with and transform data signals
    - **functional units** that operate on data
    - **hardware structures** (e.g., wires, muxes, decoders, tri-state bufs) that enable the flow of data into the functional units and registers
    - **storage units** that store data (e.g., registers)
  - Control logic: Consists of hardware elements that determine control signals, i.e., signals that specify what the datapath elements should do to the data

Recall: LC-3: A von Neumann Machine

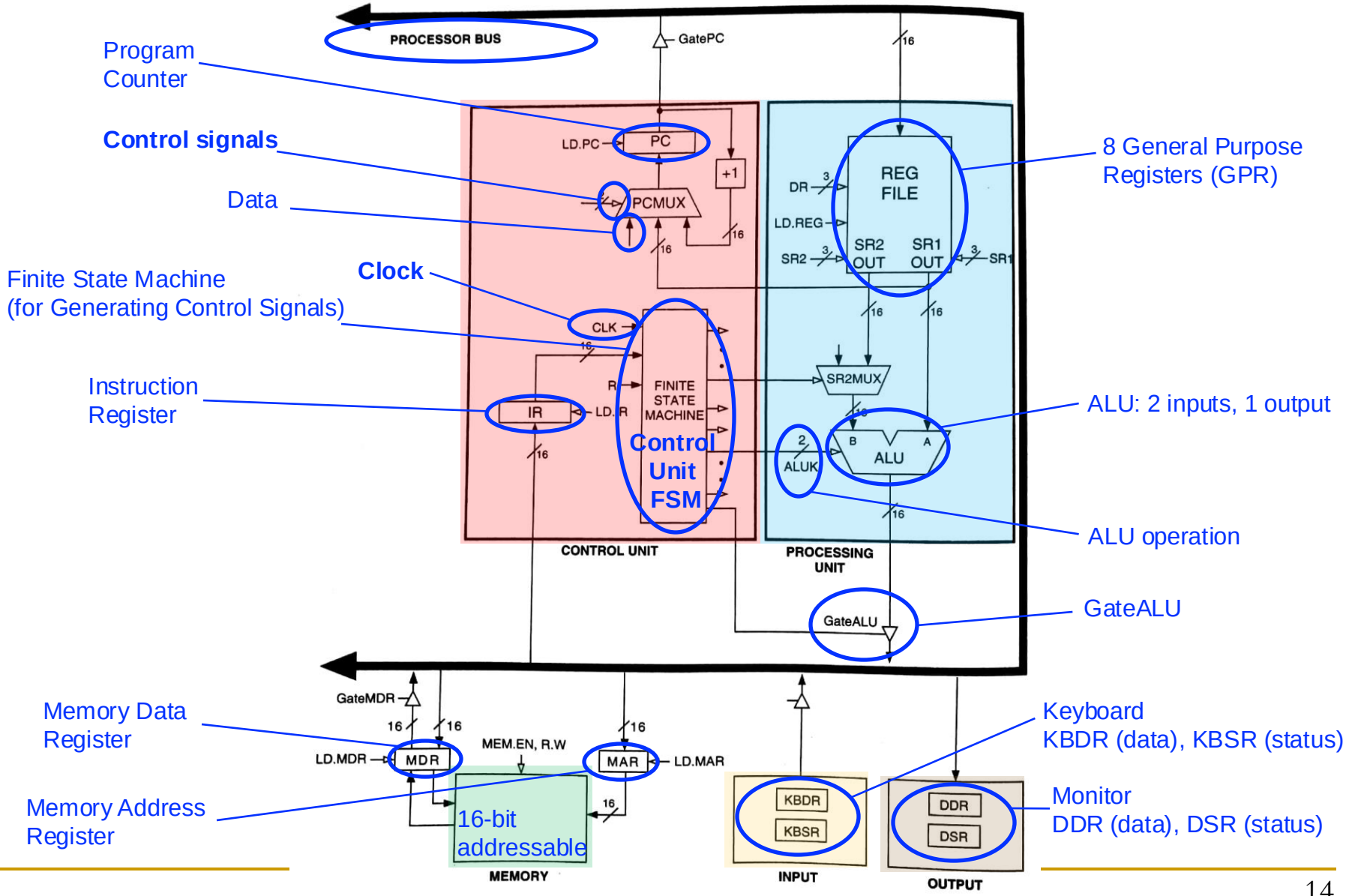


Figure 4.3 The LC-3 as an example of the von Neumann model

Single-cycle vs. Multi-cycle: Control & Data

- Single-cycle machine:
  - Control signals are generated in the same clock cycle as the one during which data signals are operated on
  - Everything related to an instruction happens in one clock cycle (serialized processing)

- Multi-cycle machine:
  - Control signals needed in the next cycle can be generated in the current cycle
  - Latency of control processing can be overlapped with latency of datapath operation (more parallelism)

- See P&P Appendix C for more (microprogrammed multi-cycle microarchitecture)

Many Ways of Datapath and Control Design

- There are many ways of designing the datapath and control logic

- Example ways
  - Single-cycle, multi-cycle, pipelined, out-of-order datapath & control
  - Single-bus vs. multi-bus datapaths
  - Hardwired/combinational vs. microcoded/microprogrammed control
    - Control signals generated by combinational logic versus
    - Control signals stored in a memory structure

- Control signals and structure depend on the datapath design

Flash-Forward: Performance Analysis

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**

CPI: Cycles Per Instruction

- Execution time of an entire program
  - Sum over all instructions [{CPI} x {clock cycle time}]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**

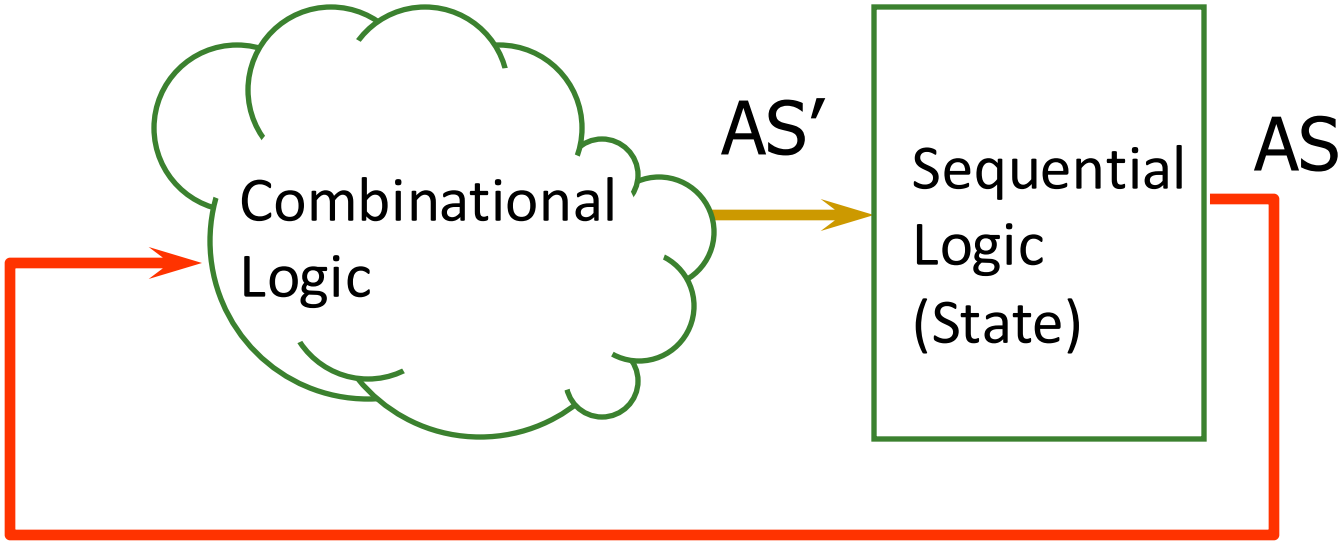
- Single-cycle microarchitecture performance
  - CPI = 1
  - Clock cycle time = long
- Multi-cycle microarchitecture performance
  - CPI = different for each instruction
    - Average CPI → hopefully small
  - Clock cycle time = short

In multi-cycle, we have two degrees of freedom to optimize independently

A Single-Cycle Microarchitecture *From the Ground Up*

Remember…

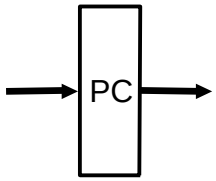
- Single-cycle machine

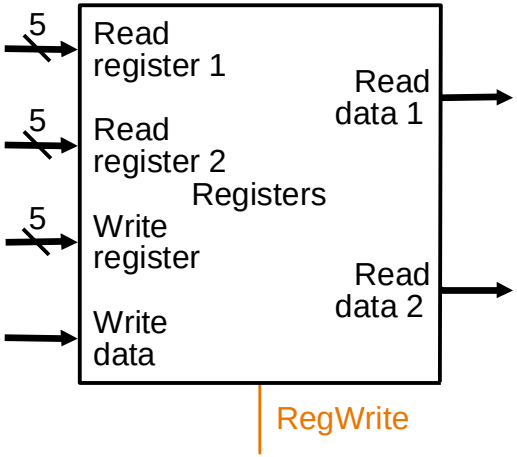


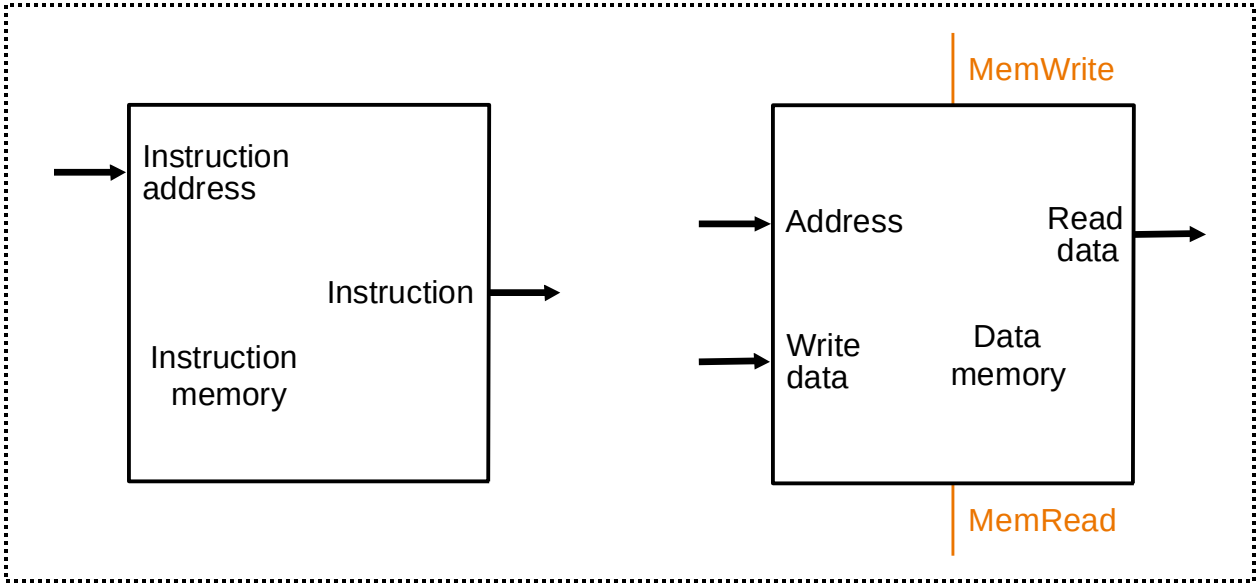
AS: Architectural State

Let's Start with the State Elements (MIPS)

- Data and control inputs



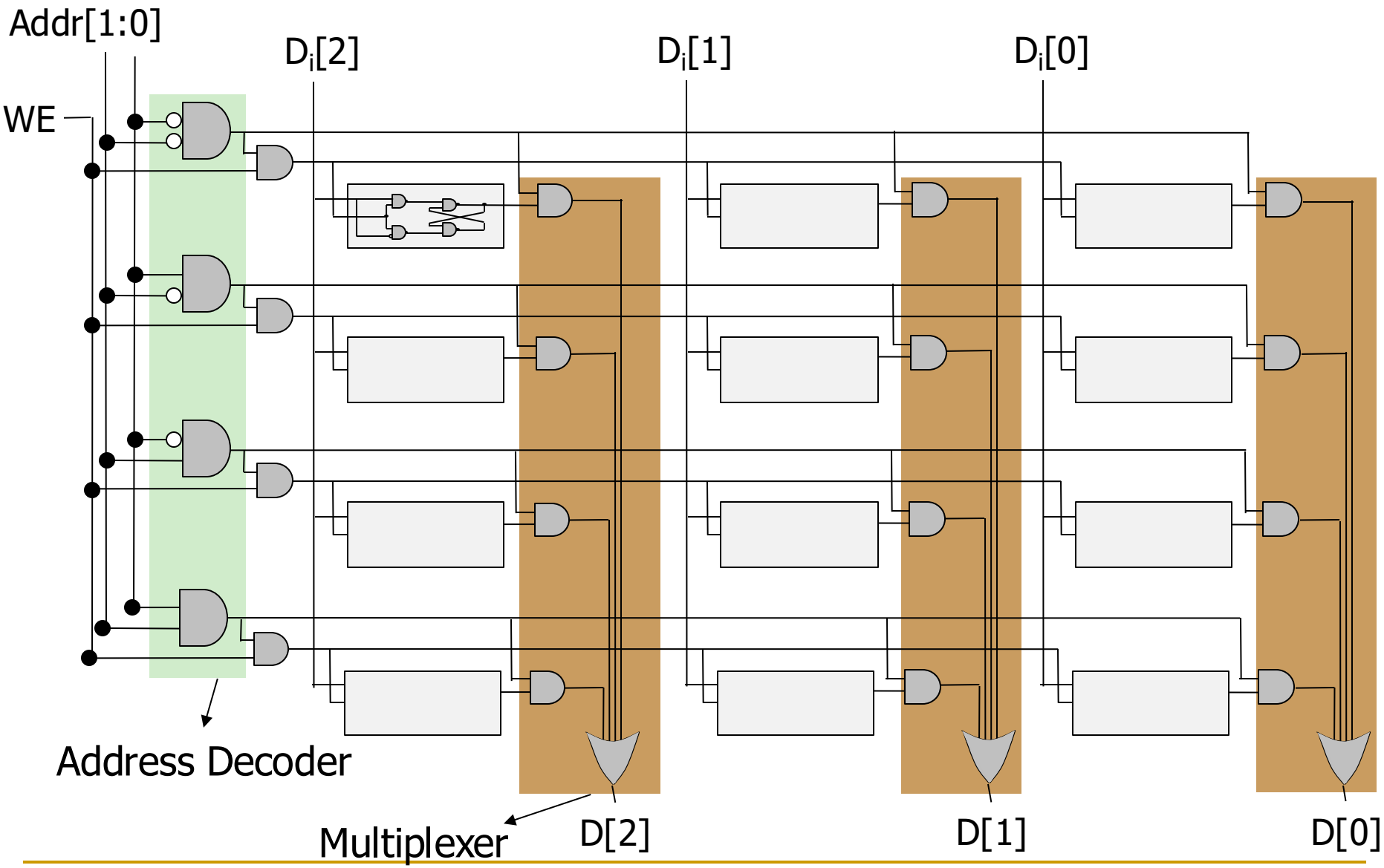




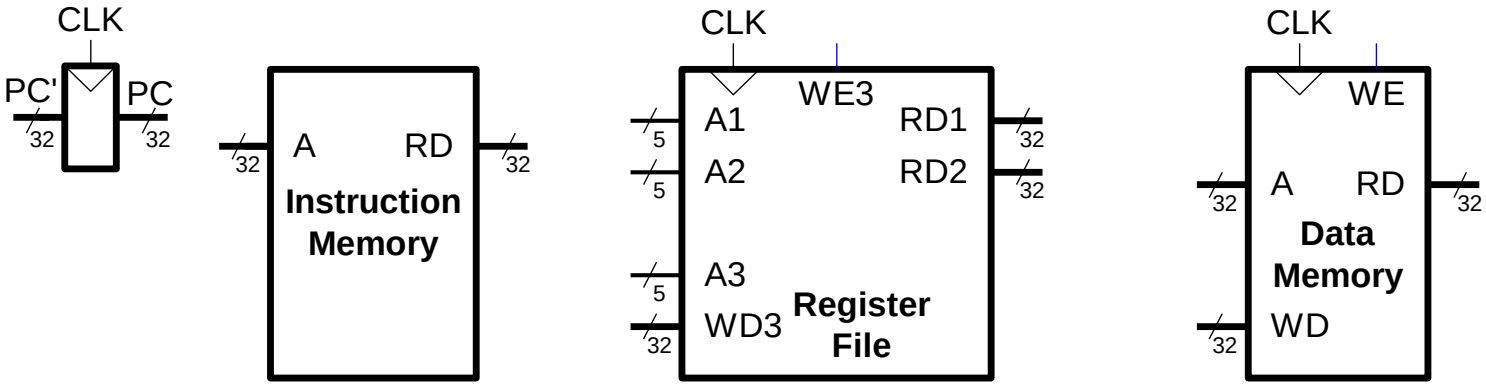
**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

We will use this notation in lectures

Recall: A Memory Array (4 locations X 3 bits)



MIPS State Elements



- Program counter:

  32-bit register

- Instruction memory:

  Takes input 32-bit address A and reads the 32-bit data (i.e., instruction) from that address to the read data output RD

- Register file:

  The 32-element, 32-bit register file has 2 read ports and 1 write port

- Data memory:

  If the write enable, WE, is 1, it writes 32-bit data WD into memory location at 32-bit address A on the rising edge of the clock.

  If the write enable is 0, it reads 32-bit data from address A onto RD.

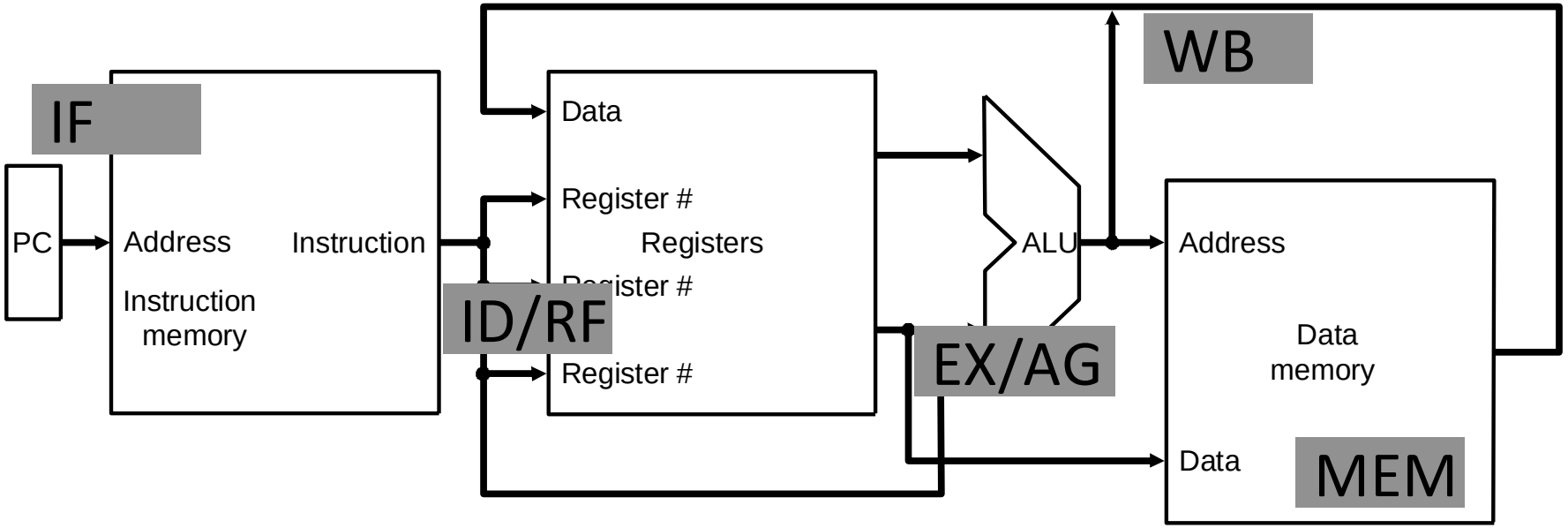
The H&H book uses this notation (H&H Chapter 7.3)

For Now, We Will Assume

- Ultra-fast (i.e., single-cycle) memory and register file
- Combinational read
  - output of the read data port is a combinational function of the memory contents and the corresponding input address
- Synchronous write
  - target location is updated at the positive edge clock transition when the write enable signal is asserted
    - Cannot affect read output in-between positive clock edges
- Single-cycle memory
  - Contrast this with memory that tells when the data is ready
    - i.e., Ready signal: indicating the read or write is done
      - See P&P Appendix C (LC3-b) for multi-cycle memory

Instruction Processing

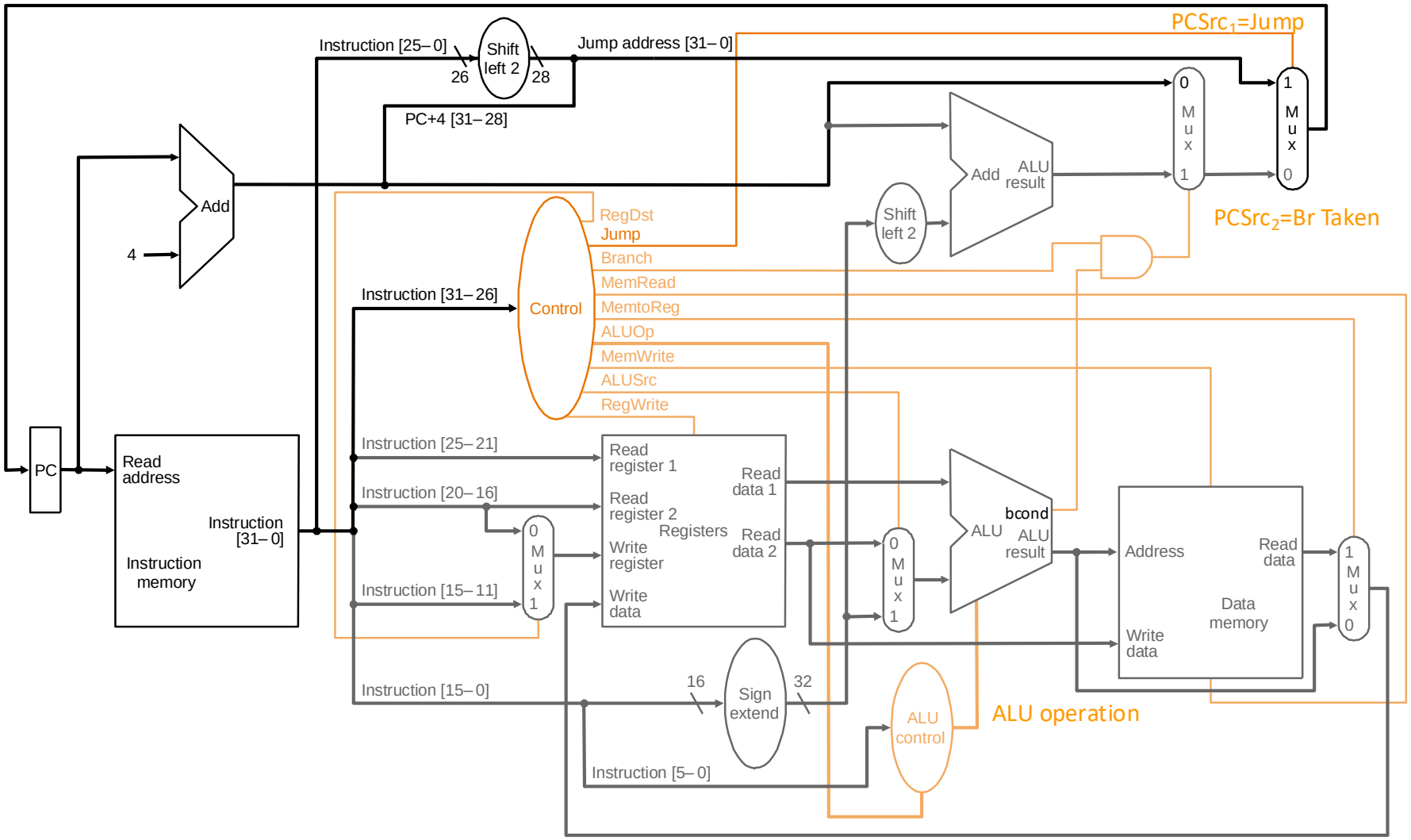
- 5 generic steps
  - Instruction fetch (IF)
  - Instruction decode and register operand fetch (ID/RF)
  - Execute/Evaluate memory address (EX/AG)
  - Memory operand fetch (MEM)
  - Store/writeback result (WB)



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

We Need to Provide the Datapath & Control Logic to Execute All ISA Instructions

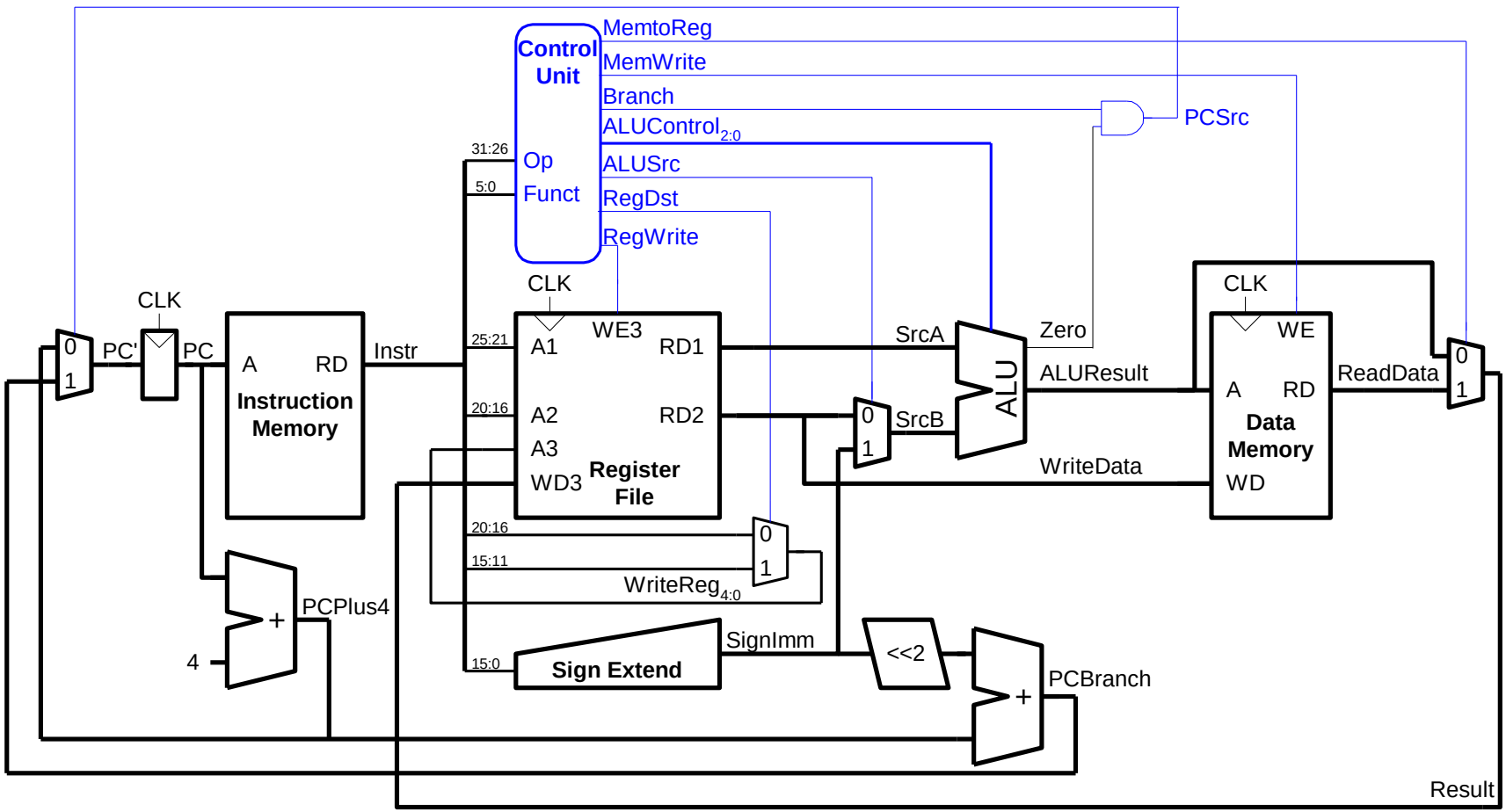
What Is To Come: Single-Cycle MIPS Processor



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

JAL, JR, JALR omitted

Another Complete Single-Cycle Processor



Single-cycle processor. Harris and Harris, Chapter 7.3.

Single-Cycle Datapath for *Arithmetic and Logical Instructions*

R-Type ALU Instructions

- R-type: 3 register operands

MIPS assembly (e.g., register-register signed addition)

```
add  $s0, $s1, $s2          #$s0=rd, $s1=rs, $s2=rt
```

Machine Encoding

0	rs	rt	rd	0	add (32)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

R-Type

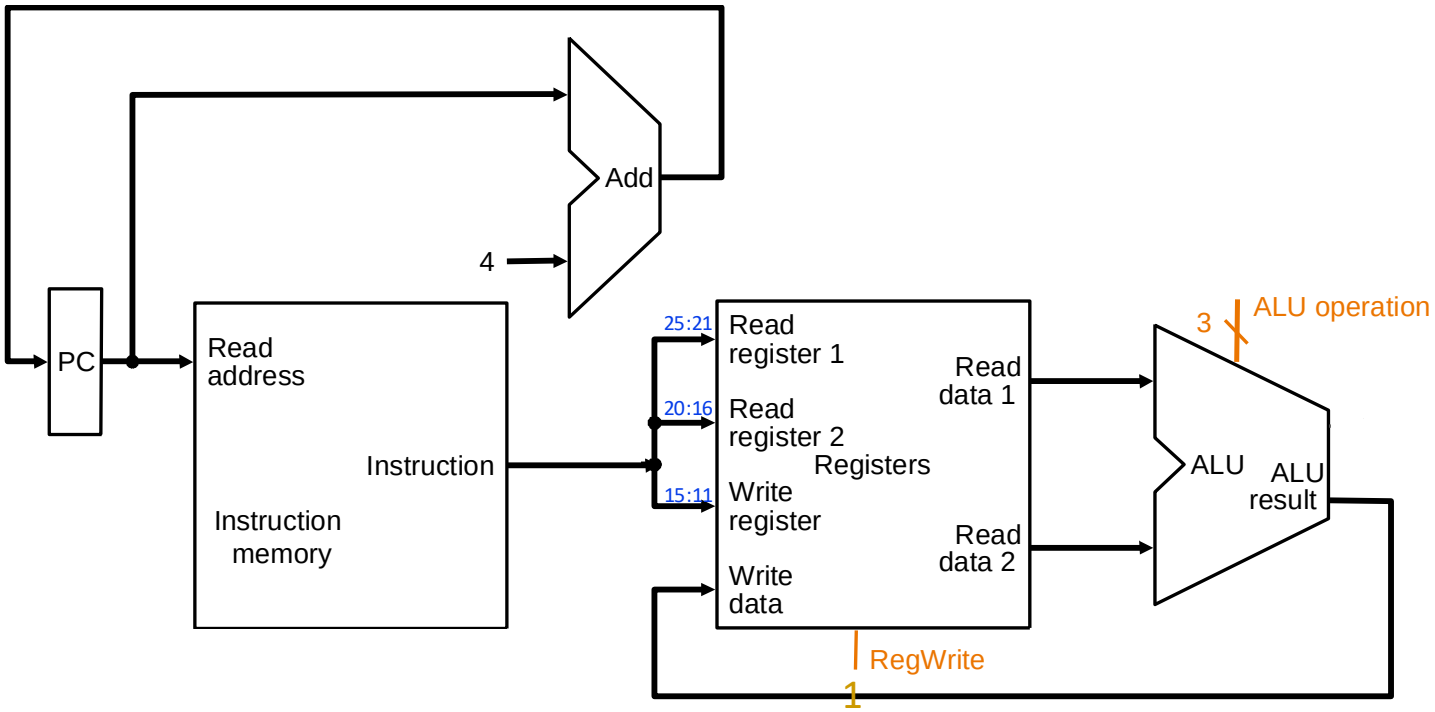
- Semantics

if MEM[PC] == add rd rs rt

GPR[rd] ← GPR[rs] + GPR[rt]

PC ← PC + 4

(R-Type) ALU Datapath



IF	ID	EX	MEM	WB

if MEM[PC] == ADD rd rs rt

GPR[rd] ← GPR[rs] + GPR[rt]

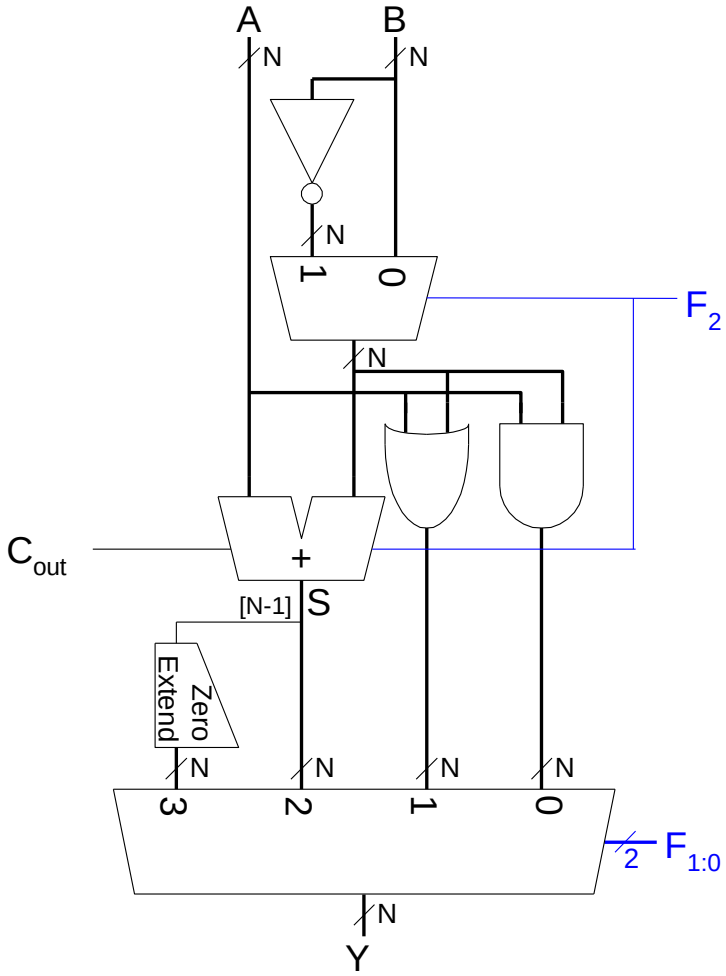
PC ← PC + 4

Combinational state update logic

**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Example: ALU Design

- ALU operation ($F_{2:0}$) comes from the control logic



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

I-Type ALU Instructions

- I-type: 2 register operands and 1 immediate

MIPS assembly (e.g., register-immediate signed addition)

```
addi $s0, $s1, 5                    #$s0=rt, $s1=rs
```

Machine Encoding

addi (0)	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

I-Type

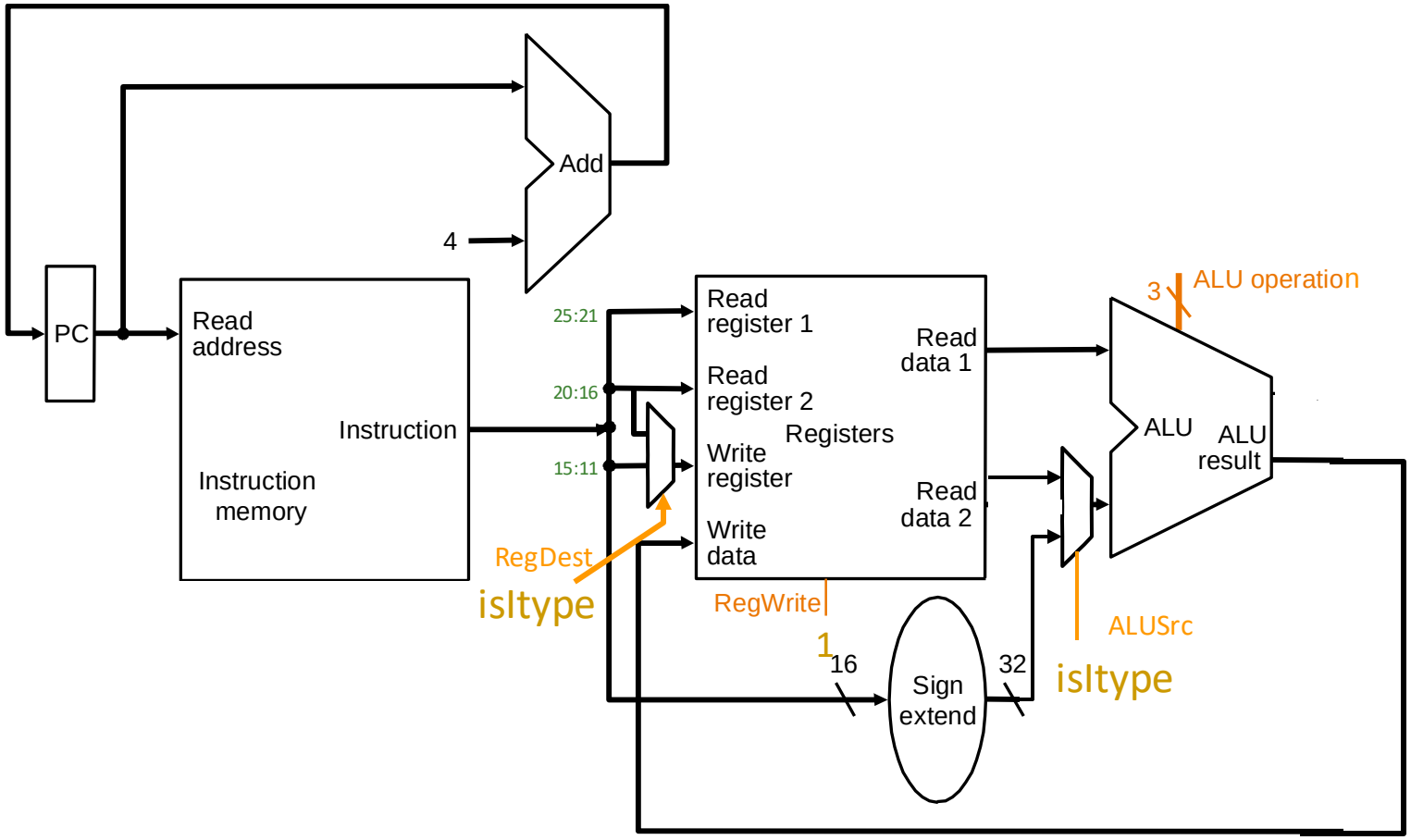
- Semantics

if MEM[PC] == addi rs rt immediate

PC ← PC + 4

GPR[rt] ← GPR[rs] + sign-extend(immediate)

Datapath for R- and I-Type ALU Insts.



IF	ID	EX	MEM	WB

if MEM[PC] == ADDI rt rs immediate

GPR[rt] ← GPR[rs] + sign-extend (immediate)

PC ← PC + 4

Combinational state update logic

**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Recall: ADD with one Literal in LC-3

- ADD assembly and machine code

LC-3 assembly

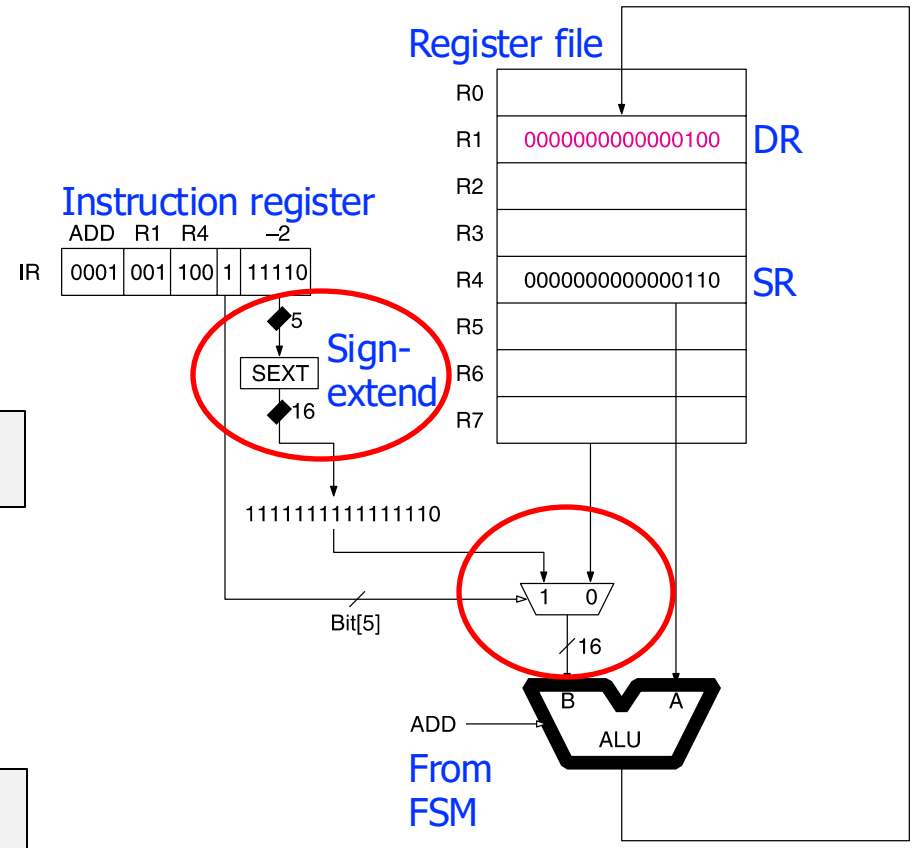
```
ADD R1, R4, #-2
```

Field Values

OP	DR	SR		imm5
1	1	4	1	-2

Machine Code

OP	DR	SR		imm5
0 0 0 1	0 0 1	1 0 0	1	1 1 1 1 0
15 12	11 9	8 6	5	4 0



Single-Cycle Datapath for *Data Movement Instructions*

Load Instructions

- Load 4-byte word

MIPS assembly

```
lw   $s3, 8($s0)                    #$s0=rs, $s3=rt
```

Machine Encoding

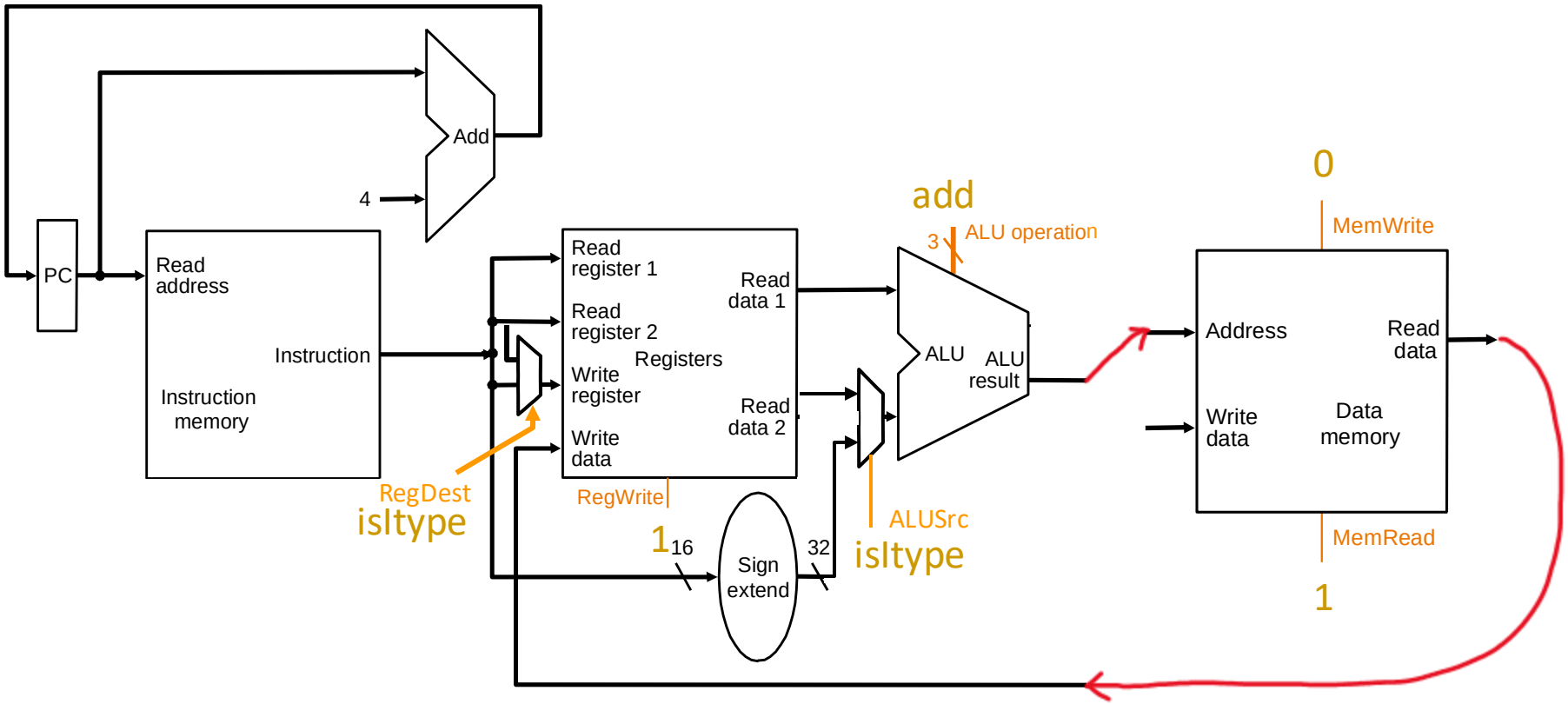
op	rs=base	rt	imm=offset
lw (35)	base	rt	offset
31 26	25 21	20 16	15 0

I-Type

- Semantics

if MEM[PC] == lw rt $offset_{16}$ (base)
  PC ← PC + 4
  address = sign-extend(offset) + GPR(base)
  GPR[rt] ← MEM[address]

LW Datapath



if MEM[PC]==LW rt offset$_{16}$ (base)

address = sign-extend(offset) + GPR[base]

GPR[rt] ← MEM[address]

PC ← PC + 4

IF	ID	EX	MEM	WB

Combinational state update logic

Store Instructions

- Store 4-byte word

MIPS assembly

```
sw    $s3, 8($s0)                    #$s0=rs, $s3=rt
```

Machine Encoding

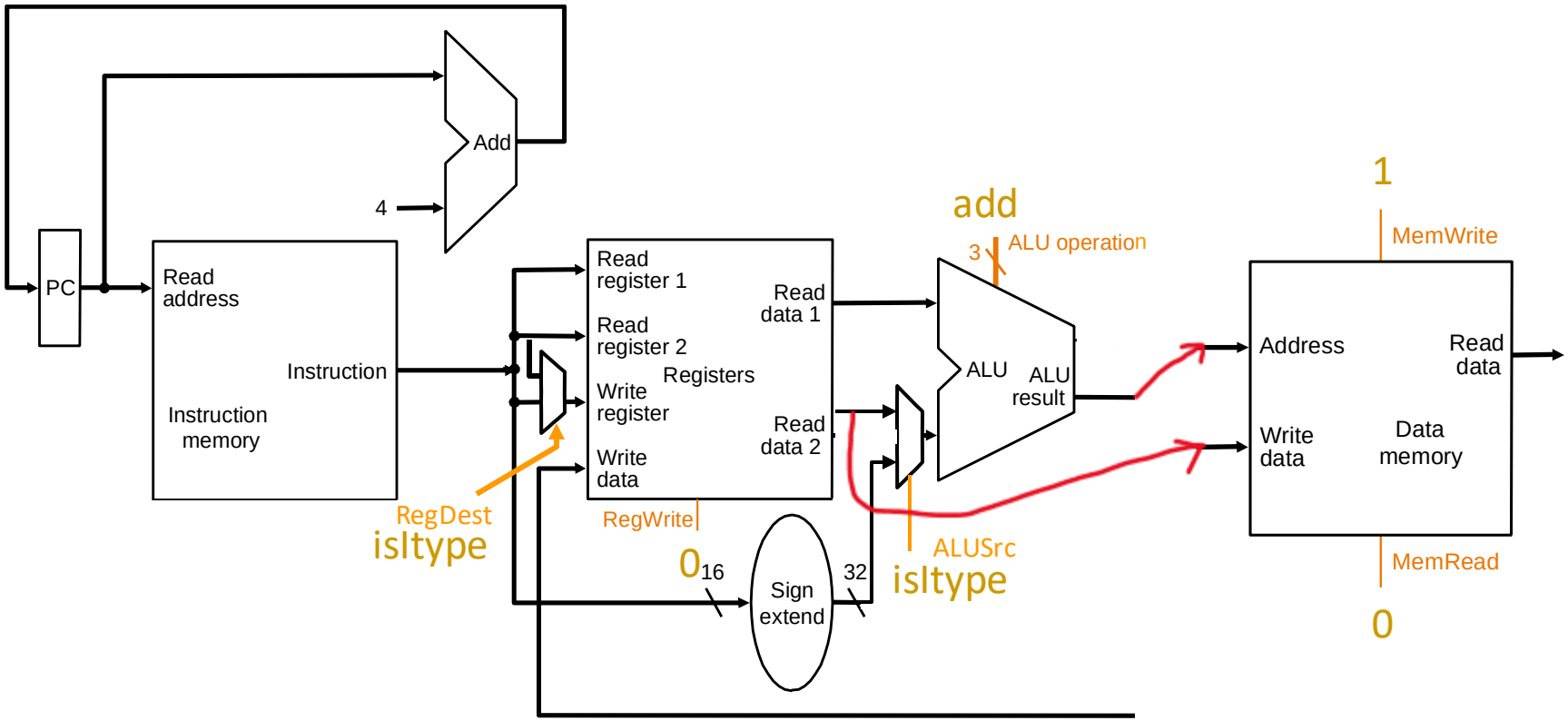
op	rs=base	rt	imm=offset
sw (43)	base	rt	offset
31 26	25 21	20 16	15 0

I-Type

- Semantics

if Mem[PC] == sw rt $offset_{16}$ (base)
  PC ← PC + 4
  address = sign-extend(offset) + GPR(base)
  MEM[address] ← GPR[rt]

SW Datapath



if MEM[PC]==SW rt offset$_{16}$ (base)

address = sign-extend(offset) + GPR[base]

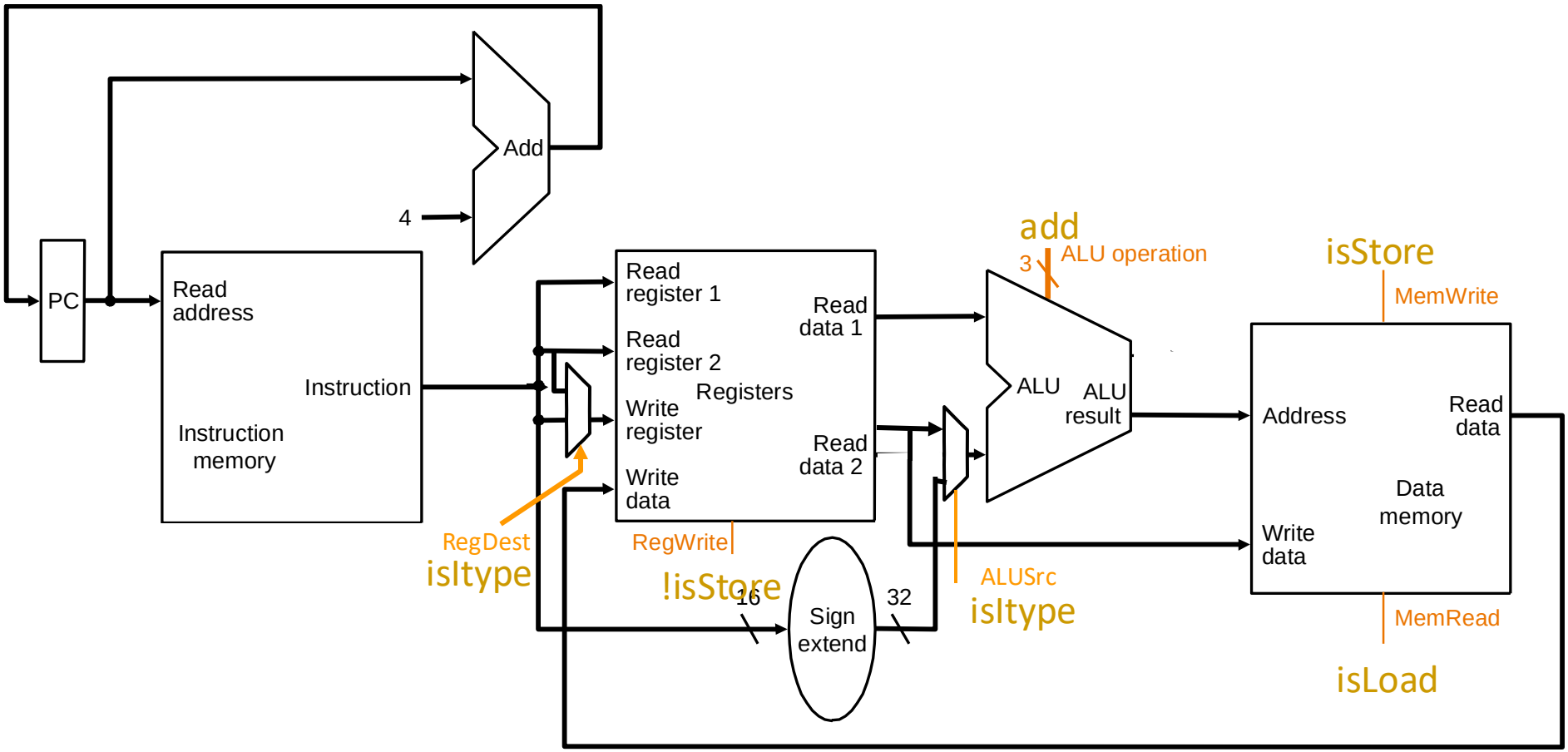
MEM[address] ← GPR[rt]

PC ← PC + 4

IF	ID	EX	MEM	WB

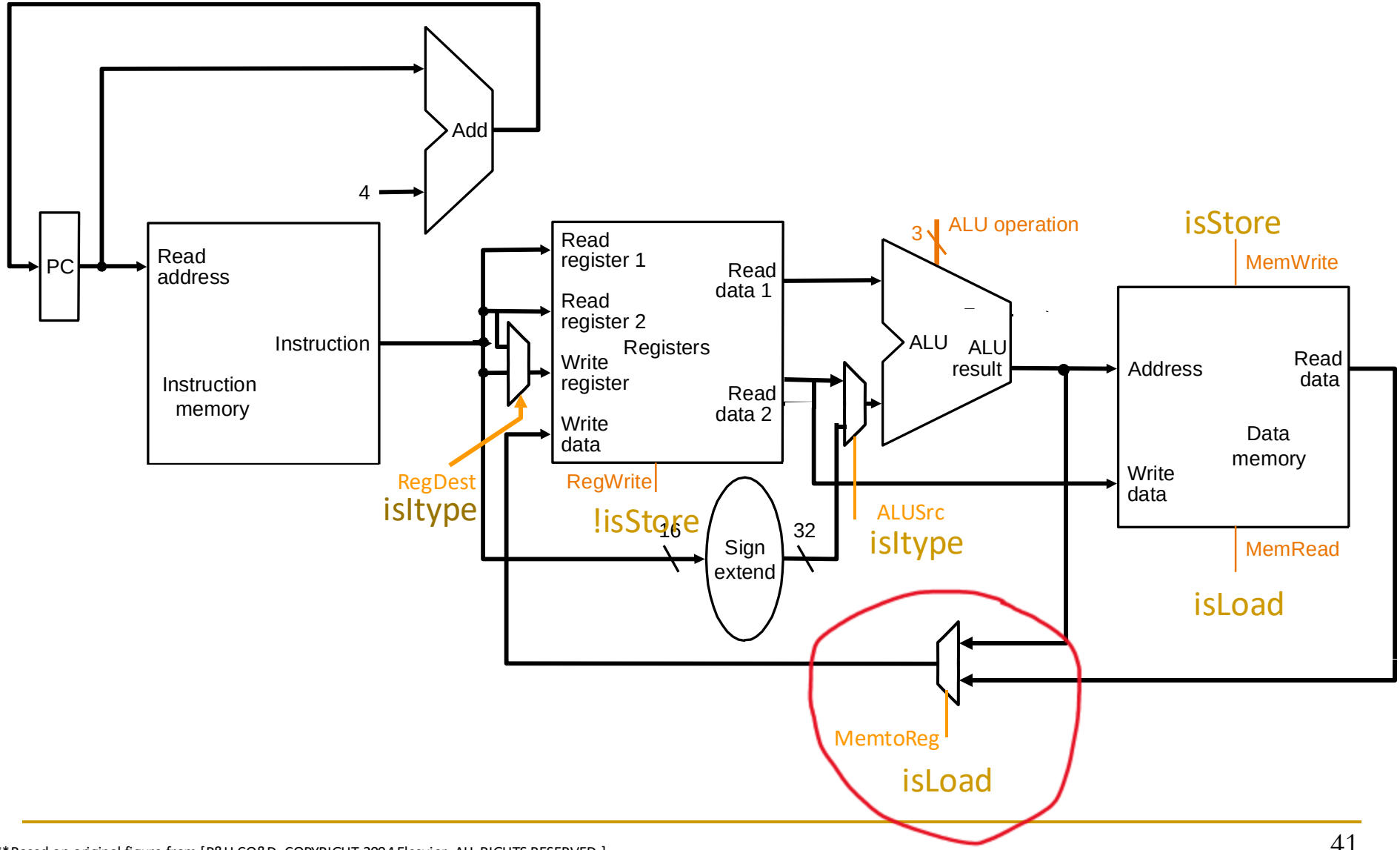
Combinational state update logic

Load-Store Datapath



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Datapath for Non-Control-Flow Insts.



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Single-Cycle Datapath for *Control Flow Instructions*

Jump Instruction

- Unconditional branch or jump

```
j   target
```

j (2)	immediate
6 bits	26 bits

J-Type

  - 2 = opcode
  - immediate (target) = target address

- Semantics

if MEM[PC]== j $immediate_{26}$

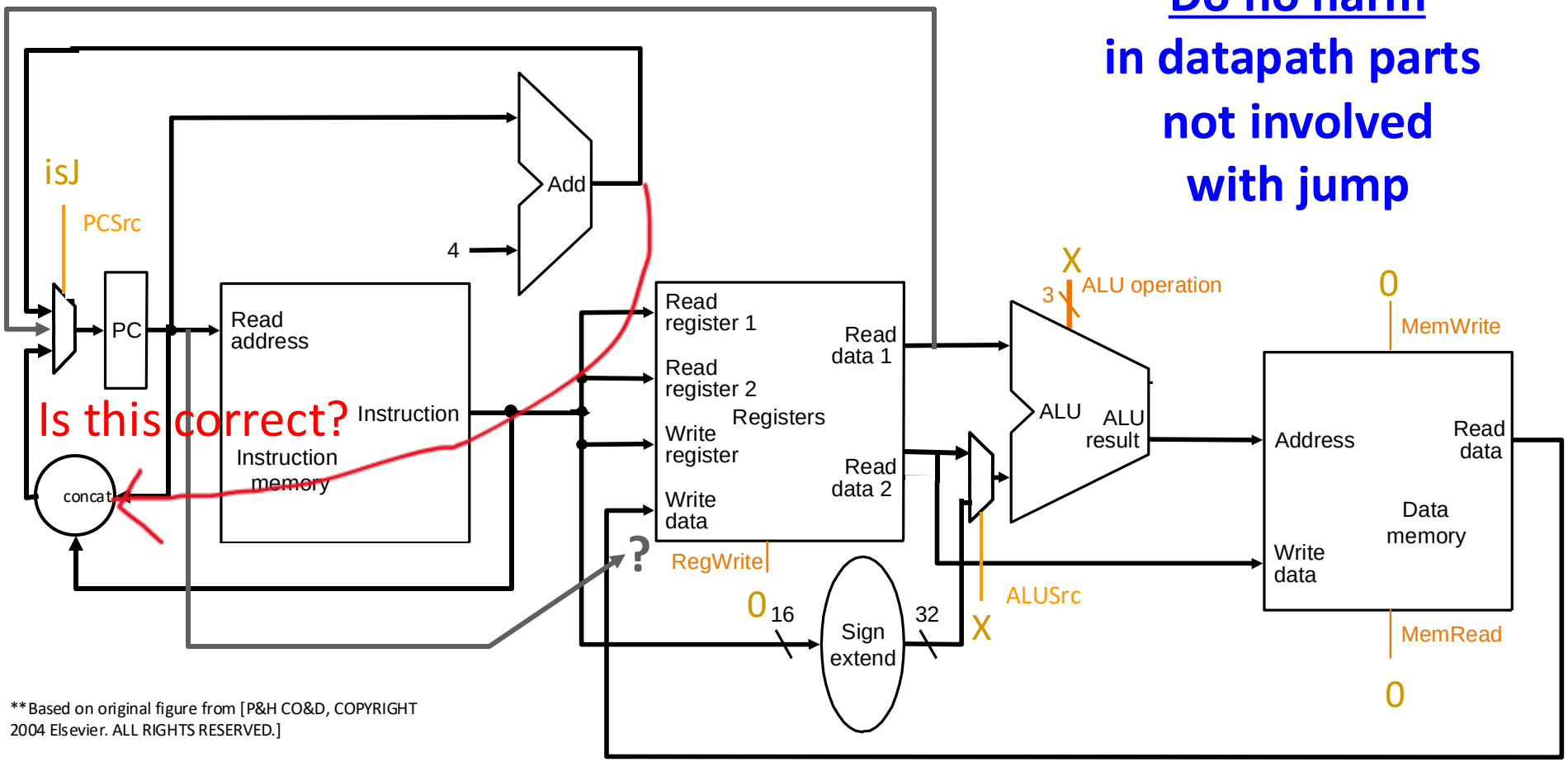
target = { PC † [31:28], $immediate_{26}$, 2' b00 }

PC ← target

† This is the incremented PC

Unconditional Jump Datapath

Do no harm in datapath parts not involved with jump



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

if MEM[PC]==J immediate26
PC = { PC † [31:28], immediate26, 2'b00 }

What about JR, JAL, JALR?

Other Jumps in MIPS

- jr: jump register

  Semantics

  if MEM[PC]== jr rs

  PC ← GPR(rs)

- jal: jump and link (function calls)

  Semantics

  if MEM[PC]== jal $immediate_{26}$

  $ra ← PC + 4

  target = { PC † [31:28], $immediate_{26}$, 2'b00 }

  PC ← target

- jalr: jump and link register

  Semantics

  if MEM[PC]== jalr rs

  $ra ← PC + 4

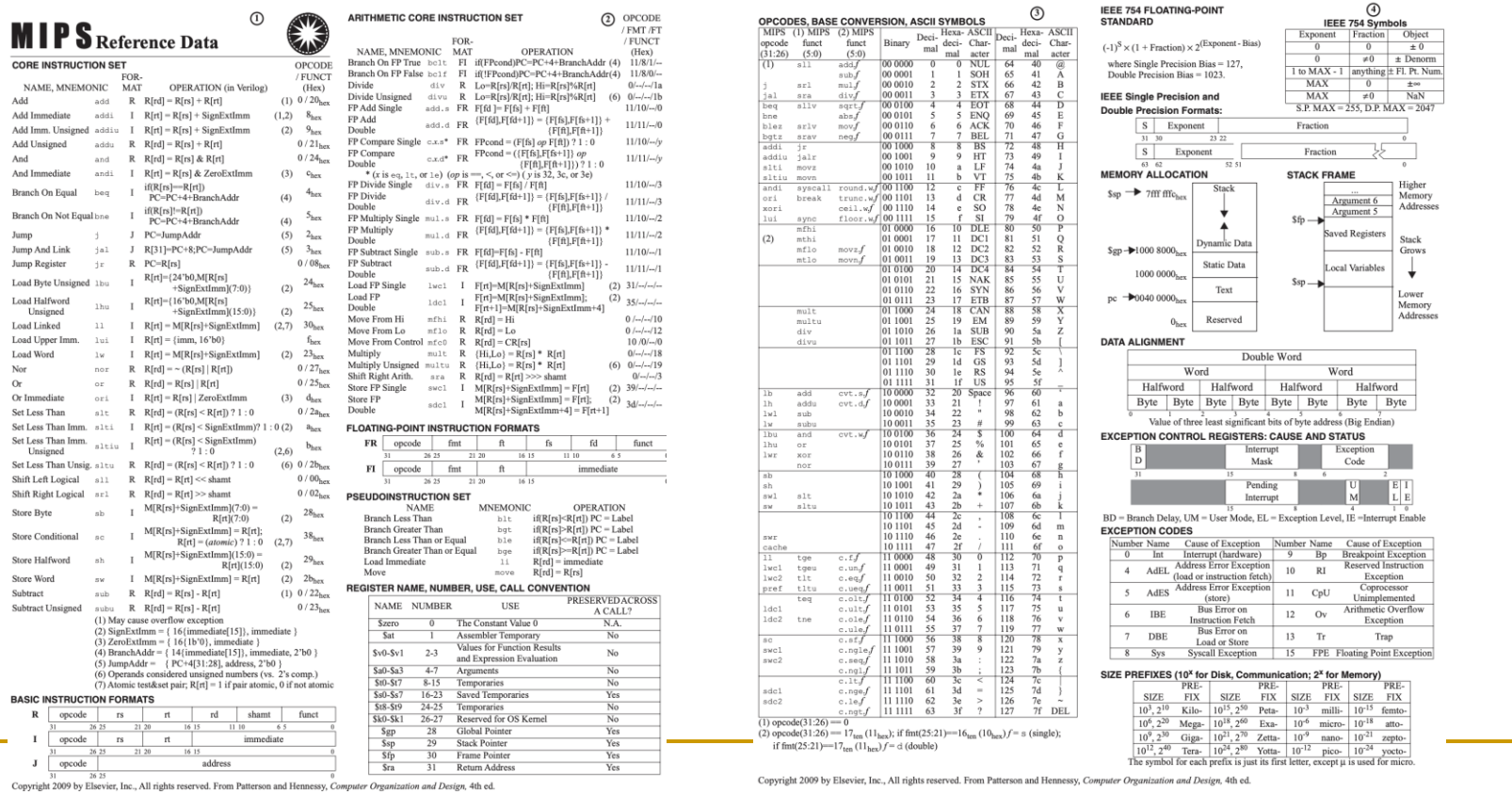
  PC ← GPR(rs)

† This is the incremented PC

Aside: MIPS Cheat Sheet

- https://safari.ethz.ch/digitaltechnik/spring2023/lib/exe/fetch.php?media=mips_reference_data.pdf

- On the course website



MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1) 0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2) 8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2) 9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]	0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]	0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3) c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4) 4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4) 5_{hex}
Jump	j	J	PC=JumpAddr	(5) 2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5) 3_{hex}
Jump Register	jr	R	PC=R[rs]	0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2) 24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2) 25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7) 30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}	f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2) 23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])	0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]	0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3) d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2) a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6) b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6) 0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt	0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt	0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2) 28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (*atomic*) ? 1 : 0	(2,7) 38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2) 29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2) $2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1) 0 / $2a_{hex}$
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]	0 / $2a_{hex}$

(1) May cause overflow exception
(2) SignExtImm = { 16{immediate[15]}, immediate }
(3) ZeroExtImm = { 16{1b'0}, immediate }
(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }
(6) Operands considered unsigned numbers (vs. 2's comp.)
(7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

	31–26	25–21	20–16	15–11	10–6	5–0
R	opcode	rs	rt	rd	shamt	funct
I	opcode	rs	rt	immediate		
J	opcode	address				

ARITHMETIC CORE INSTRUCTION SET ②

NAME, MNEMONIC		FOR-MAT	OPERATION	OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4) 11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6) 0/--/--/1b
FP Add Single	add.s	FR	F[fd] = F[fs] + F[ft]	11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}	11/11/--/0
FP Compare Single	c.x.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0	11/10/--/*y*
FP Compare Double	c.x.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0	11/11/--/*y*
* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)				
FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]	11/10/--/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}	11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]	11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}	11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]	11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}	11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2) 31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2) 35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi	0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo	0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]	10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]	0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6) 0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt	0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2) 39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2) 3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS

	31–26	25–21	20–16	15–11	10–6	5–0
FR	opcode	fmt	ft	fs	fd	funct
FI	opcode	fmt	ft	immediate		

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design*, 4th ed.

OPCODES, BASE CONVERSION, ASCII SYMBOLS ③

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexadecimal	ASCII Character	Decimal	Hexadecimal	ASCII Character
(1)	sll	add.*f*	00 0000	0	0	NUL	64	40	@
		sub.*f*	00 0001	1	1	SOH	65	41	A
j	srl	mul.*f*	00 0010	2	2	STX	66	42	B
jal	sra	div.*f*	00 0011	3	3	ETX	67	43	C
beq	sllv	sqrt.*f*	00 0100	4	4	EOT	68	44	D
bne		abs.*f*	00 0101	5	5	ENQ	69	45	E
blez	srlv	mov.*f*	00 0110	6	6	ACK	70	46	F
bgtz	srav	neg.*f*	00 0111	7	7	BEL	71	47	G
addi	jr		00 1000	8	8	BS	72	48	H
addiu	jalr		00 1001	9	9	HT	73	49	I
slti	movz		00 1010	10	a	LF	74	4a	J
sltiu	movn		00 1011	11	b	VT	75	4b	K
andi	syscall	round.*w f*	00 1100	12	c	FF	76	4c	L
ori	break	trunc.*w f*	00 1101	13	d	CR	77	4d	M
xori		ceil.*w f*	00 1110	14	e	SO	78	4e	N
lui	sync	floor.*w f*	00 1111	15	f	SI	79	4f	O
(2)	mfhi		01 0000	16	10	DLE	80	50	P
	mthi		01 0001	17	11	DC1	81	51	Q
	mflo	movz.*f*	01 0010	18	12	DC2	82	52	R
	mtlo	movn.*f*	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	]
			01 1110	30	1e	RS	94	5e	^
			01 1111	31	1f	US	95	5f	_
lb	add	cvt.s.*f*	10 0000	32	20	Space	96	60	`
lh	addu	cvt.d.*f*	10 0001	33	21	!	97	61	a
lwl	sub		10 0010	34	22	"	98	62	b
lw	subu		10 0011	35	23	#	99	63	c
lbu	and	cvt.*w f*	10 0100	36	24	$	100	64	d
lhu	or		10 0101	37	25	%	101	65	e
lwr	xor		10 0110	38	26	&	102	66	f
	nor		10 0111	39	27	'	103	67	g
sb			10 1000	40	28	(	104	68	h
sh			10 1001	41	29	)	105	69	i
swl	slt		10 1010	42	2a	*	106	6a	j
sw	sltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
swr			10 1110	46	2e	.	110	6e	n
cache			10 1111	47	2f	/	111	6f	o
ll	tge	c.*f f*	11 0000	48	30	0	112	70	p
lwc1	tgeu	c.un.*f*	11 0001	49	31	1	113	71	q
lwc2	tlt	c.eq.*f*	11 0010	50	32	2	114	72	r
pref	tltu	c.ueq.*f*	11 0011	51	33	3	115	73	s
	teq	c.olt.*f*	11 0100	52	34	4	116	74	t
ldc1		c.ult.*f*	11 0101	53	35	5	117	75	u
ldc2	tne	c.ole.*f*	11 0110	54	36	6	118	76	v
		c.ule.*f*	11 0111	55	37	7	119	77	w
sc		c.sf.*f*	11 1000	56	38	8	120	78	x
swc1		c.ngle.*f*	11 1001	57	39	9	121	79	y
swc2		c.seq.*f*	11 1010	58	3a	:	122	7a	z
		c.ngl.*f*	11 1011	59	3b	;	123	7b	{
		c.lt.*f*	11 1100	60	3c	<	124	7c	|
sdc1		c.nge.*f*	11 1101	61	3d	=	125	7d	}
sdc2		c.le.*f*	11 1110	62	3e	>	126	7e	~
		c.ngt.*f*	11 1111	63	3f	?	127	7f	DEL

(1) opcode(31:26) == 0
(2) opcode(31:26) == 17_{ten} (11_{hex}); if fmt(25:21)==16_{ten} (10_{hex}) *f* = s (single); if fmt(25:21)==17_{ten} (11_{hex}) *f* = d (double)

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design*, 4th ed.

IEEE 754 FLOATING-POINT STANDARD ④

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$

where Single Precision Bias = 127, Double Precision Bias = 1023.

IEEE 754 Symbols

Exponent	Fraction	Object
0	0	± 0
0	≠0	± Denorm
1 to MAX - 1	anything	± Fl. Pt. Num.
MAX	0	±∞
MAX	≠0	NaN

S.P. MAX = 255, D.P. MAX = 2047

IEEE Single Precision and Double Precision Formats:

	S	Exponent	Fraction
Single	1 bit (31)	8 bits (30–23)	23 bits (22–0)
Double	1 bit (63)	11 bits (62–52)	20 bits (51–0)

MEMORY ALLOCATION

$sp → 7fff ffc0$hex

$gp → 1000 8000$hex

1000 0000hex

pc → 0040 0000hex

0hex

Stack ↓ / ↑ Dynamic Data / Static Data / Text / Reserved

STACK FRAME

$fp → ...; Argument 6; Argument 5; Saved Registers; Local Variables; $sp

Higher Memory Addresses

Stack Grows ↓

Lower Memory Addresses

DATA ALIGNMENT

Double Word							
Word				Word			
Halfword		Halfword		Halfword		Halfword	
Byte	Byte	Byte	Byte	Byte	Byte	Byte	Byte
0	1	2	3	4	5	6	7

Value of three least significant bits of byte address (Big Endian)

EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS

	31–15	15–8	6–2
Cause	BD	Interrupt Mask	Exception Code
Status	Pending Interrupt	UM, EL, IE	

BD = Branch Delay, UM = User Mode, EL = Exception Level, IE =Interrupt Enable

EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RI	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10^x for Disk, Communication; 2^x for Memory)

SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX
$10^3, 2^{10}$	Kilo-	$10^{15}, 2^{50}$	Peta-	10^{-3}	milli-	10^{-15}	femto-
$10^6, 2^{20}$	Mega-	$10^{18}, 2^{60}$	Exa-	10^{-6}	micro-	10^{-18}	atto-
$10^9, 2^{30}$	Giga-	$10^{21}, 2^{70}$	Zetta-	10^{-9}	nano-	10^{-21}	zepto-
$10^{12}, 2^{40}$	Tera-	$10^{24}, 2^{80}$	Yotta-	10^{-12}	pico-	10^{-24}	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

Conditional Branch Instructions

- beq (Branch if Equal)

```
beq  $s0, $s1, offset                    #$s0=rs,$s1=rt
```

beq (4)	rs	rt	immediate=offset
6 bits	5 bits	5 bits	16 bits

I-Type

- Semantics (assuming no branch delay slot)

  if MEM[PC] == beq rs rt $immediate_{16}$

  target = PC† + sign-extend(immediate) x 4

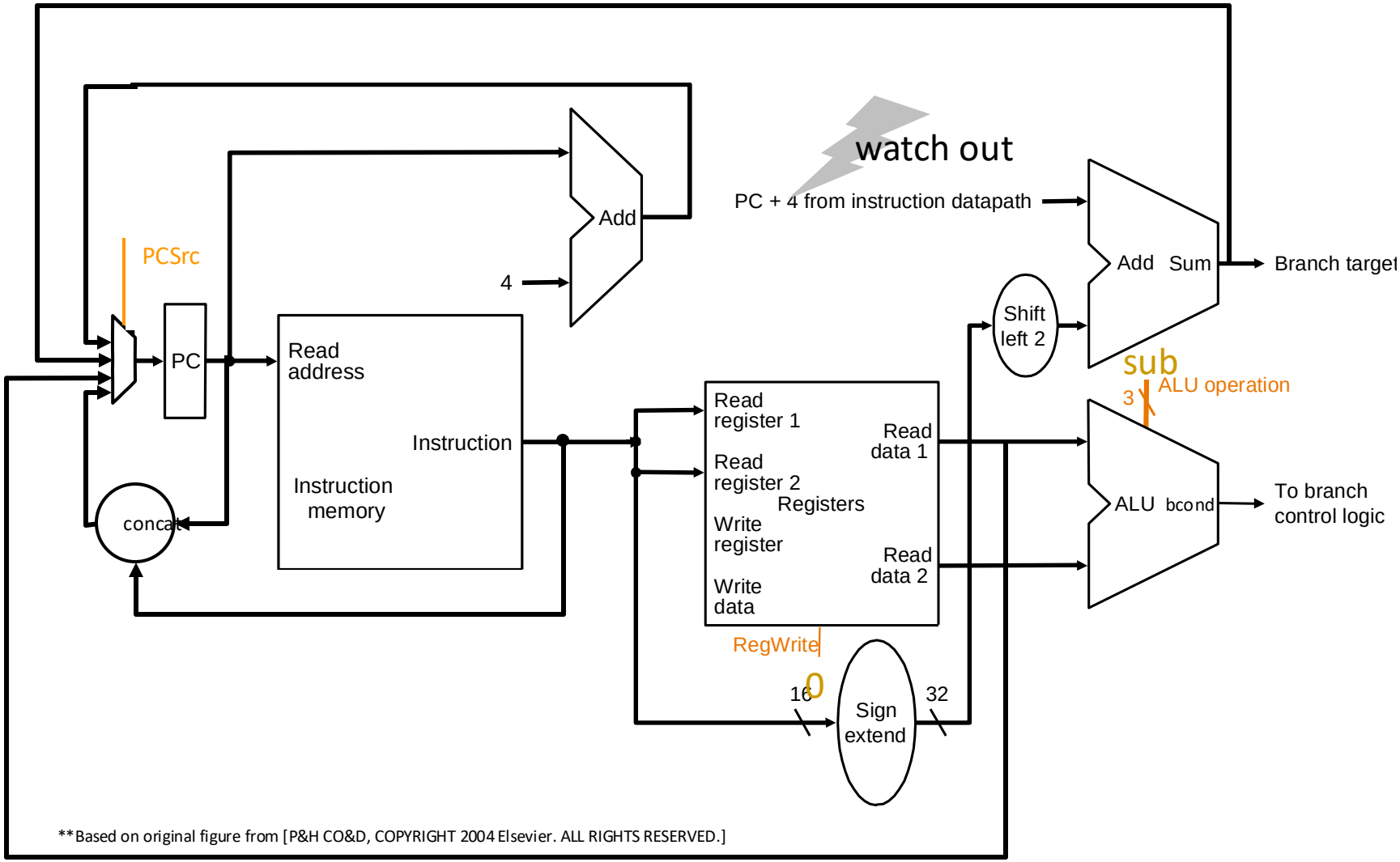
  if GPR[rs]==GPR[rt] then PC ← target

  else PC ← PC + 4

  - Variations: beq, bne, blez, bgtz

† This is the incremented PC

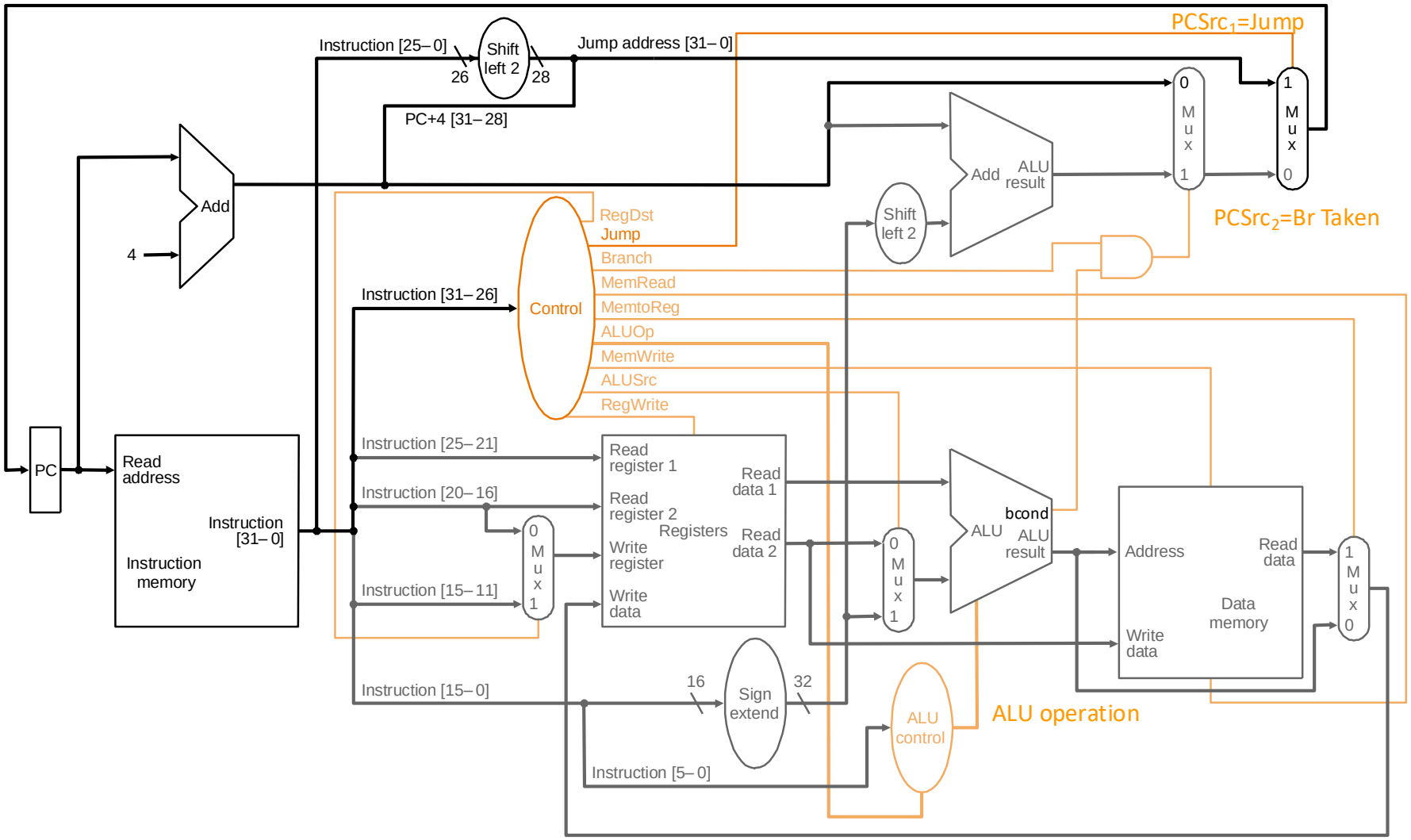
Conditional Branch Datapath (for you to finish)



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Foreshadowing: How to uphold the delayed branch semantics?

Putting It All Together



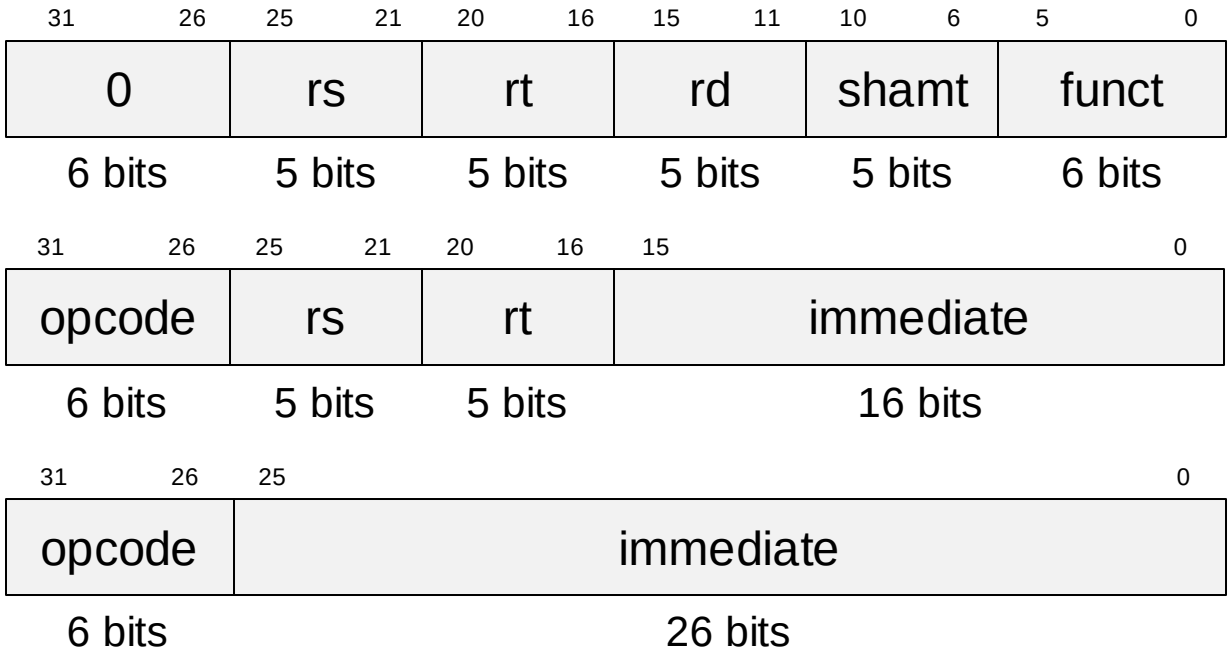
**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

JAL, JR, JALR omitted

Single-Cycle Control Logic

Single-Cycle Hardwired Control

- **As combinational function** of Inst=MEM[PC]



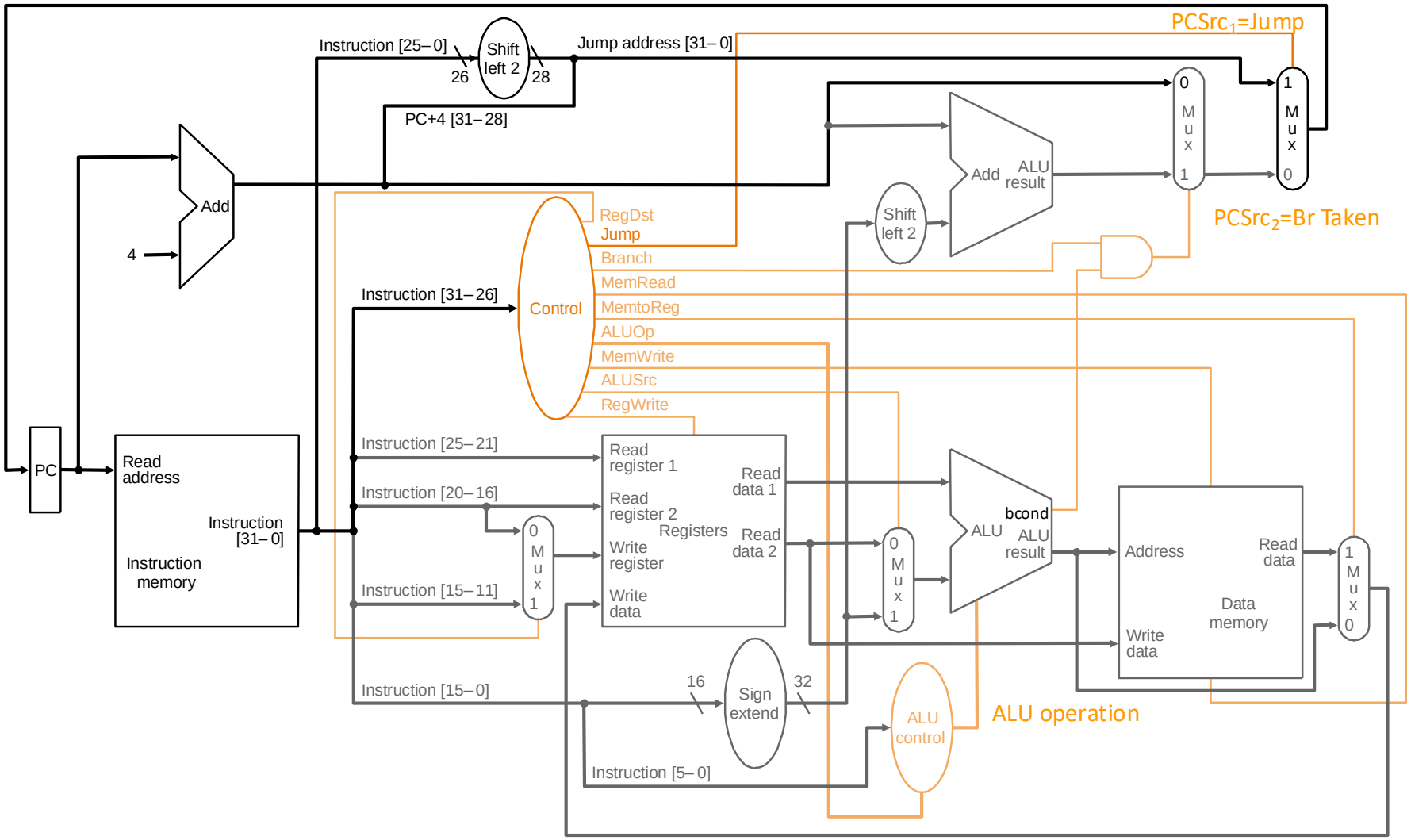
R-Type

I-Type

J-Type

- Consider
  - All R-type and I-type ALU instructions
  - lw and sw
  - beq, bne, blez, bgtz
  - j, jr, jal, jalr omitted

Generate Control Signals (in Orange Color)



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

JAL, JR, JALR omitted

Single-Bit Control Signals (I)

	When De-asserted	When asserted	Equation
RegDst	GPR write select according to **rt**, i.e., inst[20:16]	GPR write select according to **rd**, i.e., inst[15:11]	opcode==0
ALUSrc	2nd ALU input from 2nd **GPR** read port	2nd ALU input from sign-extended 16-bit **immediate**	(opcode!=0) && (opcode!=BEQ) && (opcode!=BNE)
MemtoReg	Steer **ALU result** to GPR write port	Steer **memory output** to GPR write port	opcode==LW
RegWrite	GPR **write disabled**	GPR **write enabled**	(opcode!=SW) && (opcode!=Bxx) && (opcode!=J) && (opcode!=JR))

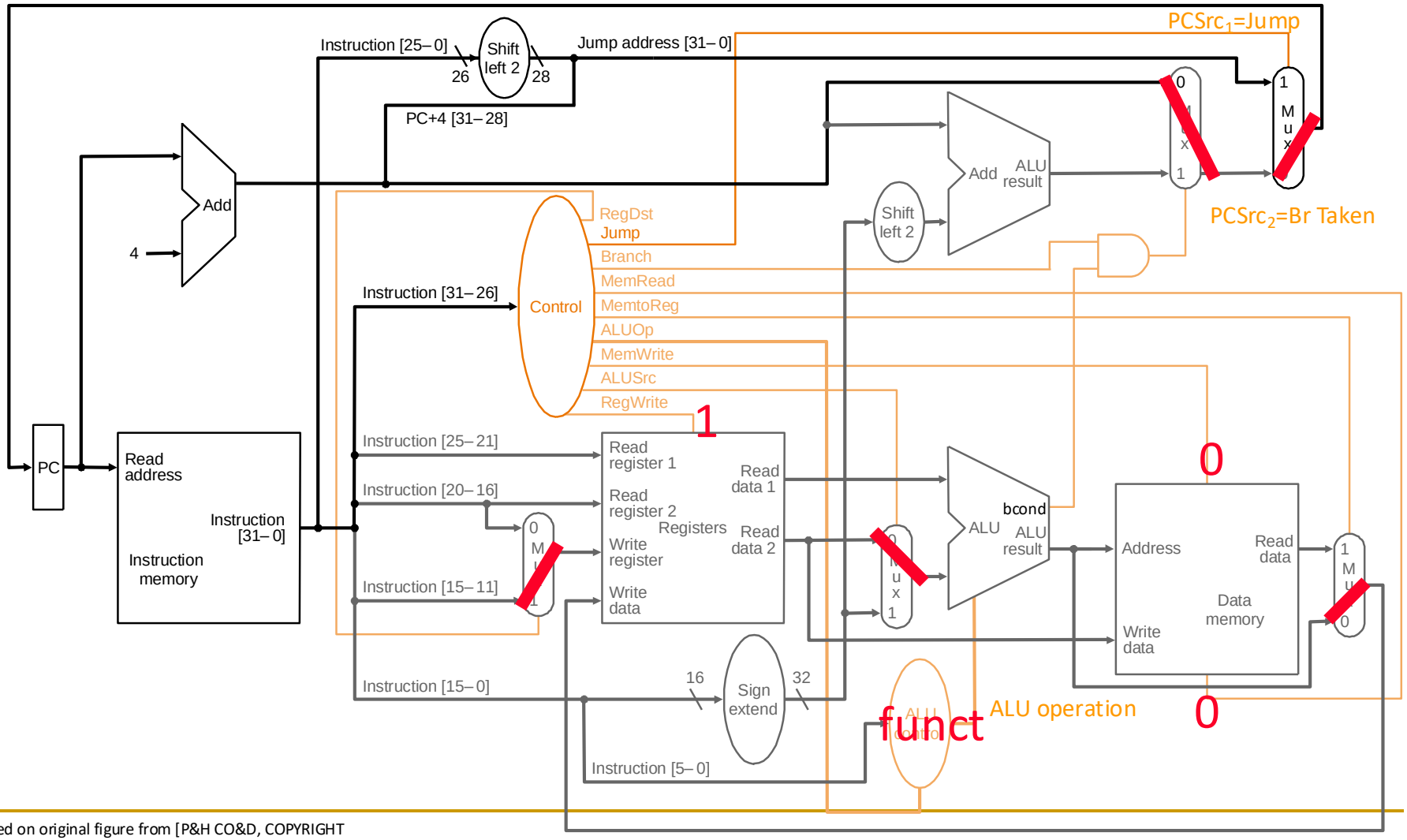
JAL and JALR require additional RegDst and MemtoReg options

Single-Bit Control Signals (II)

	When De-asserted	When asserted	Equation
MemRead	Memory **read disabled**	Memory **read port returns load value**	opcode==LW
MemWrite	Memory **write disabled**	Memory **write enabled**	opcode==SW
$PCSrc_1$	According to **$PCSrc_2$**	next PC is based on **26-bit immediate jump target**	(opcode==J) || (opcode==JAL)
$PCSrc_2$	next PC = **PC + 4**	next PC is based on **16-bit immediate branch target**	(opcode==Bxx) && "bcond is satisfied"

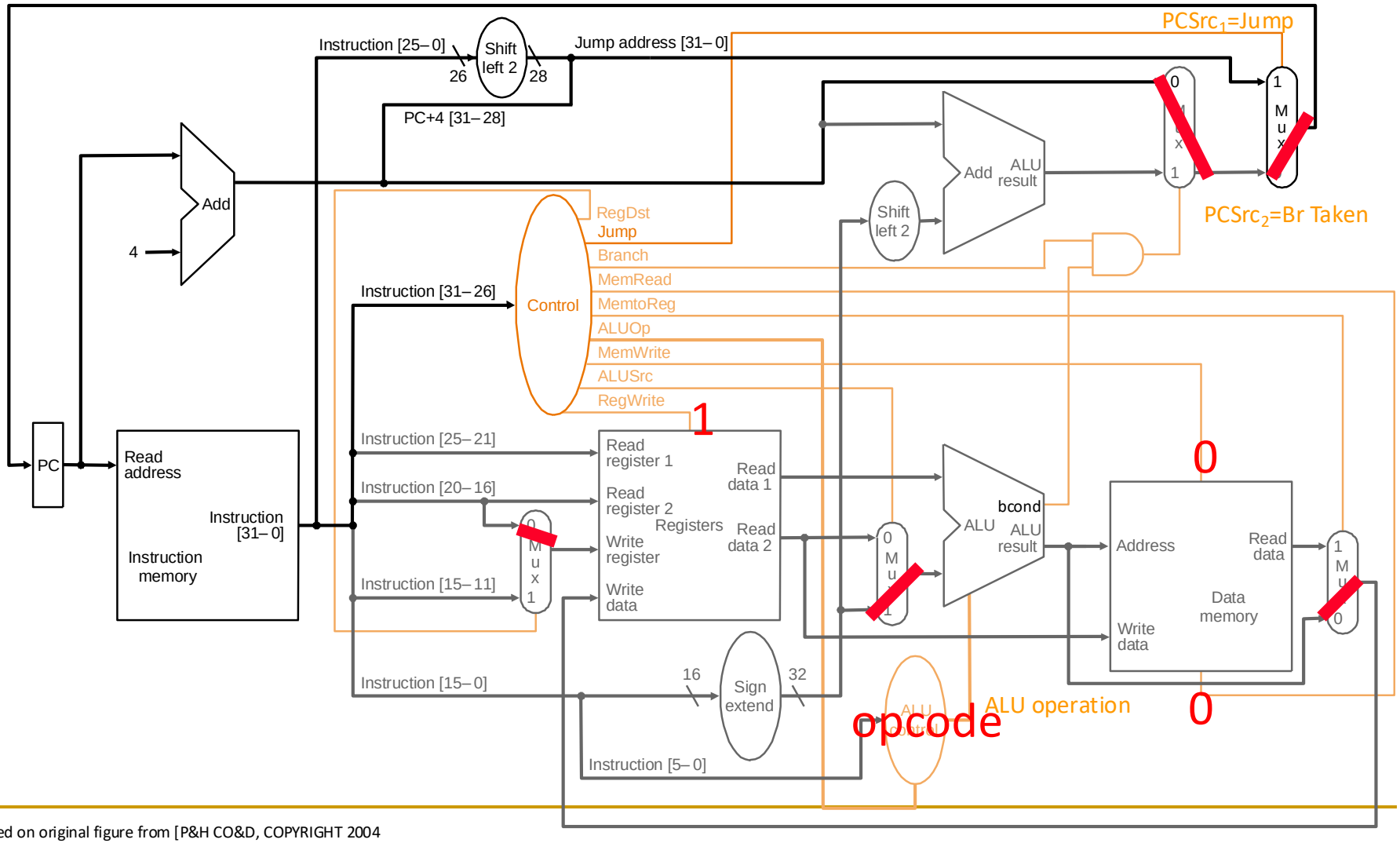
JR and JALR require additional PCSrc options

R-Type ALU



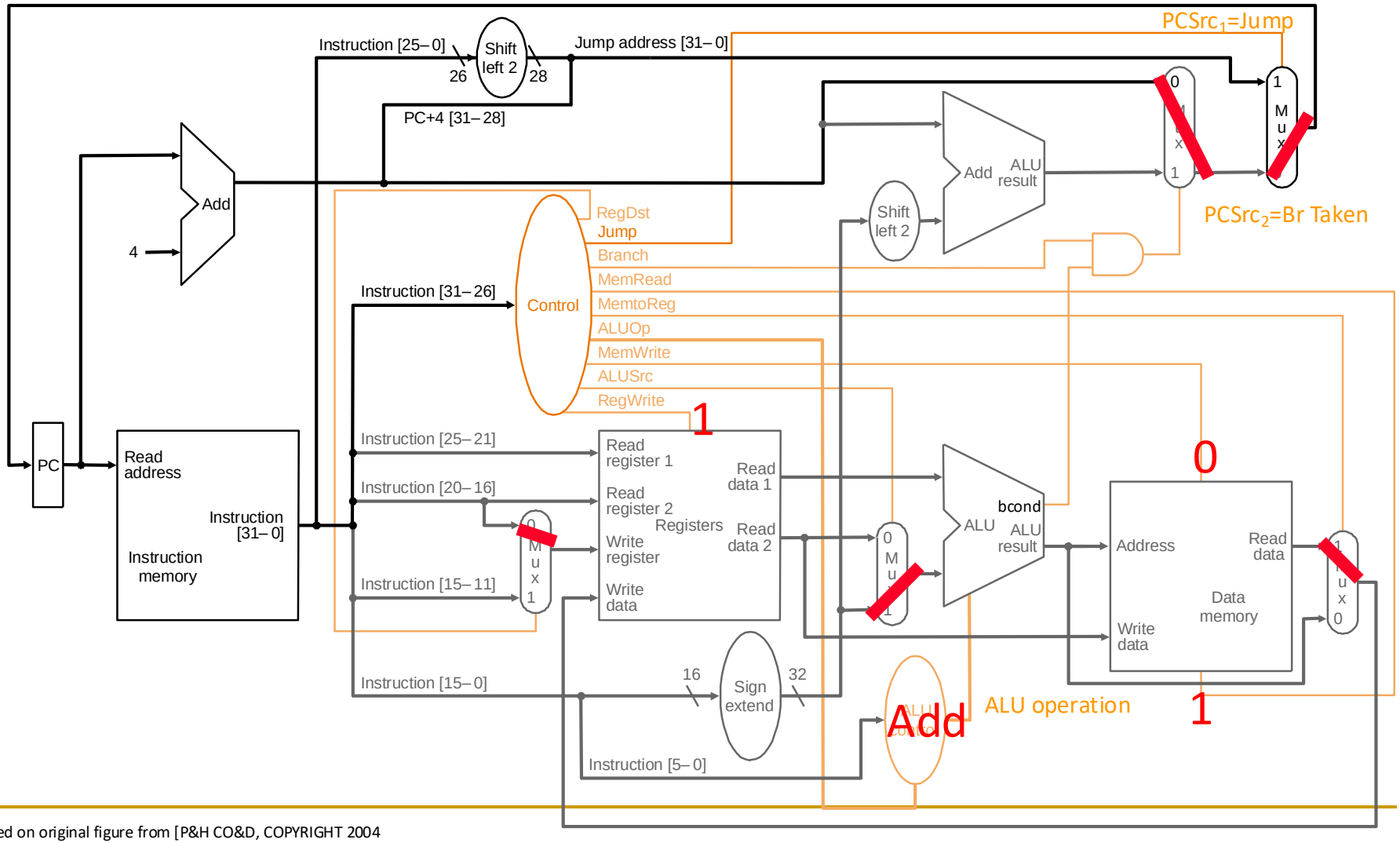
**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

I-Type ALU



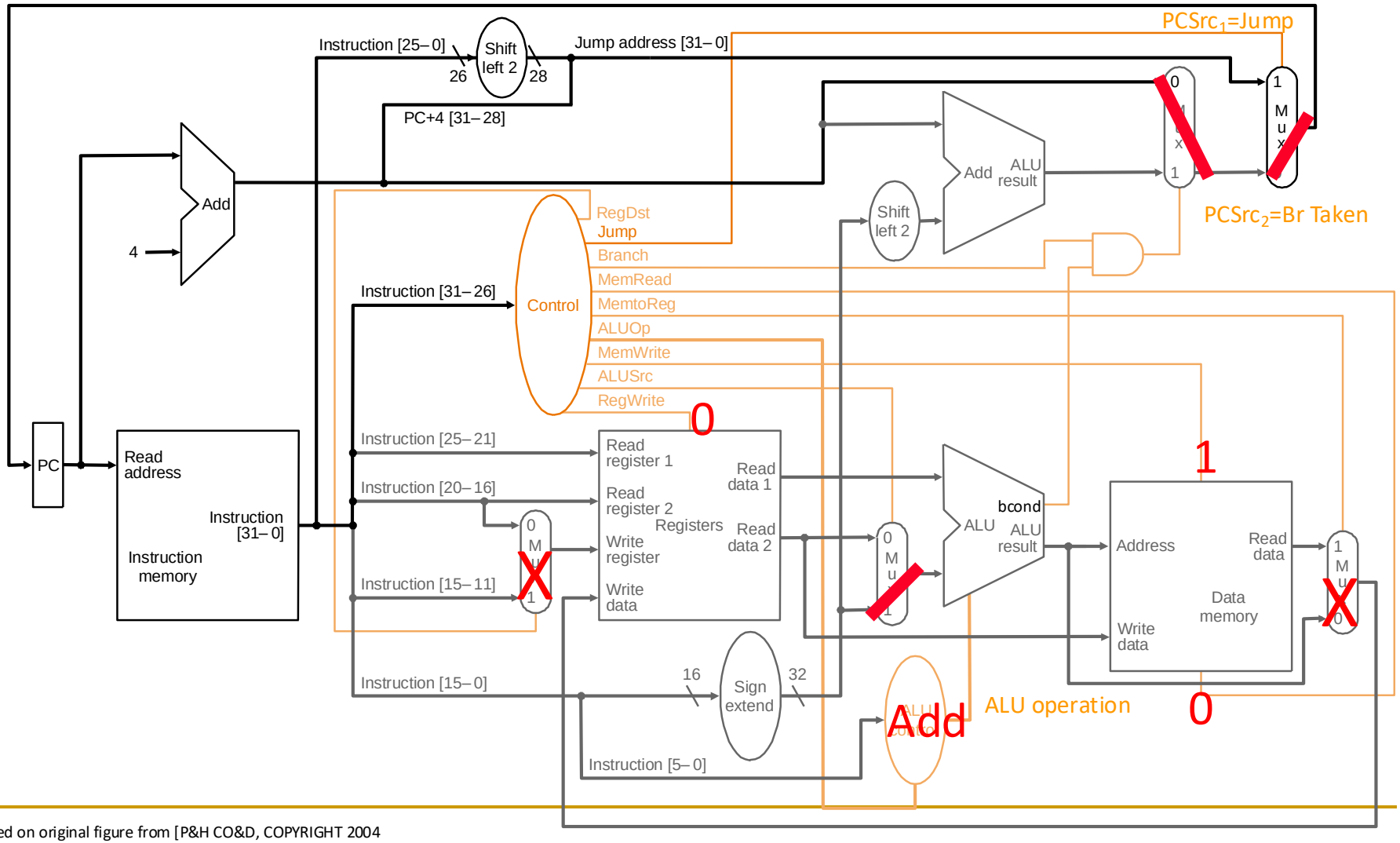
**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

LW



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

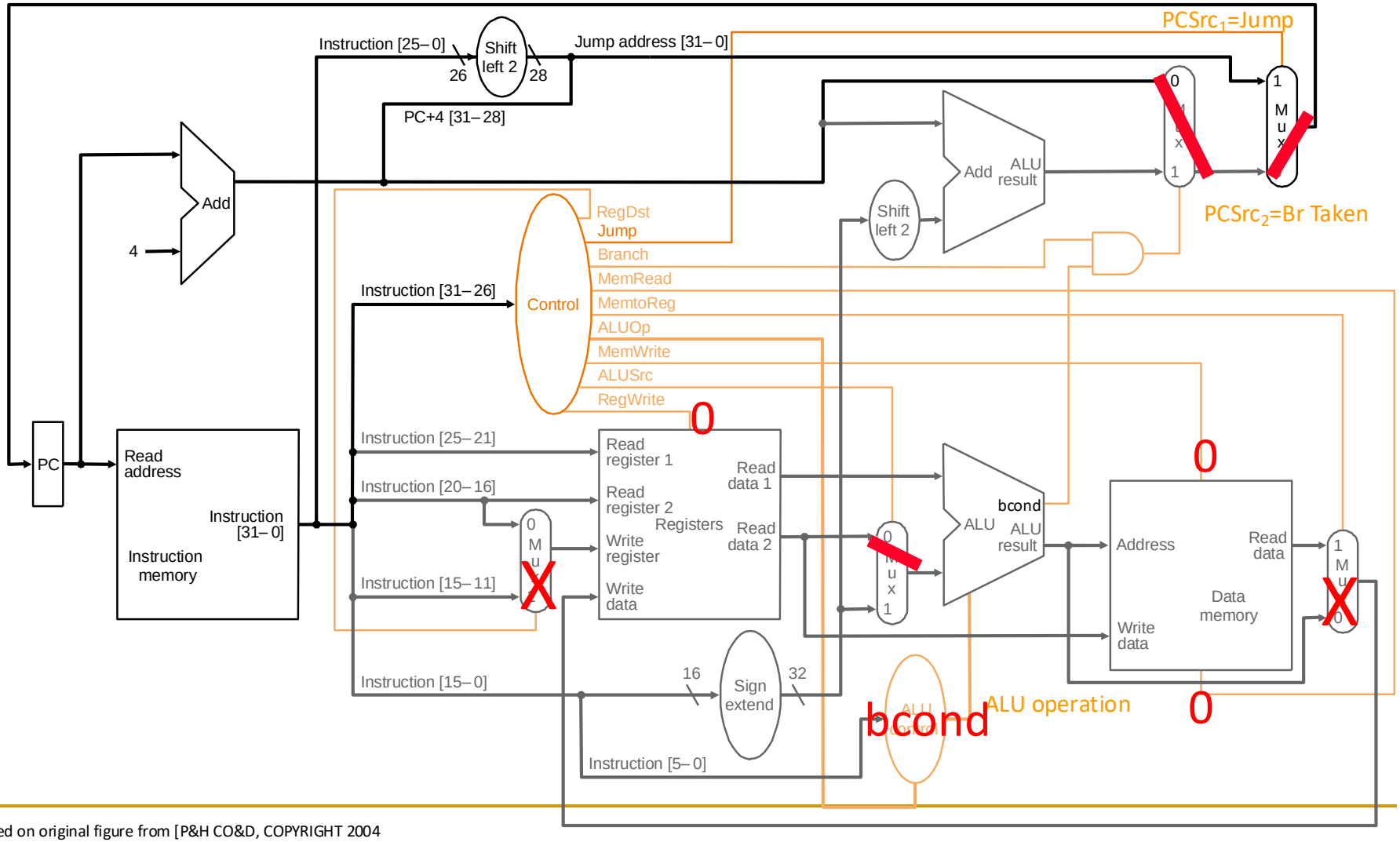
SW



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Branch (Not Taken)

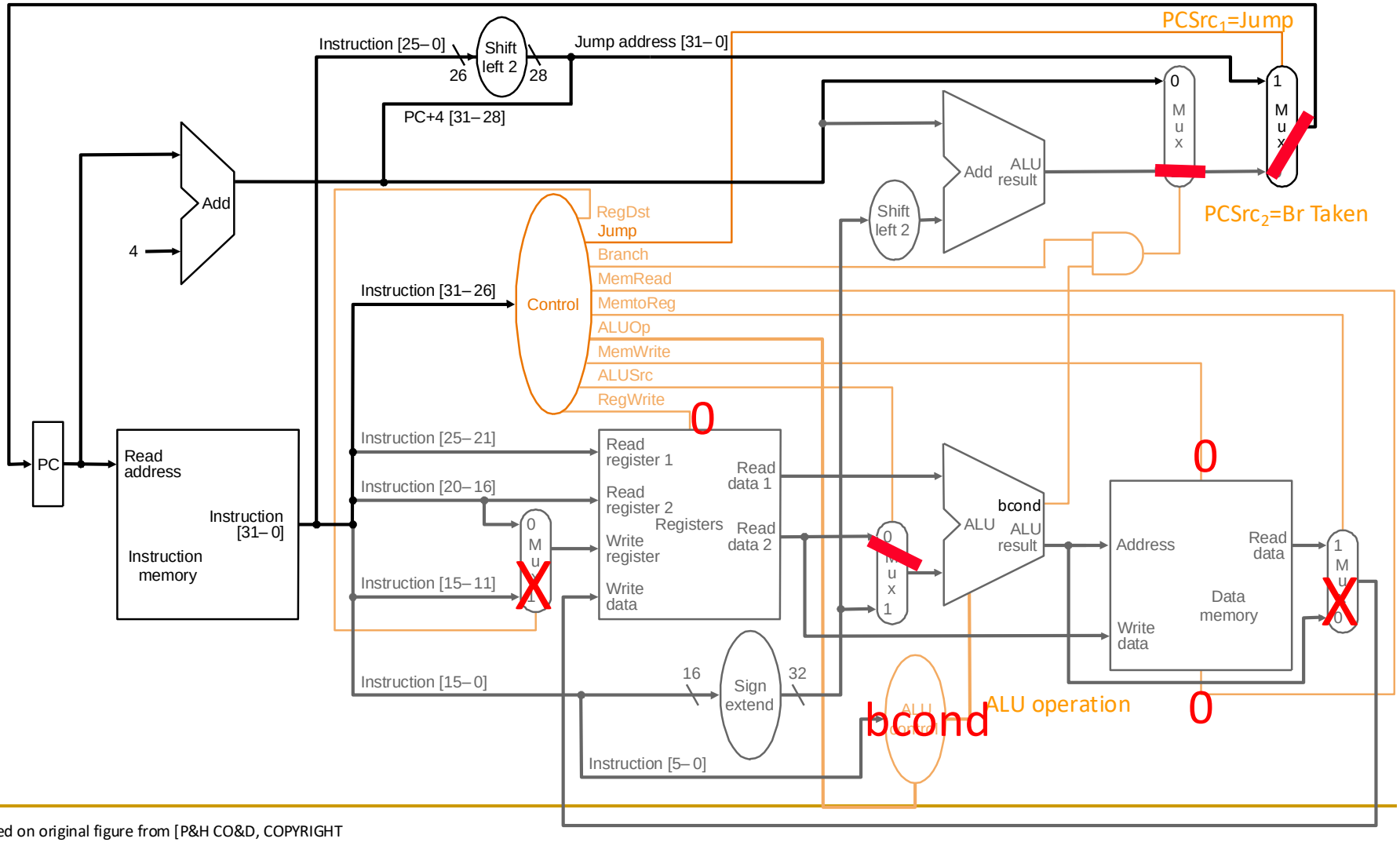
Some control signals are dependent on the processing of data



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

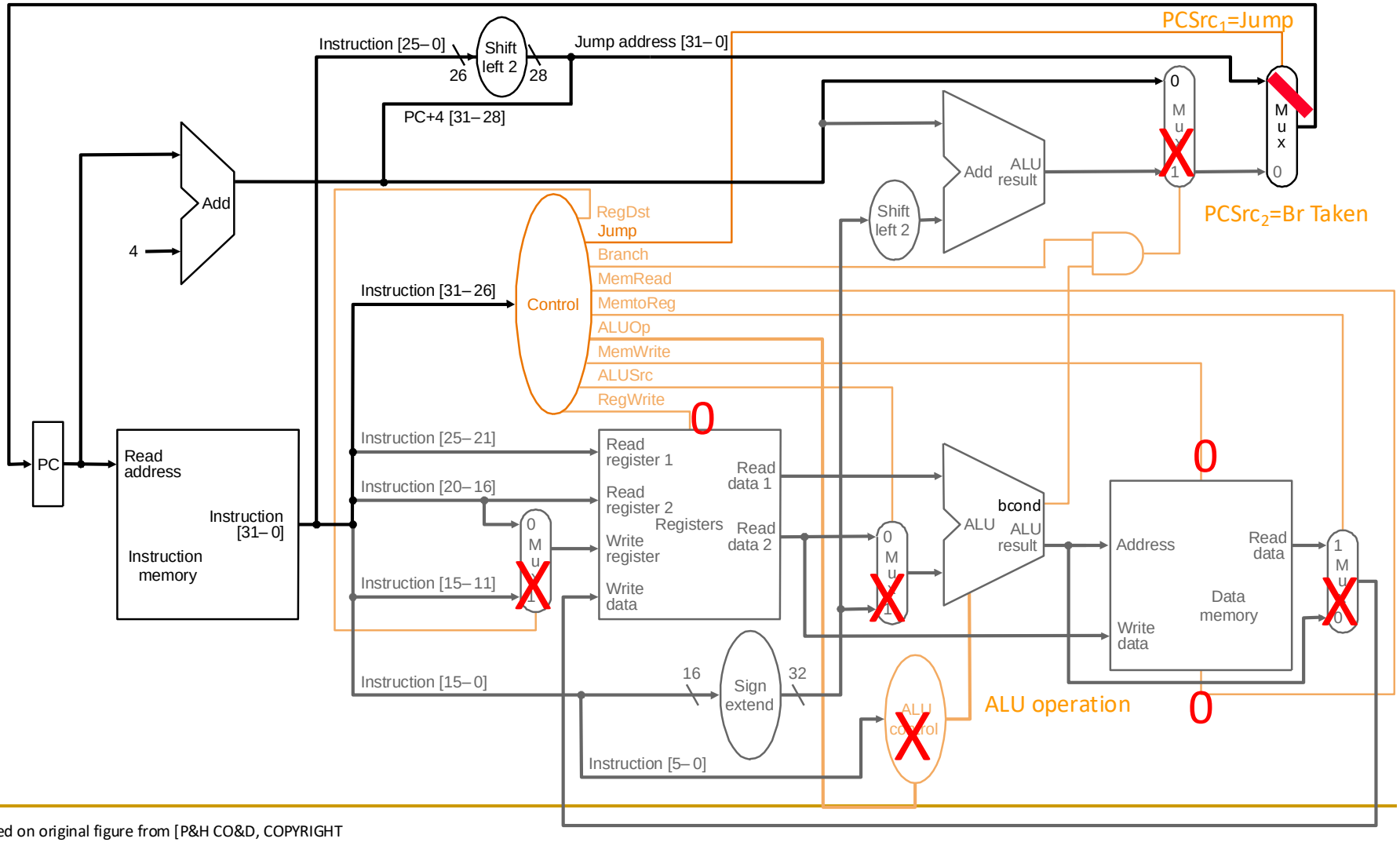
Branch (Taken)

Some control signals are dependent on the processing of data



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Jump



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

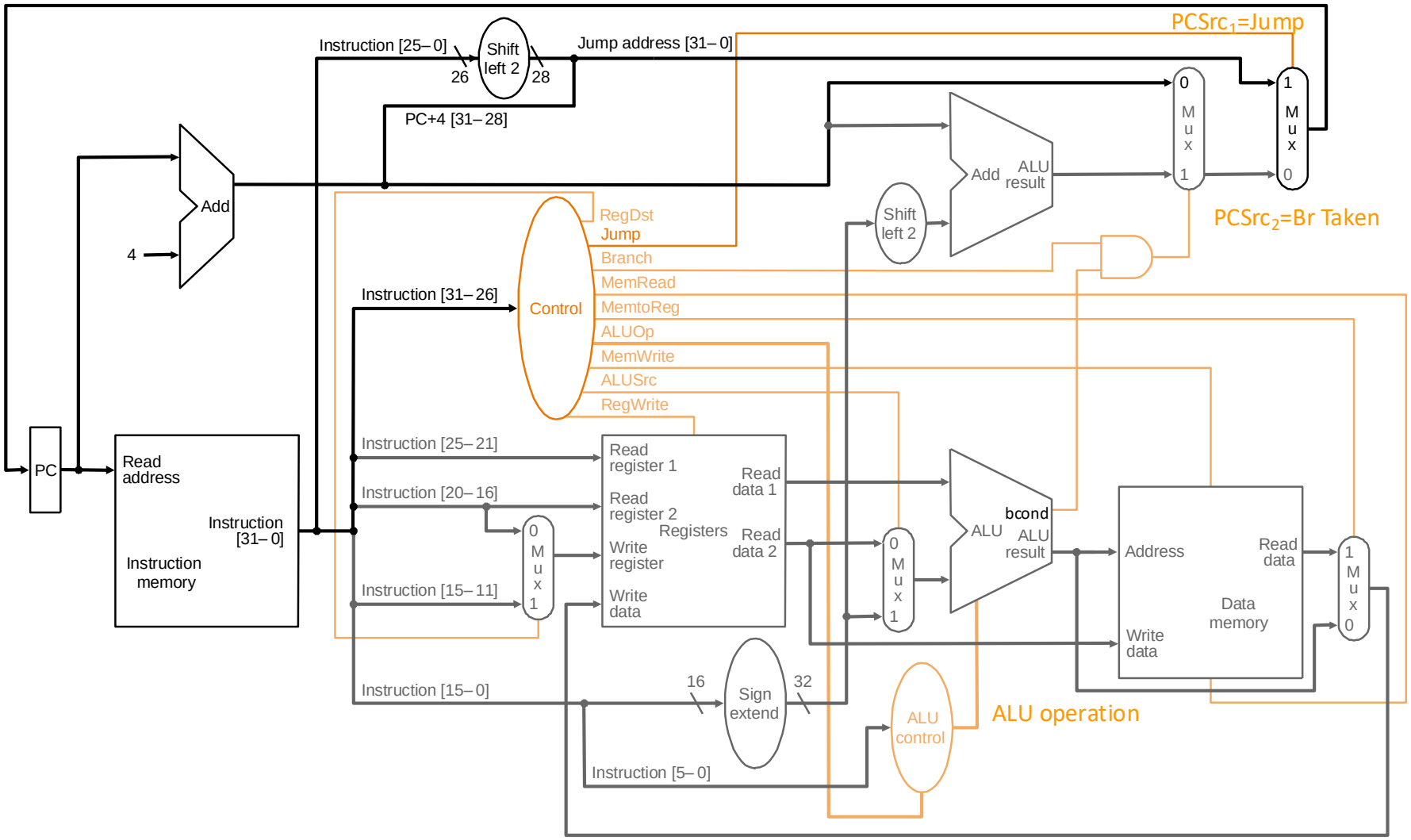
What is in That Control Box?

- Combinational Logic → Hardwired Control
  - Idea: Control signals generated combinationally based on bits in instruction encoding

- Sequential Logic → Sequential Control
  - Idea: A memory structure contains the control signals associated with an instruction
    - Called Control Store

- Both types of control structure can be used in single-cycle processors
  - Choice depends on latency of each structure + how much on the critical path control signal generation is, etc.

Review: Complete Single-Cycle Processor



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

JAL, JR, JALR omitted

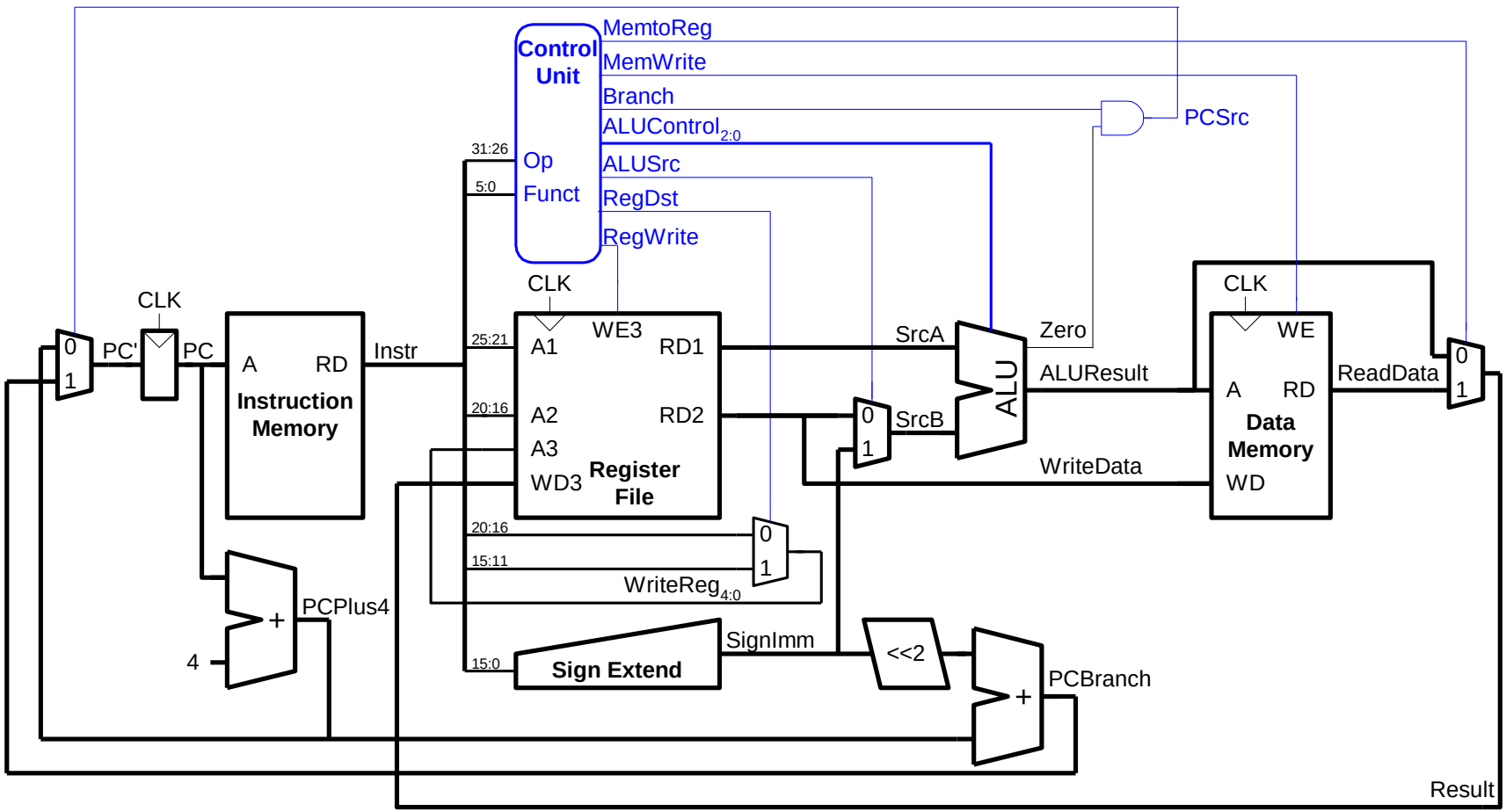
Another Single-Cycle MIPS Processor (from H&H)

See backup slides to reinforce the concepts we have covered.

They are to complement your reading:

H&H, Chapter 7.1-7.3, 7.6

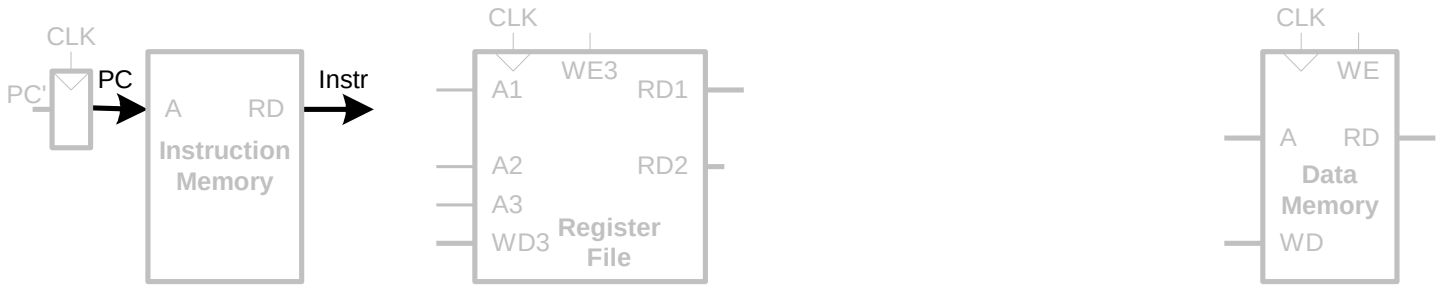
Another Complete Single-Cycle Processor



Single-cycle processor. Harris and Harris, Chapter 7.3.

Example: Single-Cycle Datapath: lw fetch

- ***STEP 1:*** **Fetch instruction**



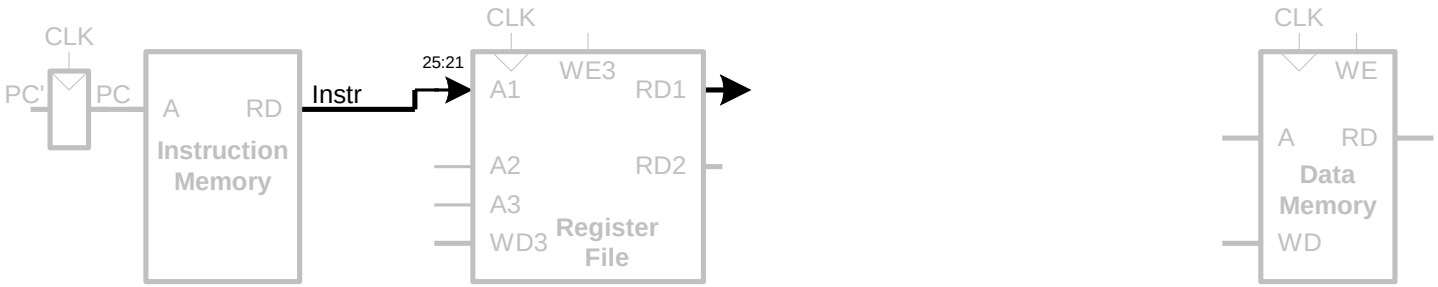

```
lw $s3, 1($0)          # read memory word 1 into $s3
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw register read

- ***STEP 2:*** **Read source operands from register file**



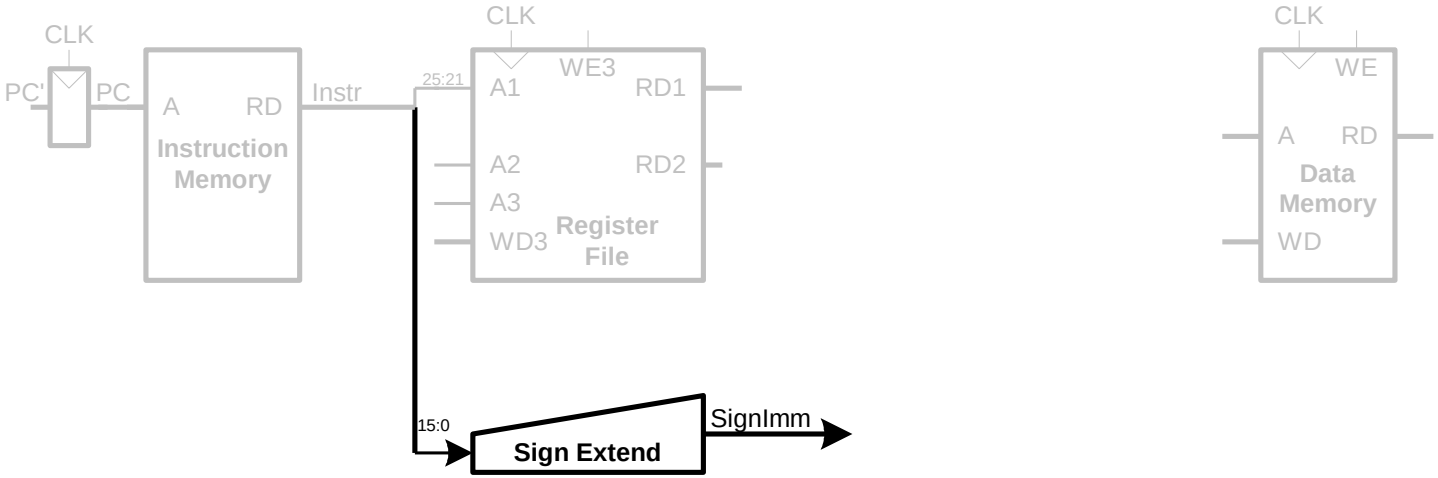

```
lw $s3, 1($0)            # read memory word 1 into $s3
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw immediate

- ***STEP 3:*** **Sign-extend the immediate**



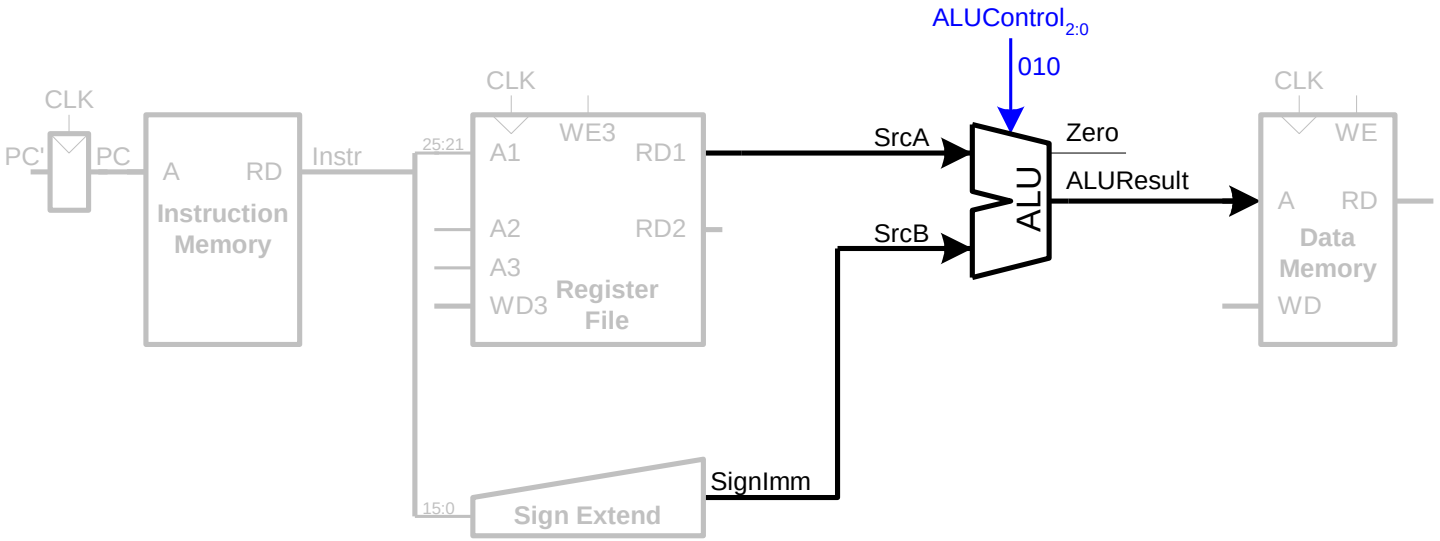

```
lw $s3, 1($0)          # read memory word 1 into $s3
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw address

- ***STEP 4:*** **Compute the memory address**



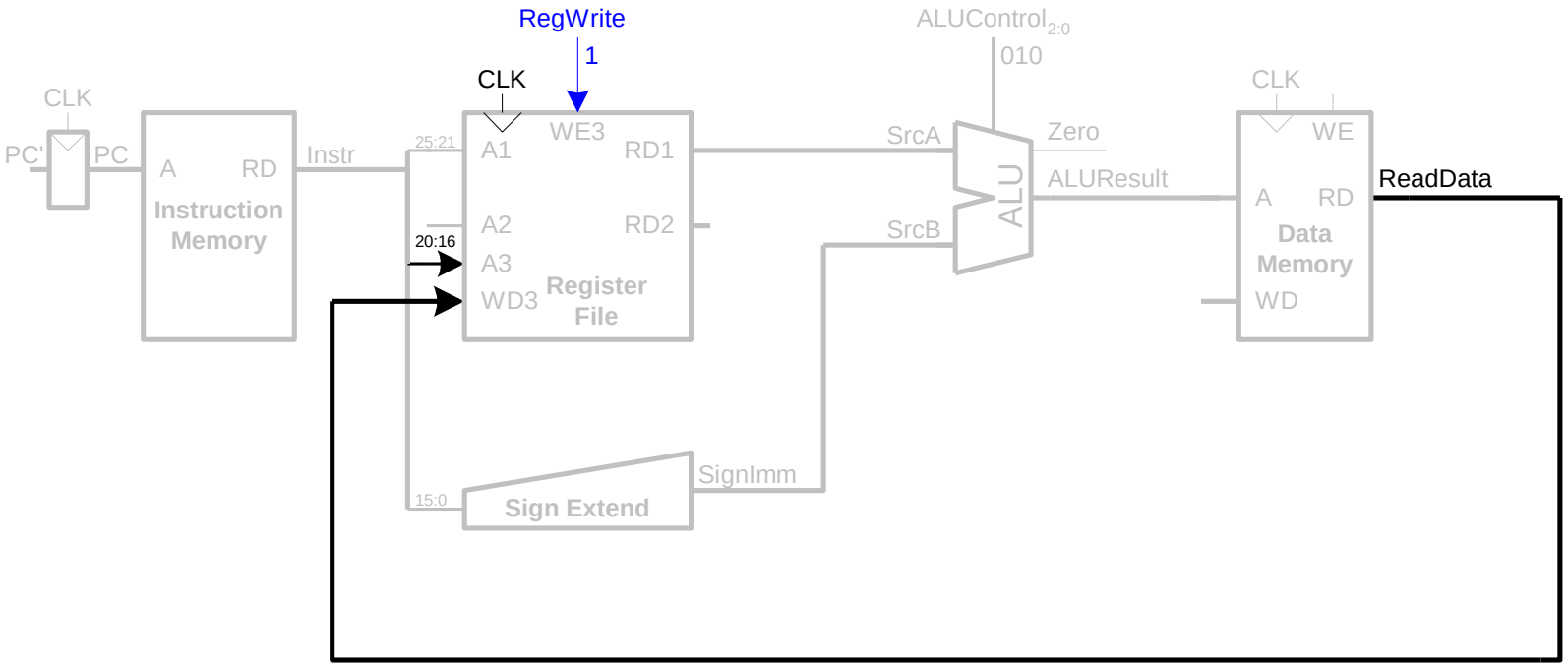

```
lw $s3, 1($0)          # read memory word 1 into $s3
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw memory read

- ***STEP 5:*** **Read from memory and write back to register file**



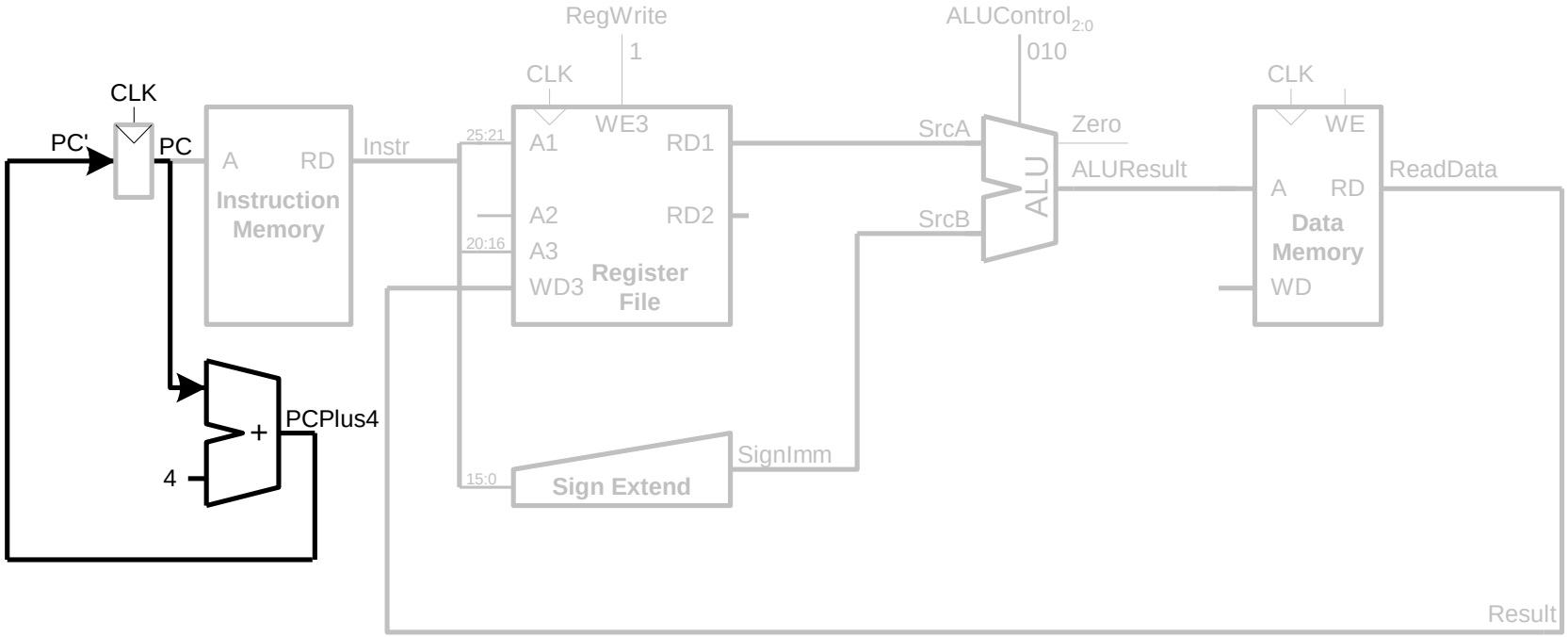

```
lw $s3, 1($0)          # read memory word 1 into $s3
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw PC increment

- ***STEP 6:*** **Determine address of next instruction**




```
lw $s3, 1($0)              # read memory word 1 into $s3
```

I-Type

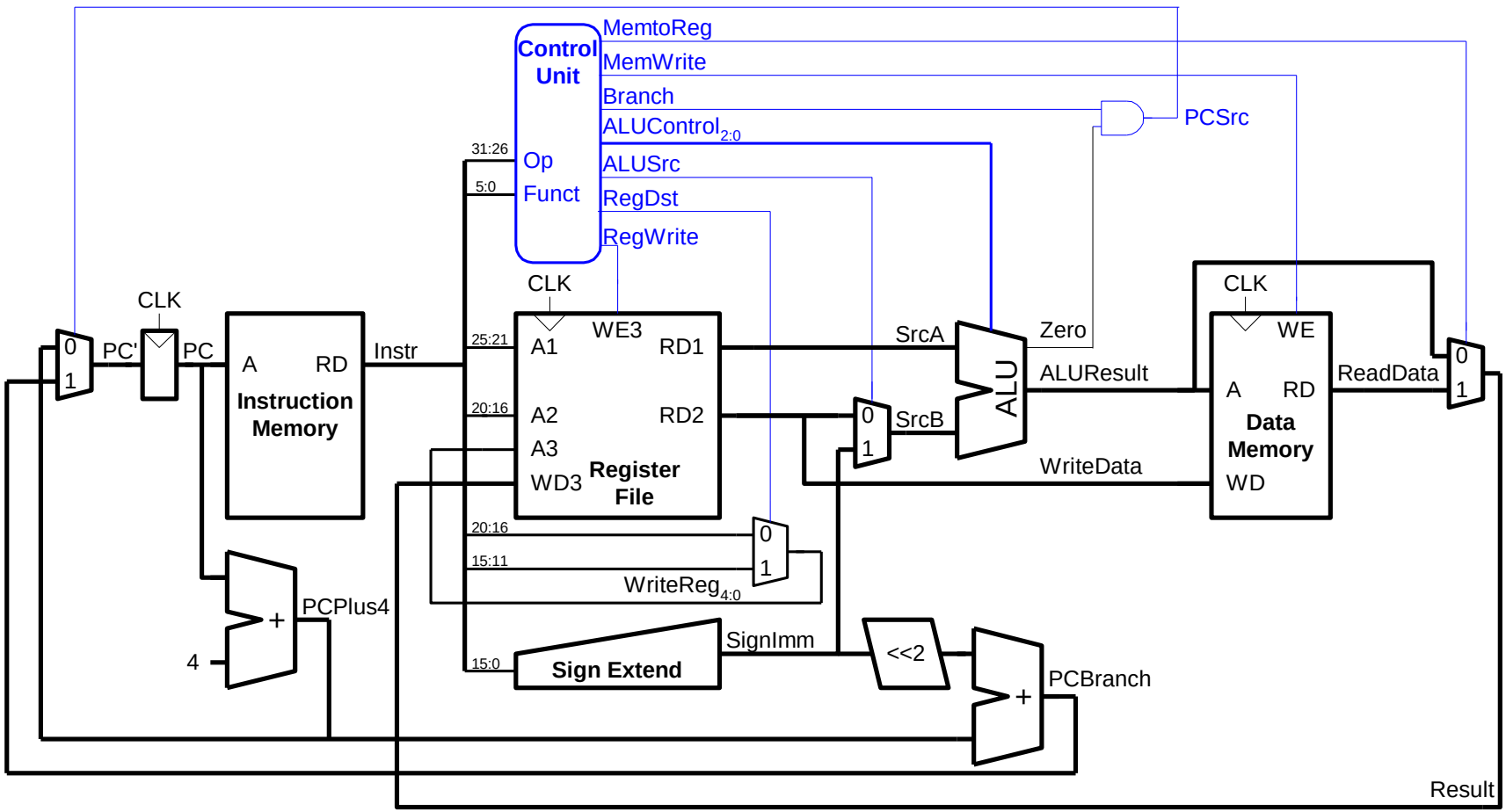
op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Similarly, We Need to Design the Control Unit

- Control signals are generated by the decoder in control unit

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
addi	001000	1	0	1	0	0	0	00	0
j	000010	0	X	X	X	0	X	XX	1

Another Complete Single-Cycle Processor (H&H)

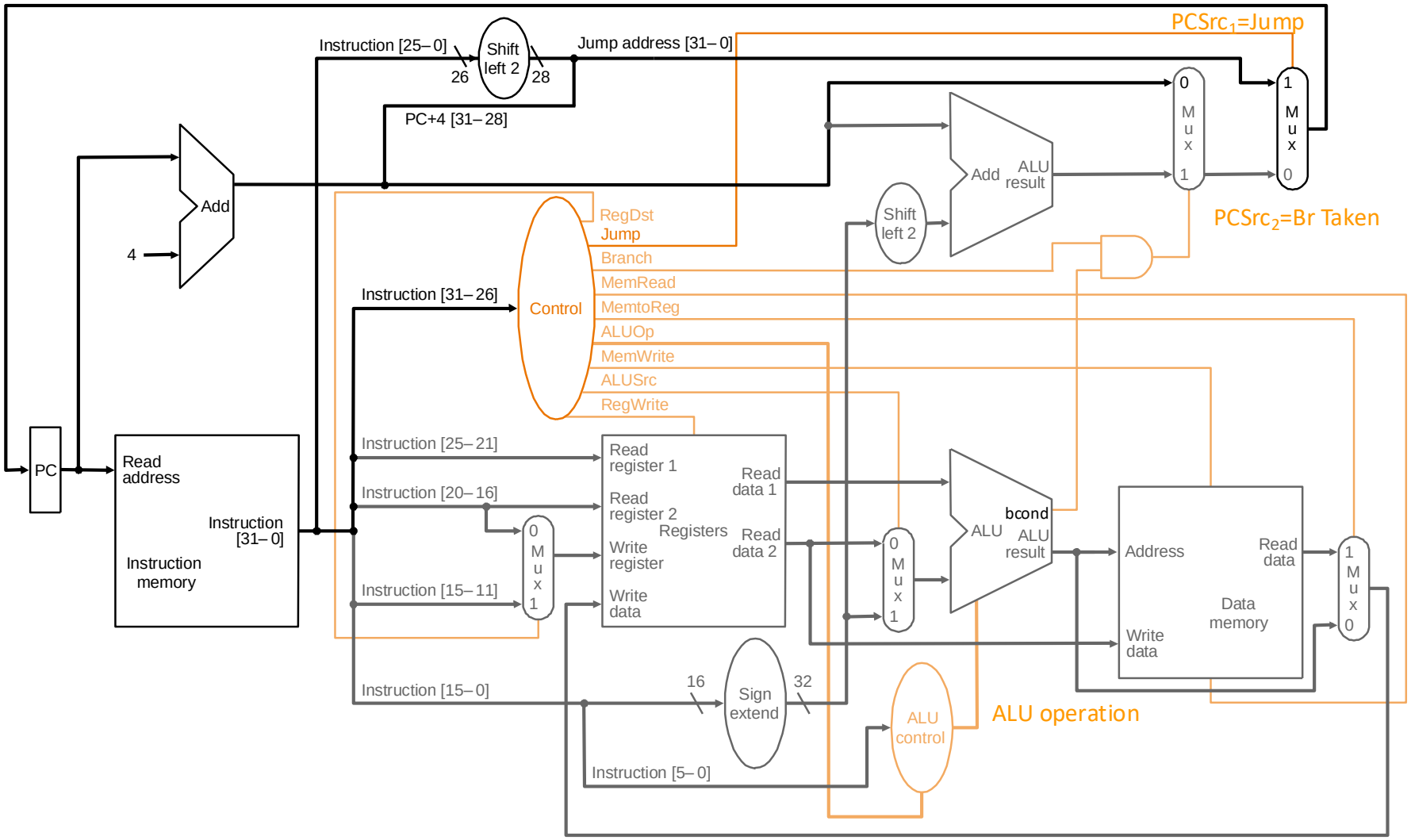


Your Reading Assignment

- Please go over and internalize the **Lecture Slides & Backup Slides**

- Please do your readings from the H&H Book
  - H&H, Chapter 7.1-7.3, 7.6

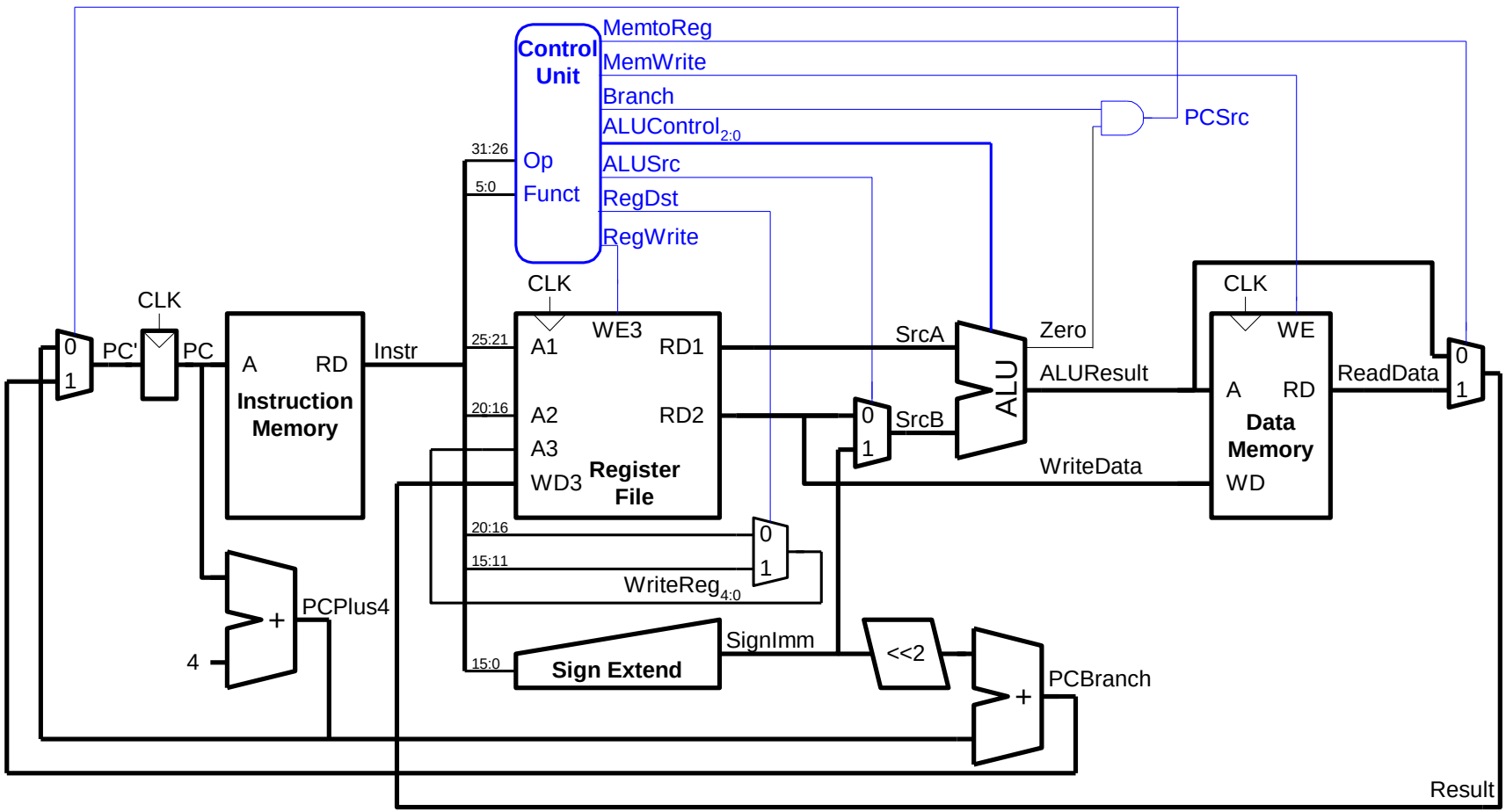
Single-Cycle Uarch I (We Developed in Lectures)



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

JAL, JR, JALR omitted

Single-Cycle Uarch II (In Your Readings)



Single-cycle processor. Harris and Harris, Chapter 7.3.

Evaluating the Single-Cycle Microarchitecture

A Single-Cycle Microarchitecture

- Is *this* a good idea/design?
- When is this a good design?
- When is this a bad design?
- How can we design a better microarchitecture?

Performance Analysis Basics

Recall: Performance Analysis Basics

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**
    - CPI: Number of cycles it takes to execute an instruction

- Execution time of an entire program
  - Sum over all instructions [{CPI} x {clock cycle time}]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**

Processor Performance

- **How fast is my program?**
  - Every program consists of a series of instructions
  - Each instruction needs to be executed

Processor Performance

- **How fast is my program?**
  - Every program consists of a series of instructions
  - Each instruction needs to be executed
- **How fast are my instructions?**
  - Instructions are realized on the hardware
  - Each instruction can take one or more clock cycles to complete
  - ***Cycles per Instruction = CPI***

Processor Performance

- **How fast is my program?**
  - Every program consists of a series of instructions
  - Each instruction needs to be executed
- **How fast are my instructions?**
  - Instructions are realized on the hardware
  - Each instruction can take one or more clock cycles to complete
  - ***Cycles per Instruction = CPI***
- **How long is one clock cycle?**
  - The critical path determines how much time one cycle requires = ***clock period***
  - 1/clock period = ***clock frequency*** = how many clock cycles are in each second

Processor Performance

- **As a general formula**
  - Our program consists of executing **N** instructions
  - Our processor needs **CPI** cycles (on average) for each instruction
  - The clock frequency of the processor is **f**

    → the clock period is therefore **T**=1/f

Processor Performance

- **As a general formula**
  - Our program consists of executing **N** instructions
  - Our processor needs **CPI** cycles (on average) for each instruction
  - The clock frequency of the processor is **f**
    → the clock period is therefore **T**=1/f

- **Our program executes in**

$$N \times CPI \times (1/f) =$$

$$N \times CPI \times T \text{ seconds}$$

Performance Analysis of Our Single-Cycle Design

A Single-Cycle Microarchitecture: Analysis

- Every instruction takes 1 cycle to execute
  - CPI (Cycles per instruction) is strictly 1
- How long each instruction takes is determined by how long the slowest instruction takes to execute
  - Even though many instructions do not need that long to execute
- Clock cycle time of the microarchitecture is determined by how long it takes to complete the slowest instruction
  - Critical path of the design is determined by the processing time of the slowest instruction

What is the Slowest Instruction to Process?

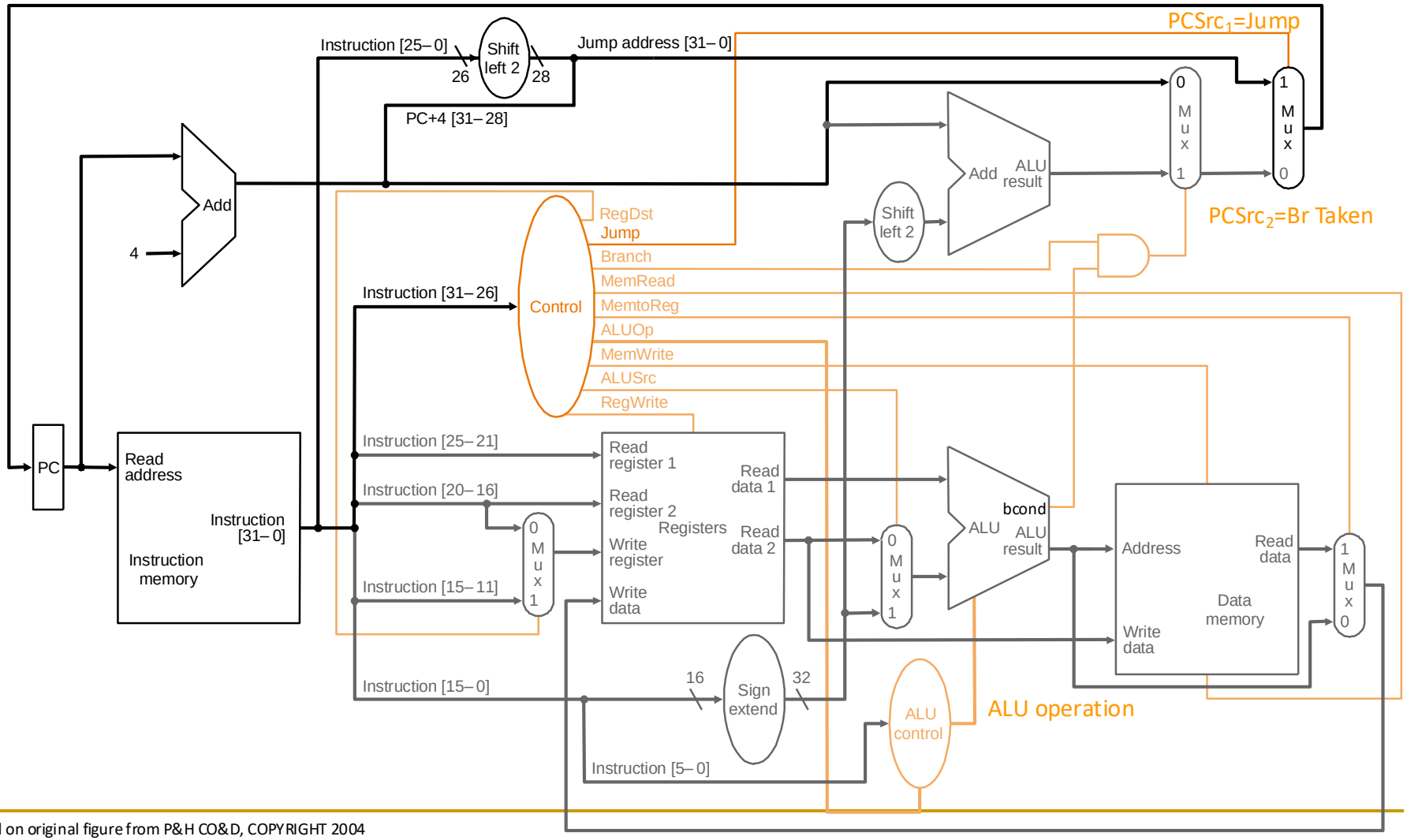
- Let's go back to the basics

- All six phases of the instruction processing cycle take a *single machine clock cycle* to complete
  - Fetch
  - Decode
  - Evaluate Address
  - Fetch Operands
  - Execute
  - Store Result

1. Instruction fetch (IF)
2. Instruction decode and register operand fetch (ID/RF)
3. Execute/Evaluate memory address (EX/AG)
4. Memory operand fetch (MEM)
5. Store/writeback result (WB)

- Does every instruction take the same time (latency) to complete?

Let's Find the Critical Path



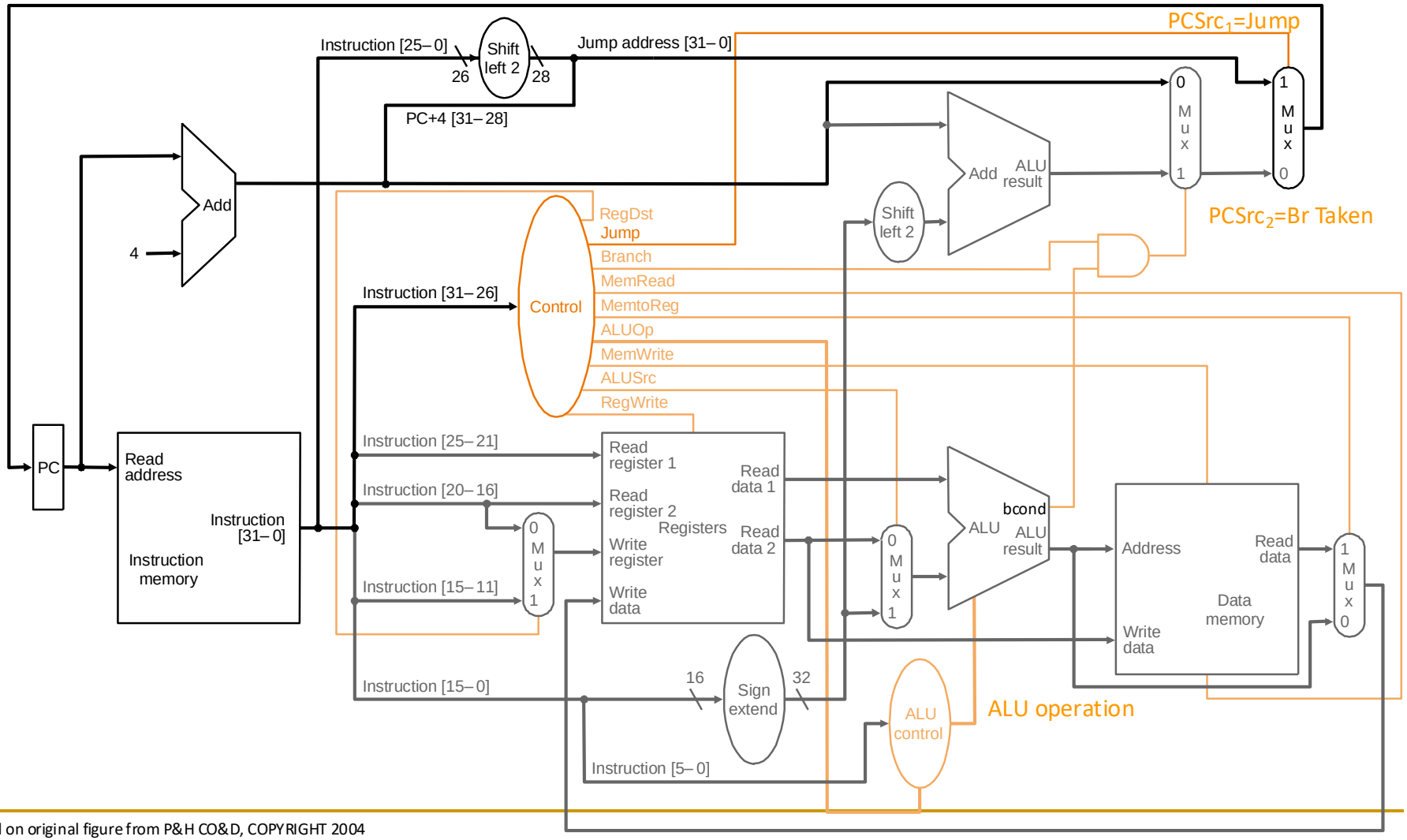
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Example Single-Cycle Datapath Analysis

- Assume (for the design in the previous slide)
  - memory units (read or write): 200 ps
  - ALU and adders: 100 ps
  - register file (read or write): 50 ps
  - other logic or wire delay: 0 ps

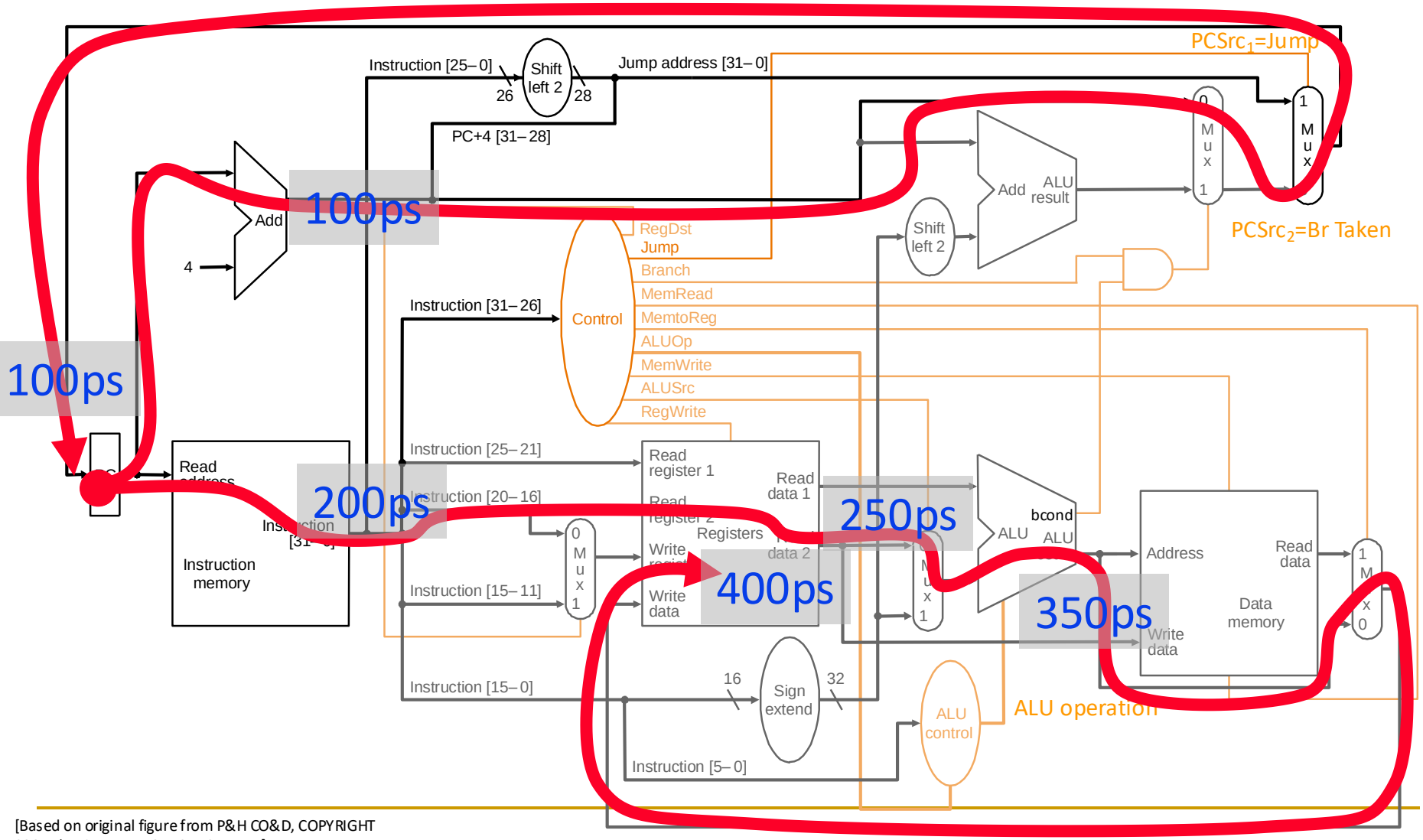
steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

Let's Find the Critical Path



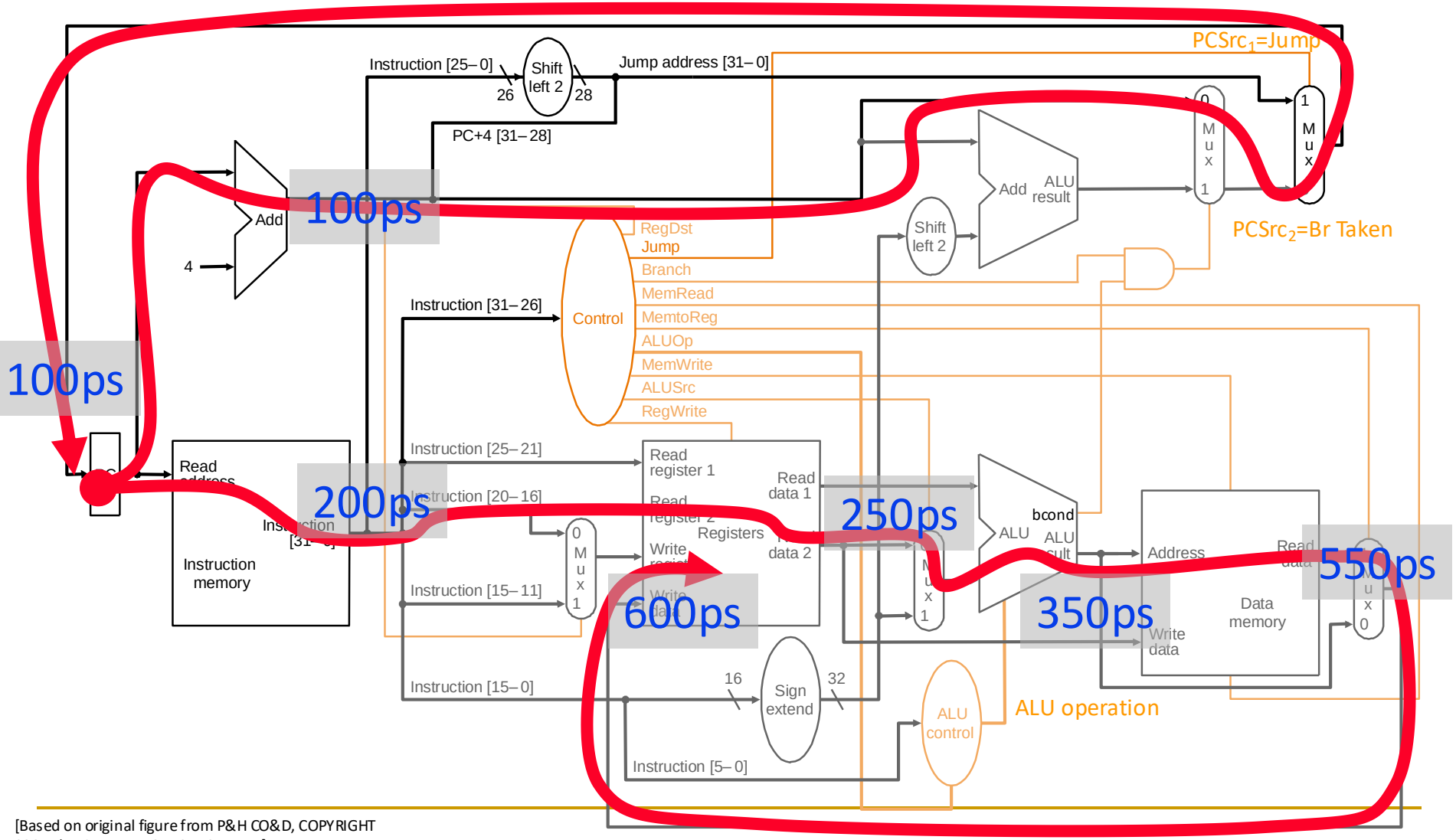
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

R-Type and I-Type ALU



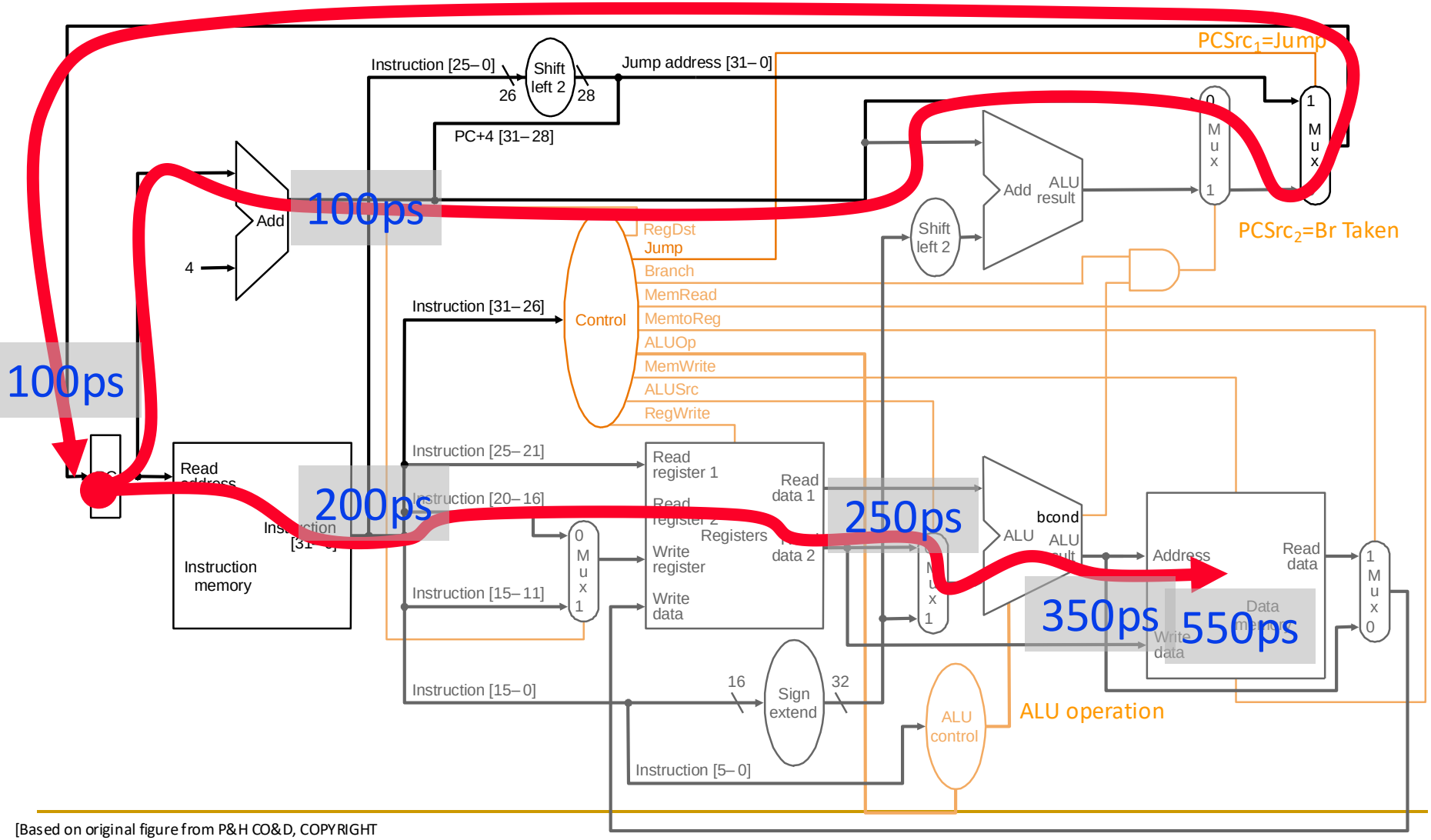
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

LW



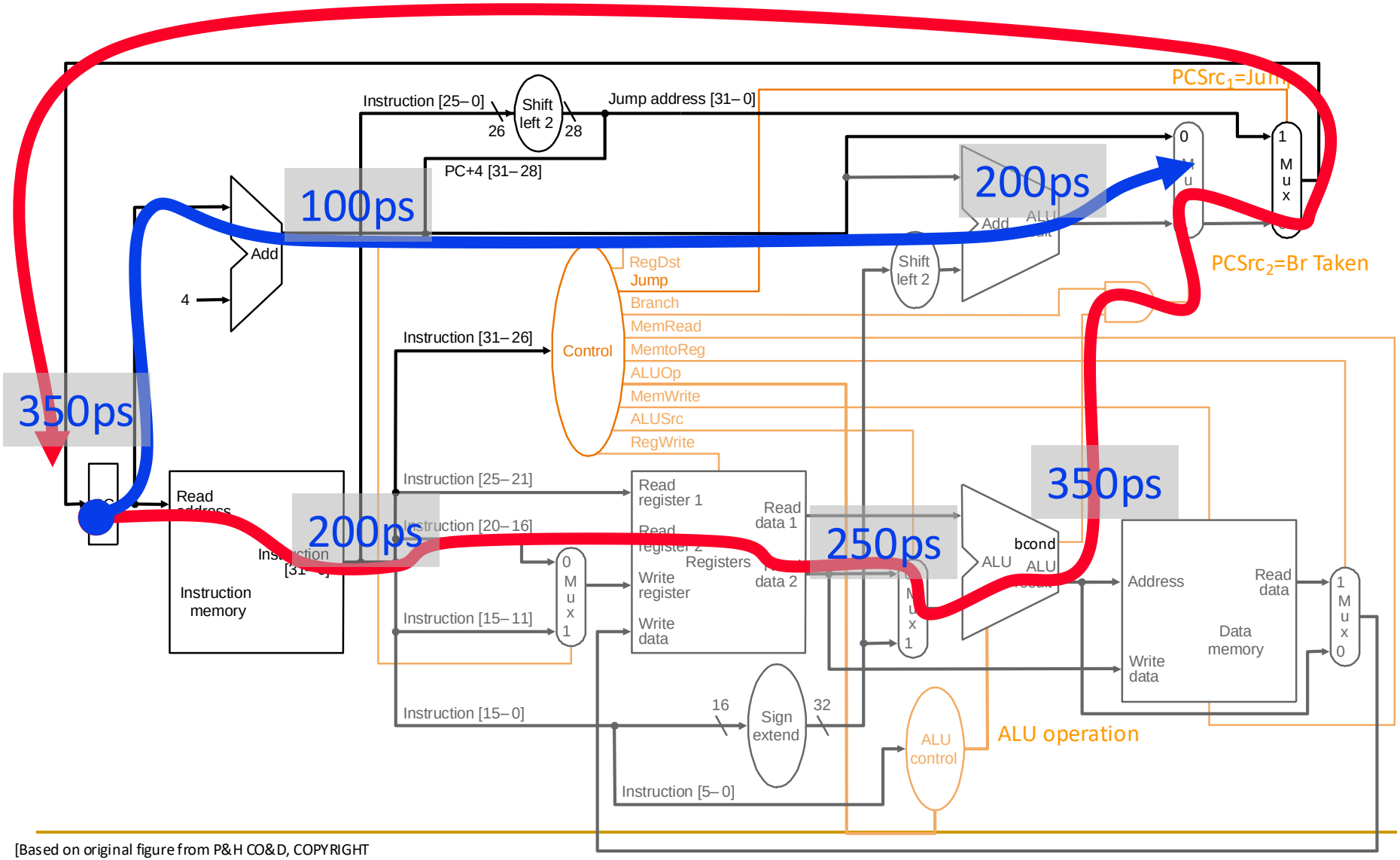
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

SW



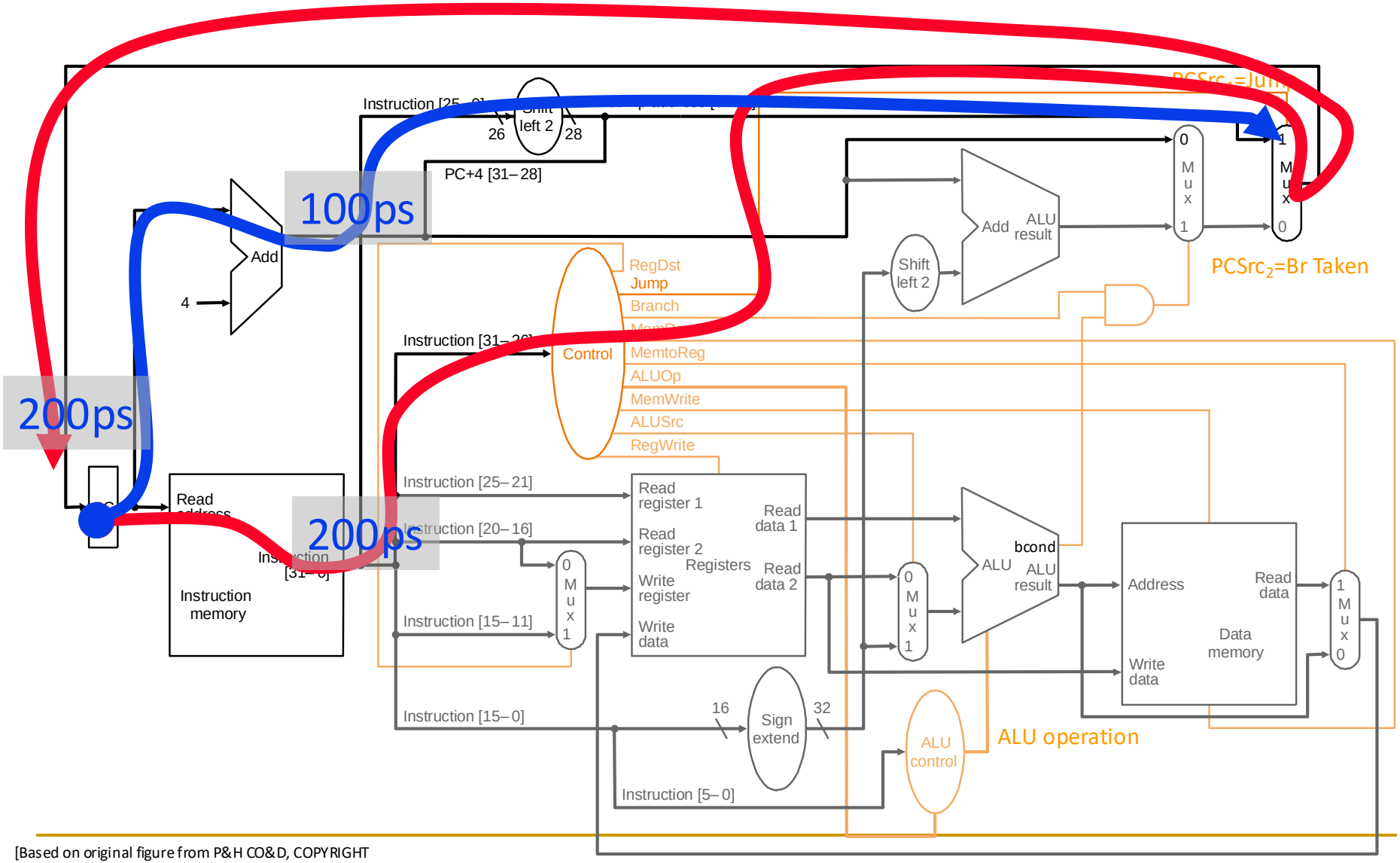
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Branch Taken



[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Jump



[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Example Single-Cycle Datapath Analysis

- Assume (for the design in the previous slide)
  - memory units (read or write): 200 ps
  - ALU and adders: 100 ps
  - register file (read or write): 50 ps
  - other logic or wire delay: 0 ps

steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

What About Control Logic?

- How does that affect the critical path?

- Food for thought for you:
  - Can control logic be on the critical path?
  - Historical example:
    - CDC 5600: control store access took too long…

What is Really the Slowest Instruction to Process?

- Real world: **Memory is slow (not magic)**
- What if memory *sometimes* takes 150ns to access?
- Does it make sense to have a simple register to register add or jump to take {150ns + all else to perform a memory operation}?
- And, what if you need to access memory more than once to process an instruction?
  - Which instructions require this?

Single Cycle uArch: Complexity

- Contrived
  - All instructions run as slow as the slowest instruction
- Inefficient
  - All instructions run as slow as the slowest instruction
  - Must provide worst-case combinational resources in parallel as required by any instruction
  - Need to replicate a resource if it is needed more than once by an instruction during different parts of the instruction processing cycle
- Not necessarily the simplest way to implement an ISA
  - Tough for complex instructions, e.g., REP MOVS (x86) or INDEX (VAX)
- Not easy to optimize/improve performance
  - Optimizing the common case (frequent instructions) does not work
  - Need to optimize the worst case all the time

(Micro)architecture Design Principles

- Critical path design
  - Find and decrease the maximum combinational logic delay
  - Break a path into multiple cycles if it takes too long

- Bread and butter (common case) design
  - Spend time and resources on where it matters most
    - i.e., improve what the machine is really designed to do
  - Common case vs. uncommon case

- Balanced design
  - Balance instruction/data flow through hardware components
  - Design to eliminate bottlenecks: balance the hardware for the work

Single-Cycle Design vs. Design Principles

- Critical path design
- Bread and butter (common case) design
- Balanced design

How does a single-cycle microarchitecture fare with respect to these principles?

Aside: System Design Principles

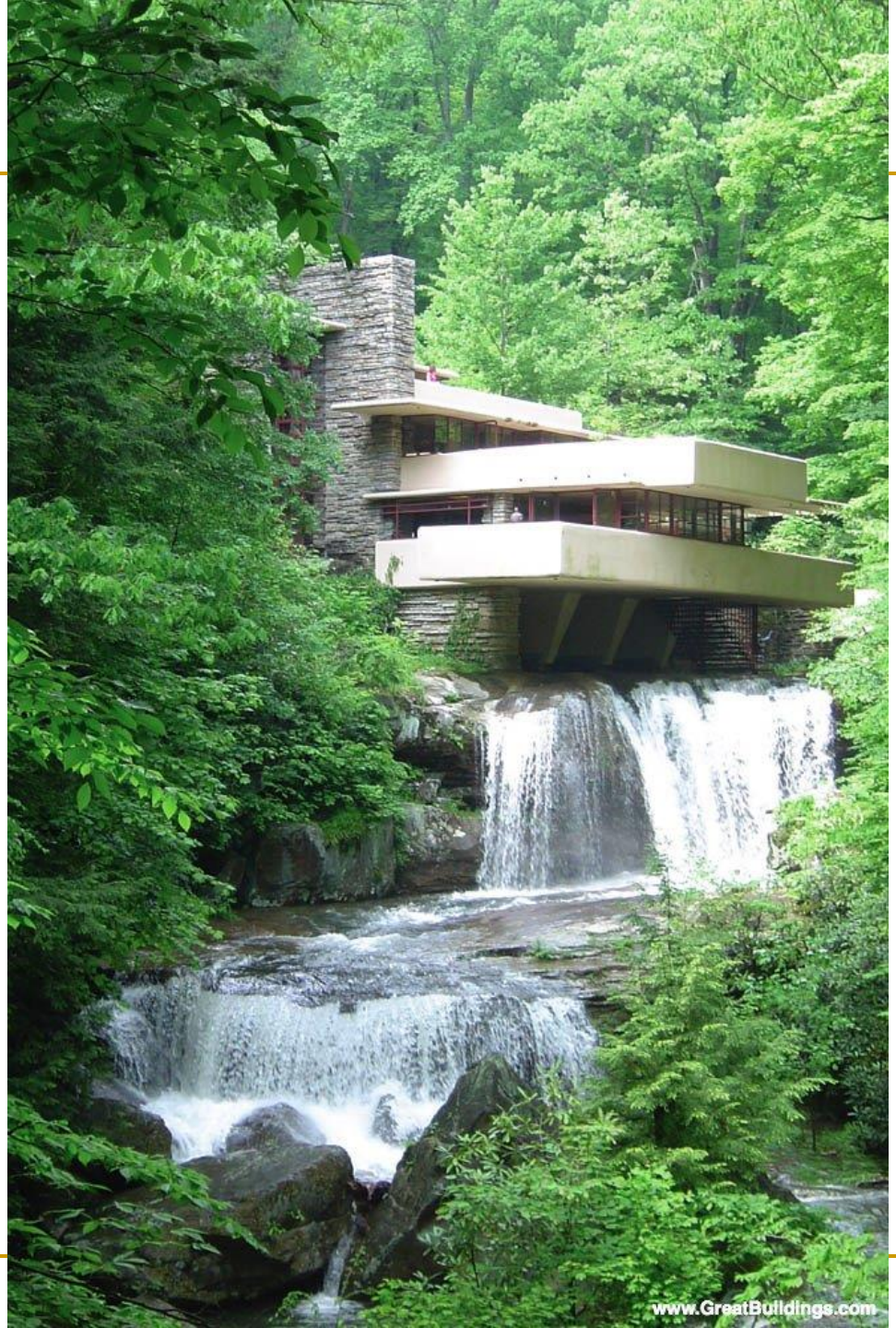
- When designing computer systems/architectures, it is important to follow good principles
  - Actually, this is true for **any system design**
    - Real architectures, buildings, bridges, train stations, …
    - Good consumer products
    - Security & safety-critical systems
    - Decision making systems
    - …

- Remember: "principled design" from our second lecture
  - Frank Lloyd Wright: "architecture […] based upon principle, and not upon precedent"

Aside: Principled Architecture

- "architecture […] based upon principle, and not upon precedent"

This



That

Recall: Takeaways

- It all starts from the basic building blocks and design principles
- And, knowledge of how to use, apply, enhance them
- Underlying technology might change (e.g., steel vs. wood)
  - but methods of taking advantage of technology bear resemblance
  - methods used for design depend on the principles employed

Aside: System Design Principles

- We will continue to cover key principles in this course
- Here are some references where you can learn more

- Yale Patt, "Requirements, Bottlenecks, and Good Fortune: Agents for Microprocessor Evolution," Proc. of IEEE, 2001. (Levels of transformation, design point, etc)

- Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966. (Flynn's Bottleneck → Balanced design)

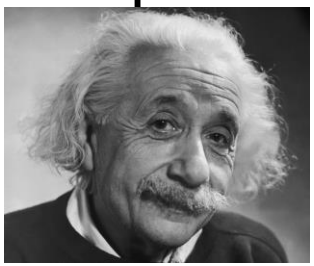
- Gene M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS Conference, April 1967. (Amdahl's Law → Common-case design)

- Butler W. Lampson, "Hints for Computer System Design," ACM Operating Systems Review, 1983.

A Key System Design Principle

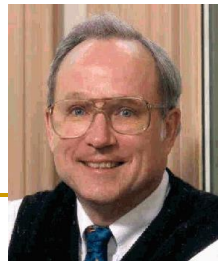
- **Keep it simple**

- "Everything should be made as simple as possible, but no simpler."
  - Albert Einstein (paraphrased)



- And, **keep it low cost**: "An engineer is a person who can do for a dime what any fool can do for a dollar."

- For more, see:
  - Butler W. Lampson, "Hints for Computer System Design," ACM Operating Systems Review, 1983, updated 2021.
  - https://arxiv.org/pdf/2011.02455



Can We Do Better?

Multi-Cycle Microarchitectures

Multi-Cycle Microarchitectures

- Goal: Let each instruction take (close to) only as much time it really needs

- Idea
  - Determine clock cycle time independently of instruction processing time
  - Each instruction takes as many clock cycles as it needs to take
    - Multiple state transitions per instruction
    - The states followed by each instruction is different

Recall: The "Process Instruction" Step

- ISA specifies abstractly what AS' should be, given an instruction and AS
  - It defines an **abstract finite state machine** where
    - State = programmer-visible state
    - Next-state logic = instruction execution specification
  - From ISA point of view, there are no "intermediate states" between AS and AS' during instruction execution
    - One state transition per instruction
- Microarchitecture implements how AS is transformed to AS'
  - There are many choices in implementation
  - We can have programmer-invisible state to optimize the speed of instruction execution: **multiple** state transitions per instruction
    - Choice 1: AS → AS' (transform AS to AS' in a single clock cycle)
    - Choice 2: AS → AS+MS1 → AS+MS2 → AS+MS3 → AS' (take multiple clock cycles to transform AS to AS')

Multi-Cycle Microarchitecture

AS = Architectural (programmer visible) state
at the beginning of an instruction

↓

Step 1: Process part of instruction in one clock cycle

↓

Step 2: Process part of instruction in the next clock cycle

↓

...

↓

AS' = Architectural (programmer visible) state
at the end of a clock cycle

Recall: Control of the Instruction Cycle

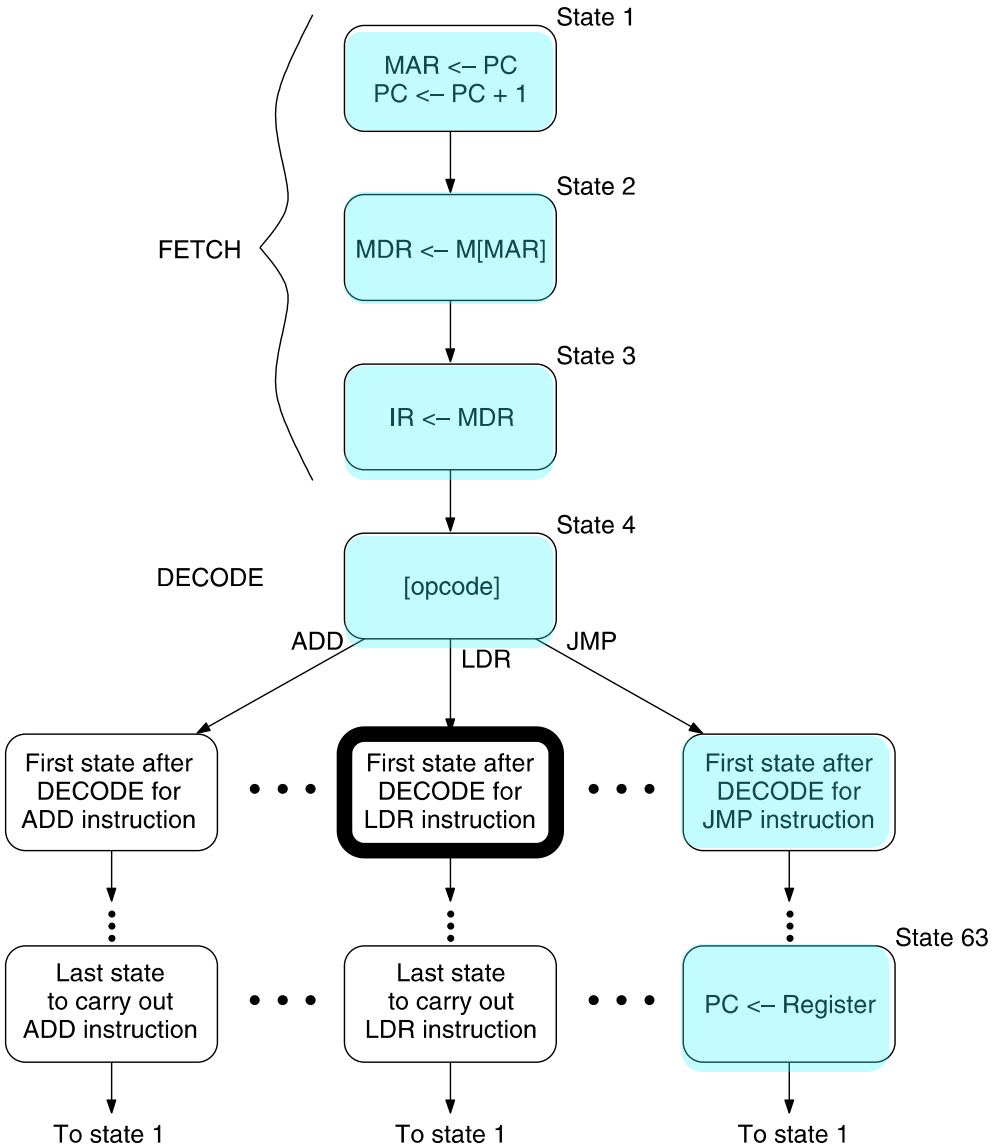


Figure 4.4 An abbreviated state diagram of the LC-3

- State 1
  - The FSM asserts GatePC and LD.MAR
  - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
  - MDR is loaded with the instruction
- State 3
  - The FSM asserts GateMDR and LD.IR
- State 4
  - The FSM goes to next state depending on opcode
- State 63
  - JMP loads register into PC
- Full state diagram in Patt&Pattel, Appendix C

This is an FSM Controlling a Multi-Cycle LC-3 Microarchitecture

Recall: Full State Machine for LC-3b

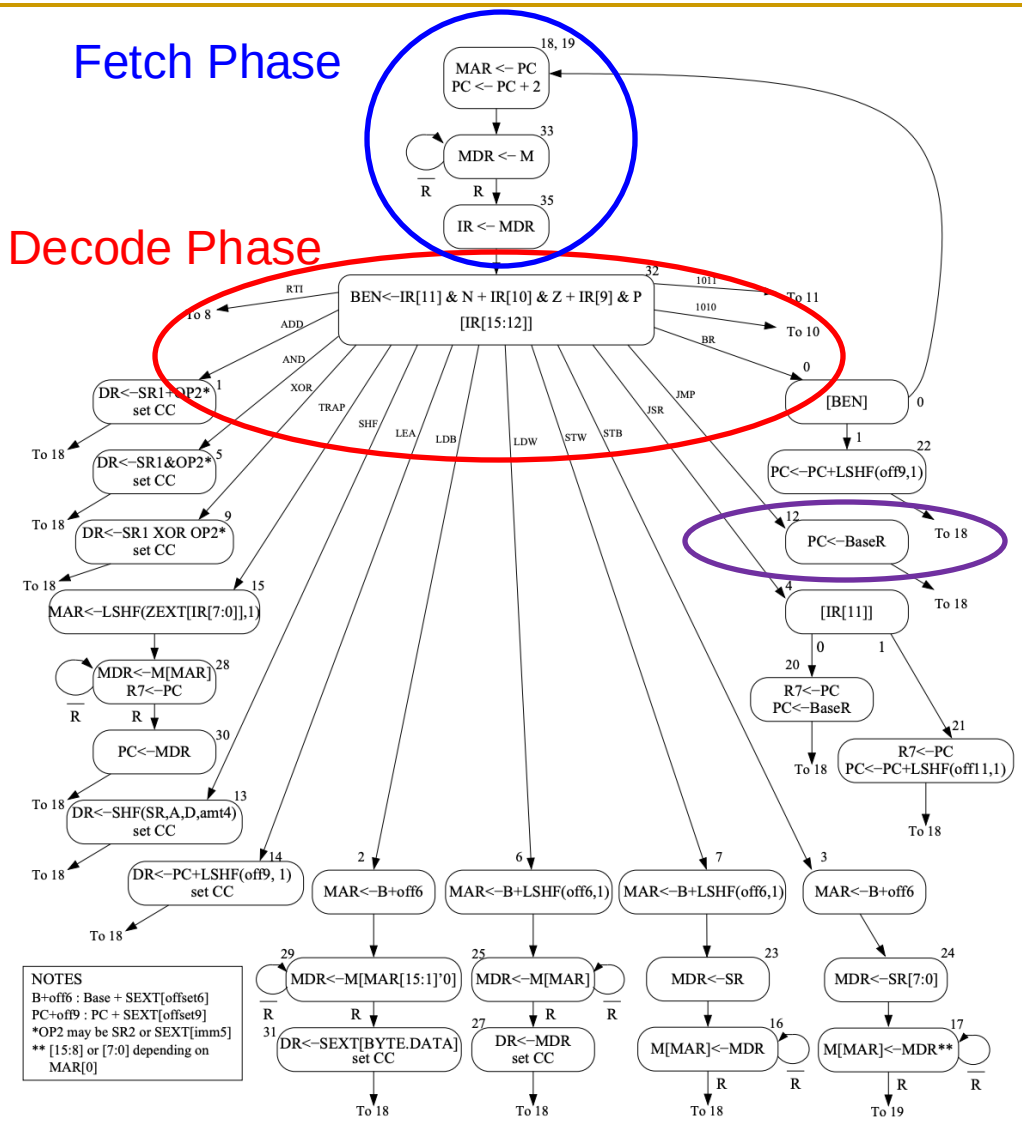


Figure C.2: A state machine for the LC-3b

Full FSM Controlling a Multi-Cycle LC-3b Microarchitecture

https://safari.ethz.ch/digitaltechnik/spring2022/lib/exe/fetch.php?media=pp-appendixc.pdf

Recall: LC-3 Multi-Cycle Implementation

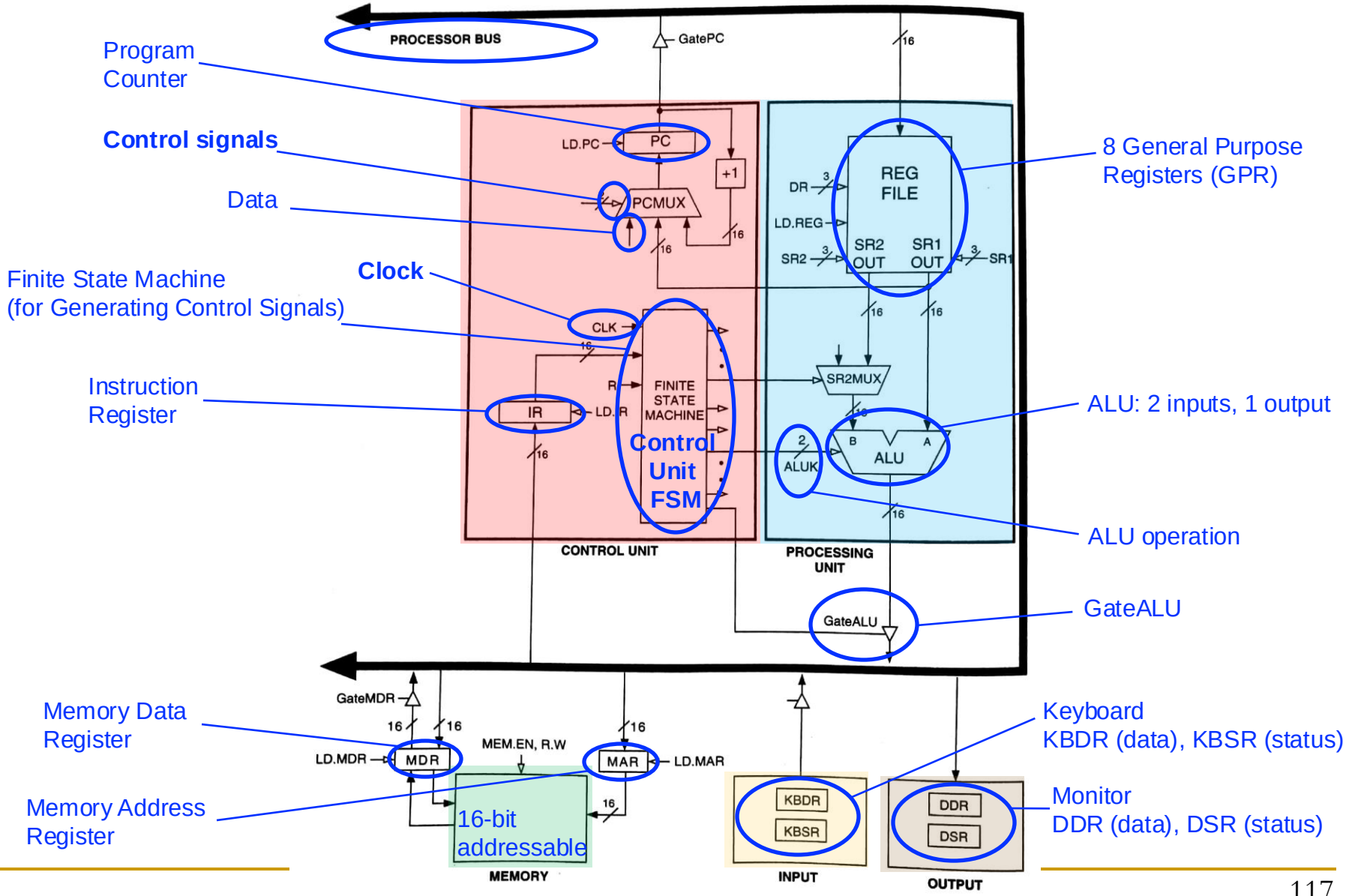


Figure 4.3 The LC-3 as an example of the von Neumann model

Benefits of Multi-Cycle Design

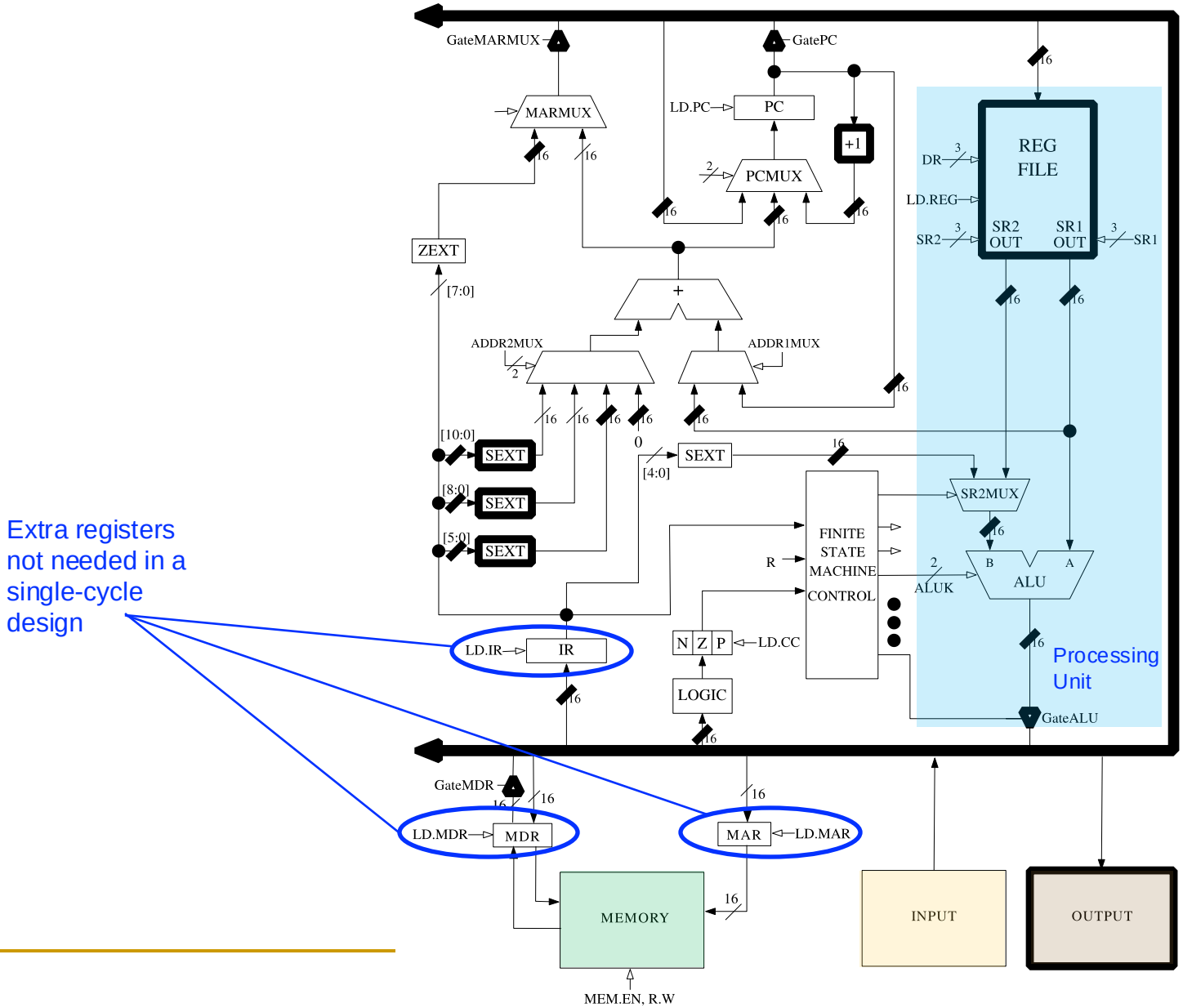
- Critical path design
  - Can keep reducing the critical path independently of the worst-case processing time of any instruction
- Bread and butter (common case) design
  - Can optimize the number of states it takes to execute "important" instructions that make up much of the execution time
- Balanced design
  - No need to provide more capability or resources than really needed
    - An instruction that needs resource X multiple times does **not** require multiple X's to be implemented
    - Leads to more efficient hardware: Can reuse hardware components needed multiple times for an instruction

Downsides of Multi-Cycle Design

- Need to store the intermediate results at the end of each clock cycle
  - Hardware overhead for microarchitectural registers
  - Register setup/hold overhead (i.e., sequencing overhead) is paid multiple times for an instruction

- Limited concurrency
  - Only a small part of the machine is used at any point in time

Multi-Cycle LC-3 Data Path



Remember: Performance Analysis

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**

CPI: Cycles Per Instruction

- Execution time of an entire program
  - Sum over all instructions [{CPI} x {clock cycle time}]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**

- Single-cycle microarchitecture performance
  - CPI = 1
  - Clock cycle time = long
- Multi-cycle microarchitecture performance
  - CPI = different for each instruction
    - Average CPI → hopefully small
  - Clock cycle time = short

In multi-cycle, we have two degrees of freedom to optimize independently

We Covered Until Here in Lecture

Digital Design & Computer Arch.

Lecture 10: Microarchitecture Fundamentals & Design II

Prof. Onur Mutlu

ETH Zürich

Spring 2025

21 March 2025

Further Slides for Your Own Study (May Be Covered in Future Lectures)

A Multi-Cycle Microarchitecture

A Closer Look

An Elegant Multi-Cycle Processor Design

- Maurice Wilkes, "The Best Way to Design an Automatic Calculating Machine," Manchester Univ. Computer Inaugural Conf., 1951.

THE BEST WAY TO DESIGN AN AUTOMATIC CALCULATING MACHINE

By M. V. Wilkes, M.A., Ph.D., F.R.A.S.

- An elegant implementation:
  - The concept of microcoded/microprogrammed machines

Multi-Cycle Microarchitectures

- Key Idea for Realization
  - One can implement the "process instruction" step as a finite state machine that sequences between states and eventually returns back to the "fetch instruction" state
  - A state is defined by the control signals asserted in it
  - Control signals for the next state are determined in current state

Recall: The Instruction Processing "Cycle"

- FETCH
- DECODE
- EVALUATE ADDRESS
- FETCH OPERANDS
- EXECUTE
- STORE RESULT

A Basic Multi-Cycle Microarchitecture

- Instruction processing cycle divided into "states"
  - A stage in the instruction processing cycle can take multiple states
- A multi-cycle microarchitecture sequences from state to state to process an instruction
  - The behavior of the machine in a state is completely determined by control signals in that state
- The behavior of the entire processor is specified fully by a *finite state machine*
- In a state (clock cycle), control signals control two things:
  - How the datapath should process the data
  - How to generate the control signals for the (next) clock cycle

Recall: Control of the Instruction Cycle

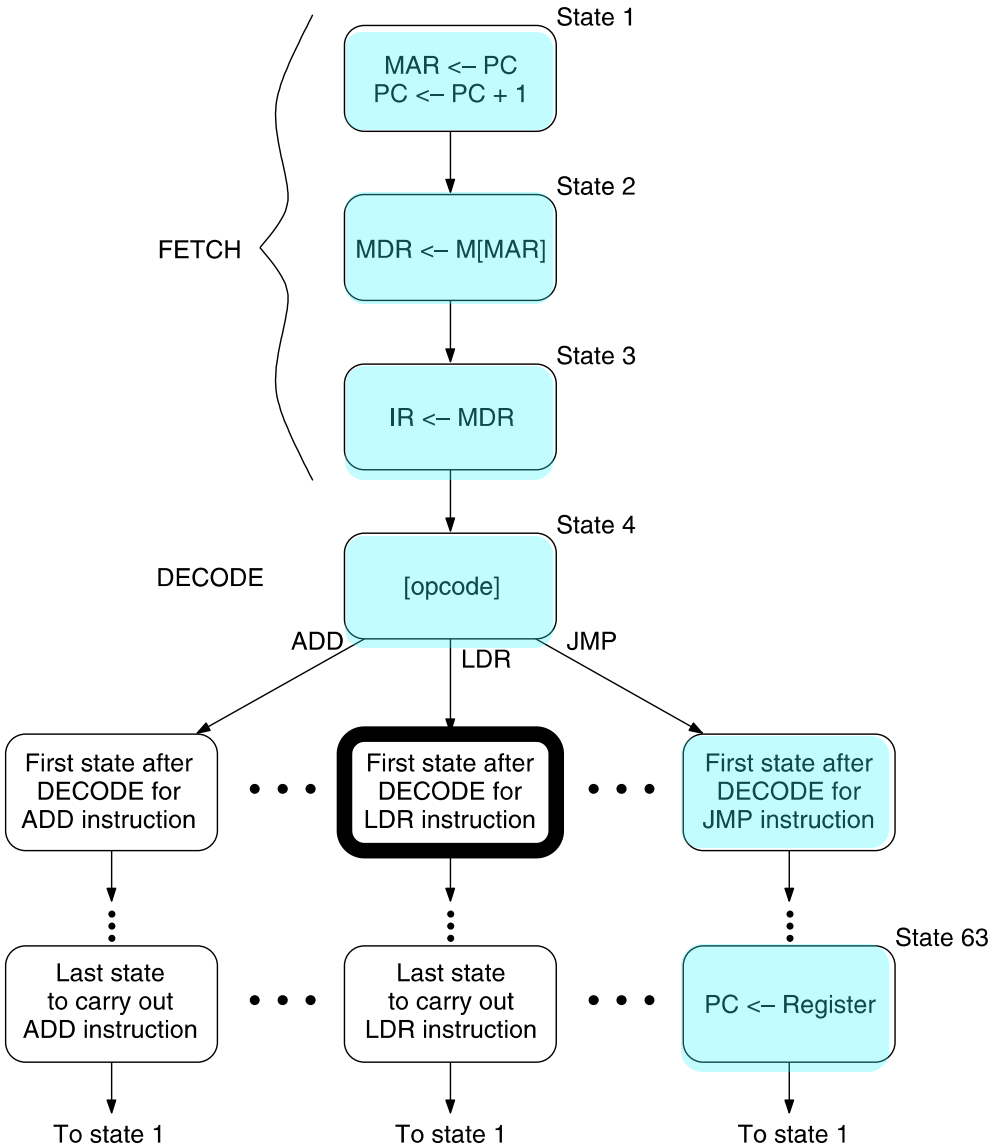


Figure 4.4 An abbreviated state diagram of the LC-3

- State 1
  - The FSM asserts GatePC and LD.MAR
  - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
  - MDR is loaded with the instruction
- State 3
  - The FSM asserts GateMDR and LD.IR
- State 4
  - The FSM goes to next state depending on opcode
- State 63
  - JMP loads register into PC
- Full state diagram in Patt&Pattel, Appendix C

This is an FSM Controlling a Multi-Cycle LC-3 Microarchitecture

Recall: Full State Machine for LC-3b

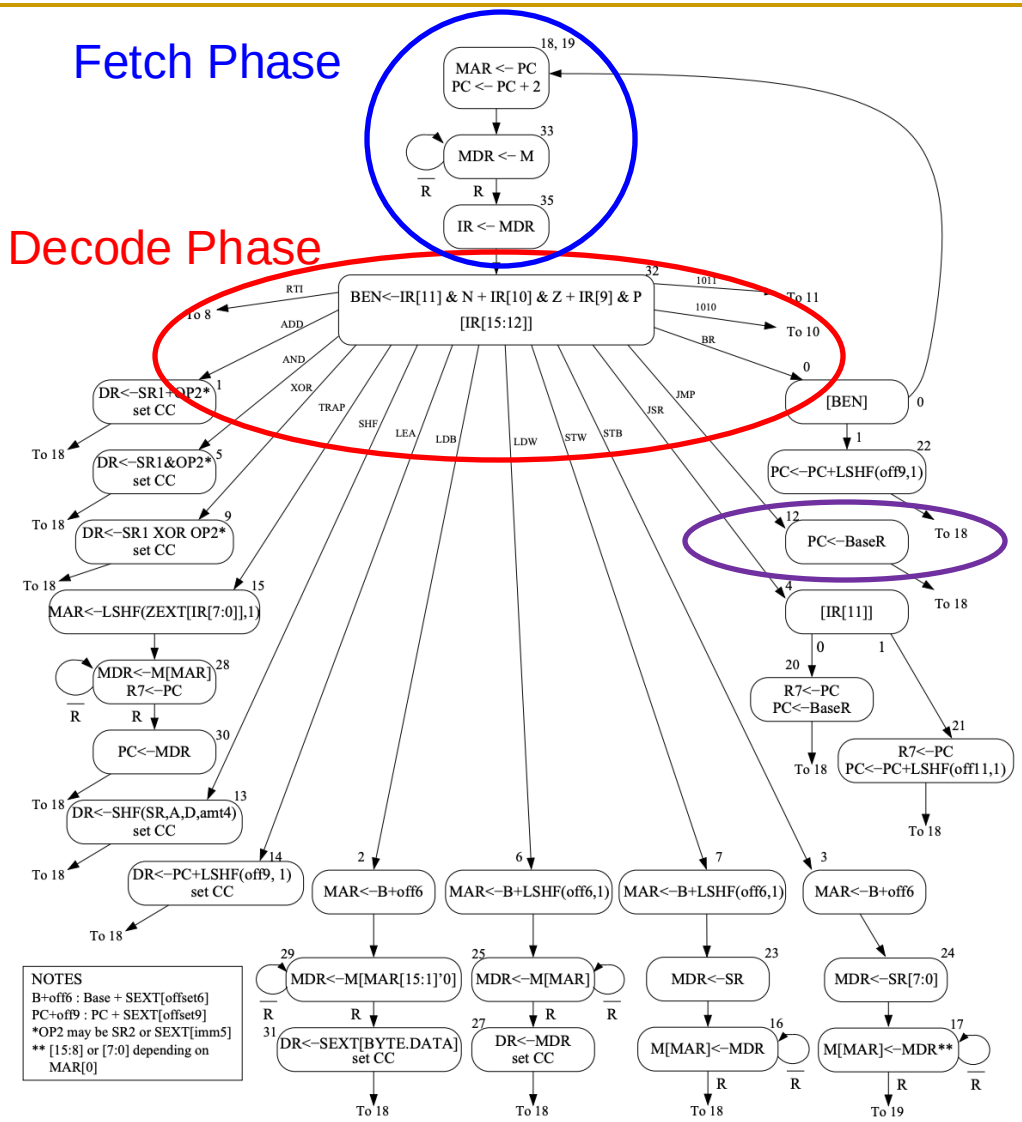


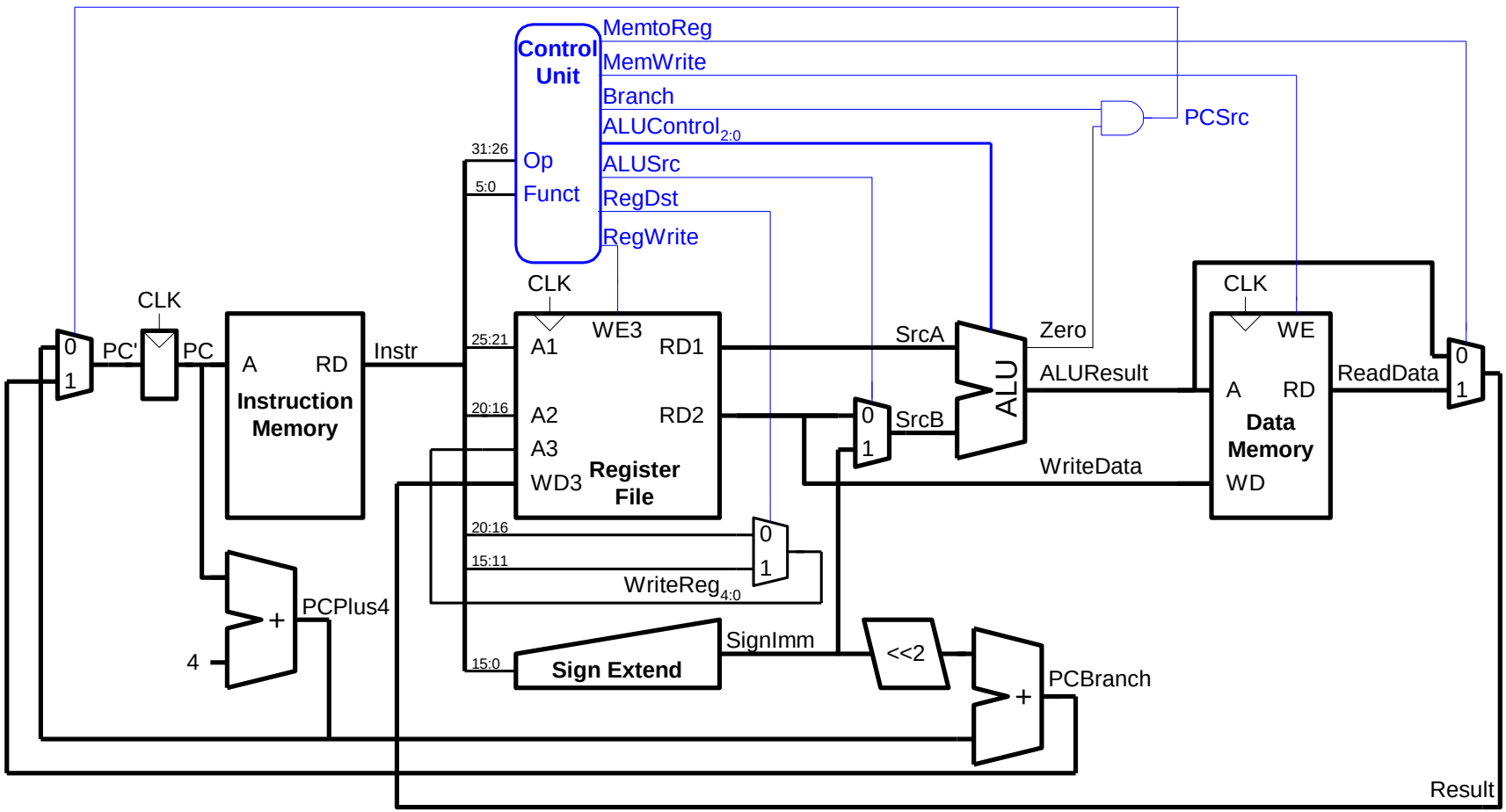
Figure C.2: A state machine for the LC-3b

Full FSM Controlling a Multi-Cycle LC-3b Microarchitecture

https://safari.ethz.ch/digitaltechnik/spring2022/lib/exe/fetch.php?media=pp-appendixc.pdf

Another Example Multi-Cycle Microarchitecture

Remember the Single-Cycle Uarch II (Readings)



Single-cycle processor. Harris and Harris, Chapter 7.3.

Why Do We Want Multi-Cycle?

- Single-cycle microarchitecture:
  -- cycle time limited by longest instruction (`lw`) → low clock frequency
  -- three adders/ALUs and two memories → high hardware cost

- Multi-cycle microarchitecture:
  + higher clock frequency
  + simpler instructions take only a few clock cycles
  + reuse expensive hardware across multiple cycles
  -- hardware overhead for storing intermediate results
  -- sequential logic overhead paid many times for each instruction

- Multi-cycle requires the same design steps as single cycle:
  - datapath
  - control logic

What Can We Optimize with Multi-Cycle

- Single-cycle microarchitecture uses two memories
  - One memory stores instructions, the other data
  - We want to use a single memory (lower cost)

- Single-cycle microarchitecture needs three adders
  - ALU, PC, Branch address calculation
  - We want to use only one ALU for all operations (lower cost)

- Single-cycle microarchitecture: each instruction takes one cycle
  - The slowest instruction slows down every single instruction
  - We want to determine clock cycle time independently of instruction processing time
    - Divide each instruction into multiple clock cycles
    - Simpler instructions can be very fast (compared to the slowest)

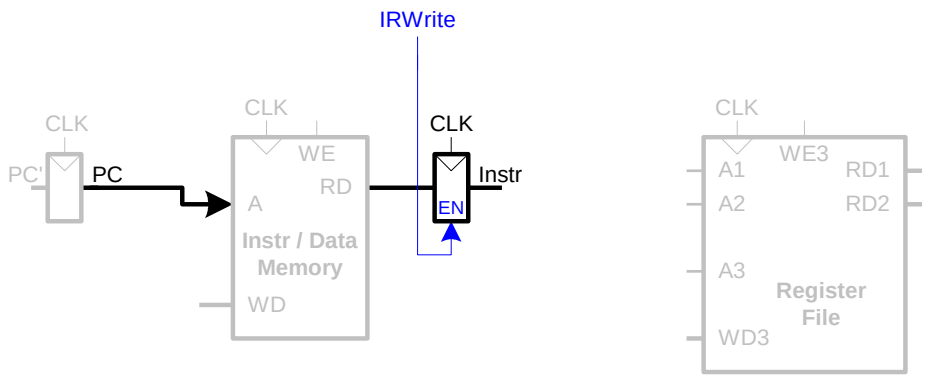
Let's Construct the Multi-Cycle Datapath

Consider the lw Instruction

- **For an instruction such as: `lw $t0, 0x20($t1)`**
- **We need to:**
  - Read the instruction from memory
  - Then read **`$t1`** from register array
  - Add the immediate value (**`0x20`**) to calculate the memory address
  - Read the content of this address
  - Write to the register **`$t0`** this content

Multi-Cycle Datapath: Instruction Fetch

- **We will consider lw, but fetch is the same for all instructions**
  - STEP 1: Fetch instruction

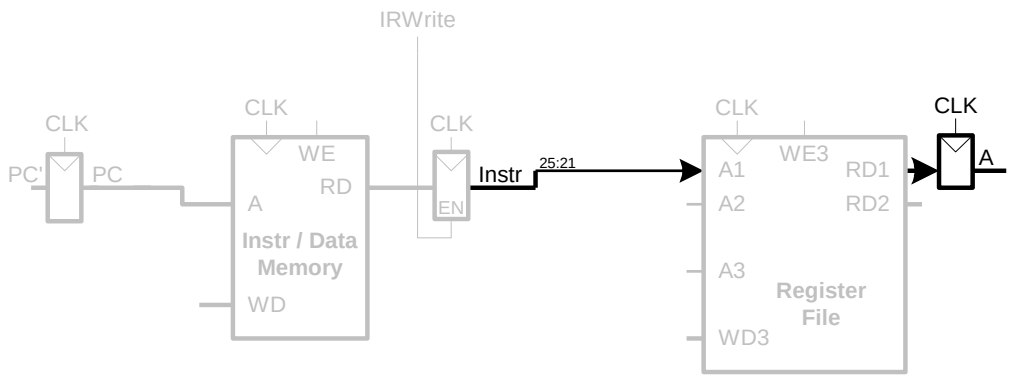



```
read from the memory location [rs]+imm to location [rt]
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

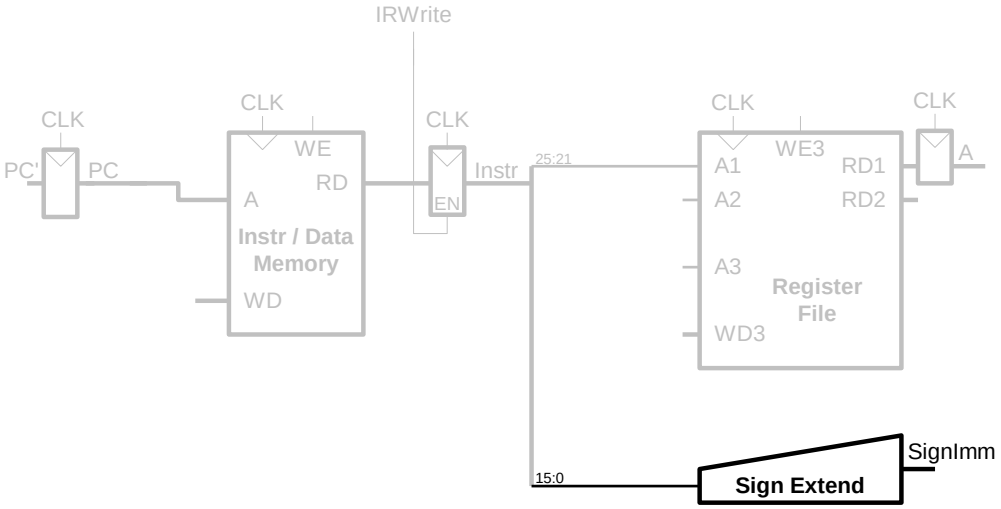
Multi-Cycle Datapath: lw register read



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

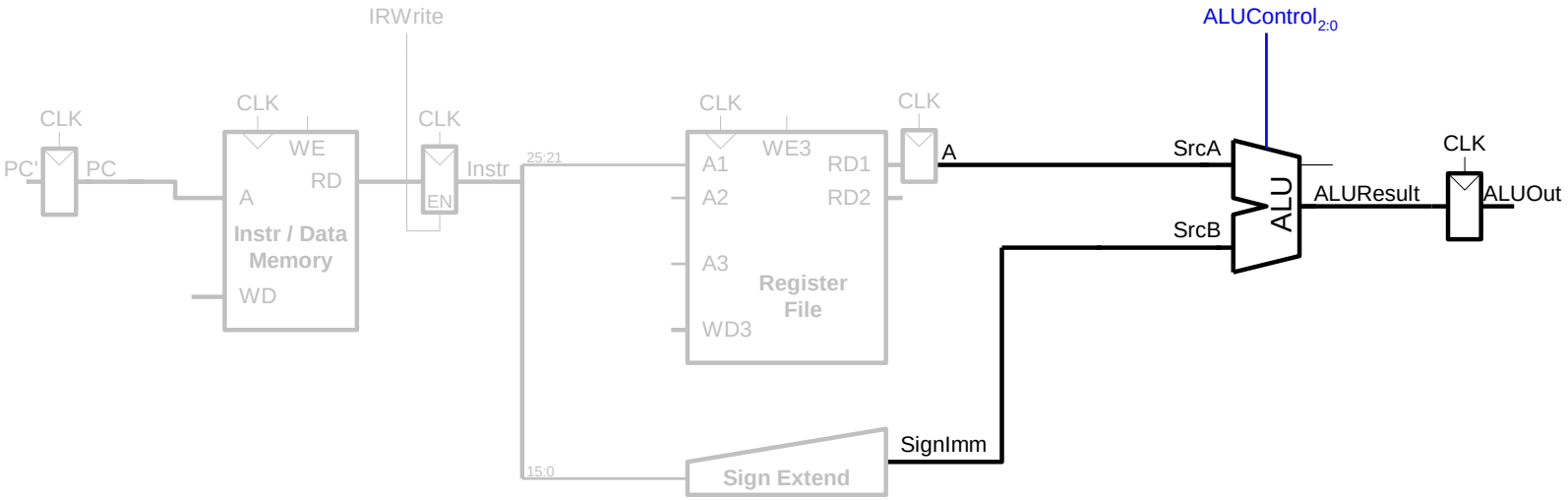
Multi-Cycle Datapath: lw immediate



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw address

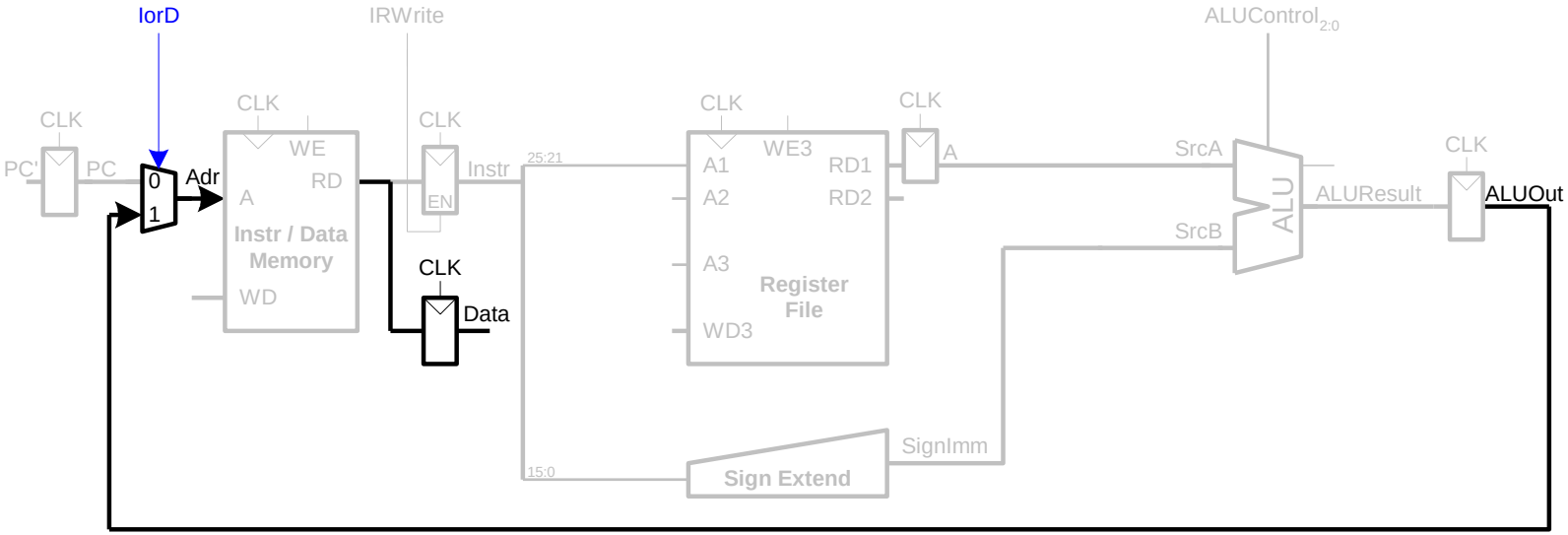


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw memory read

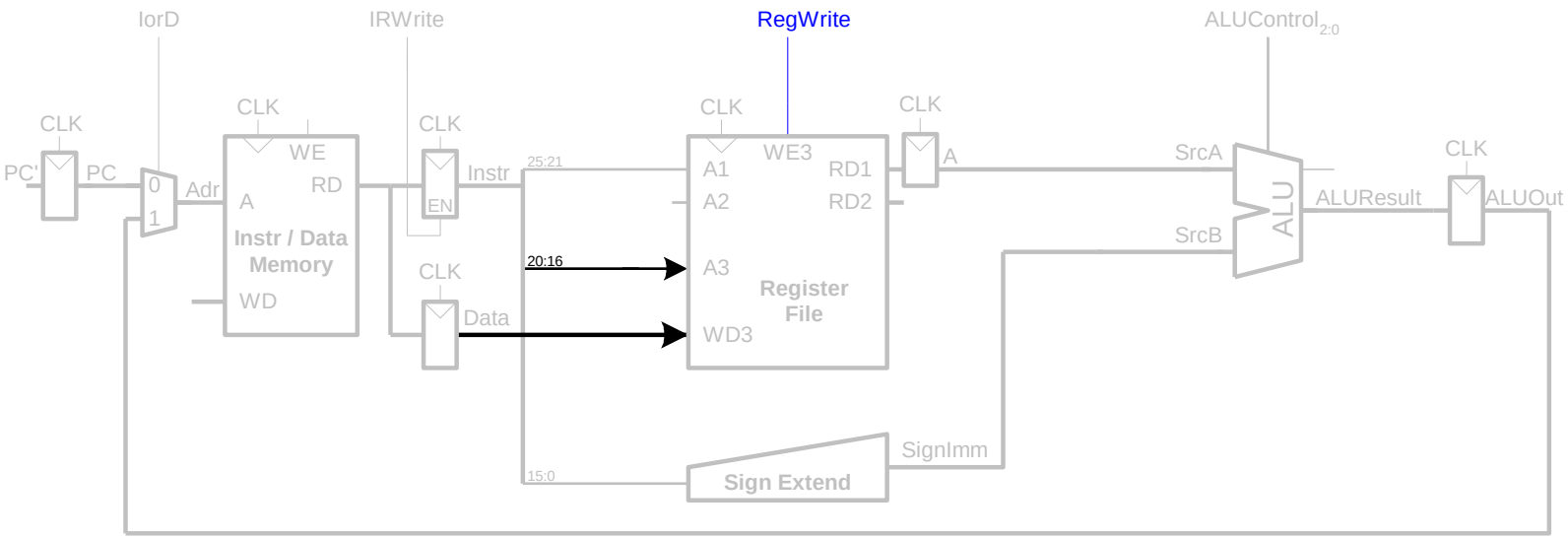
Now we can use **a single memory** to
both fetch an instruction and fetch an operand
(in different clock cycles)



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw write register

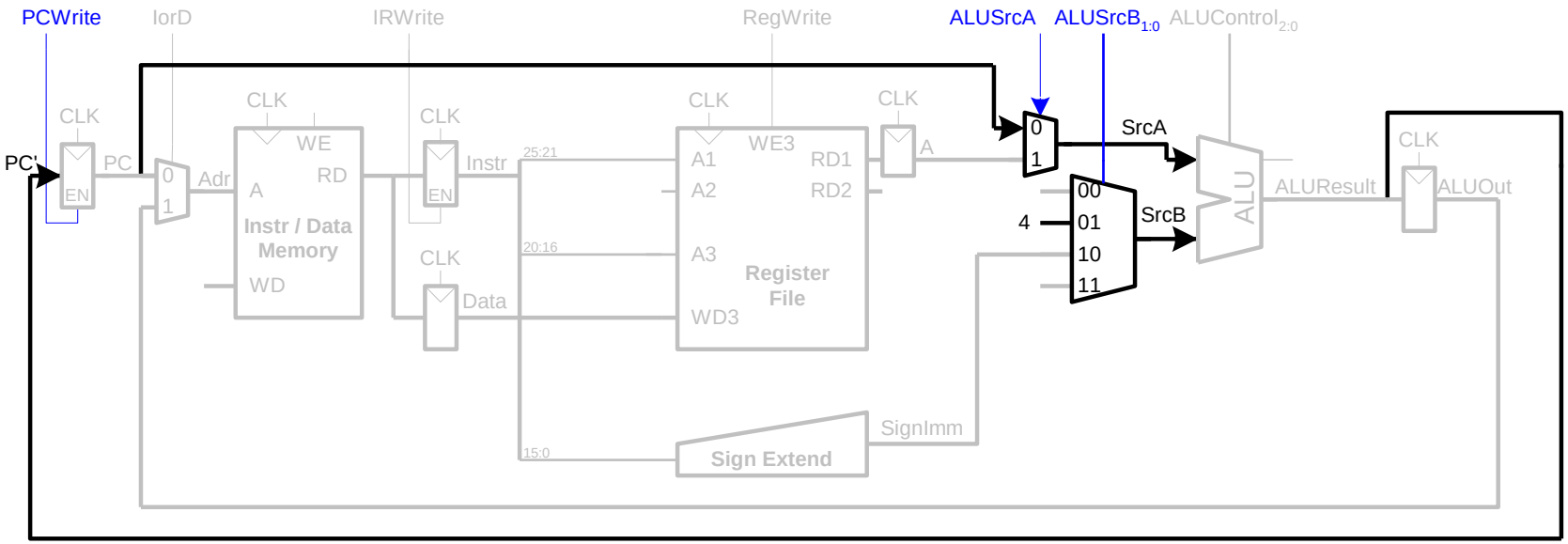


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

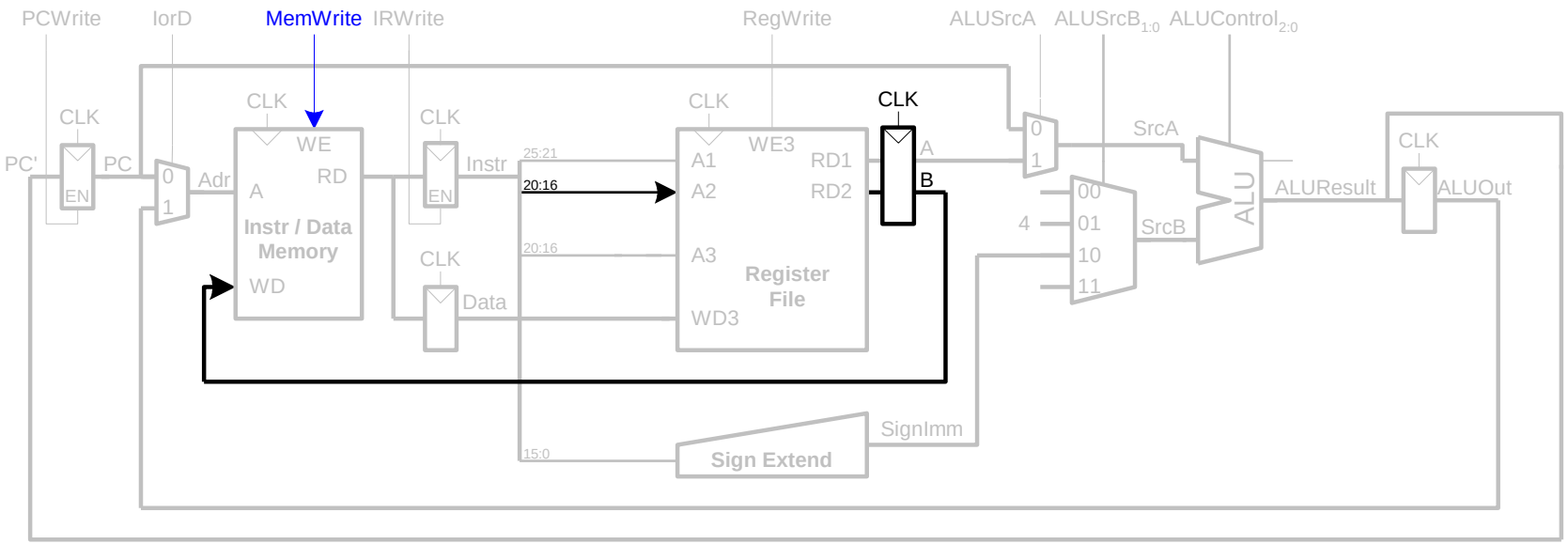
Multi-Cycle Datapath: increment PC

Now we can use **a single ALU** to both increment PC
and do address calculation or arithmetic operations
(in different clock cycles)



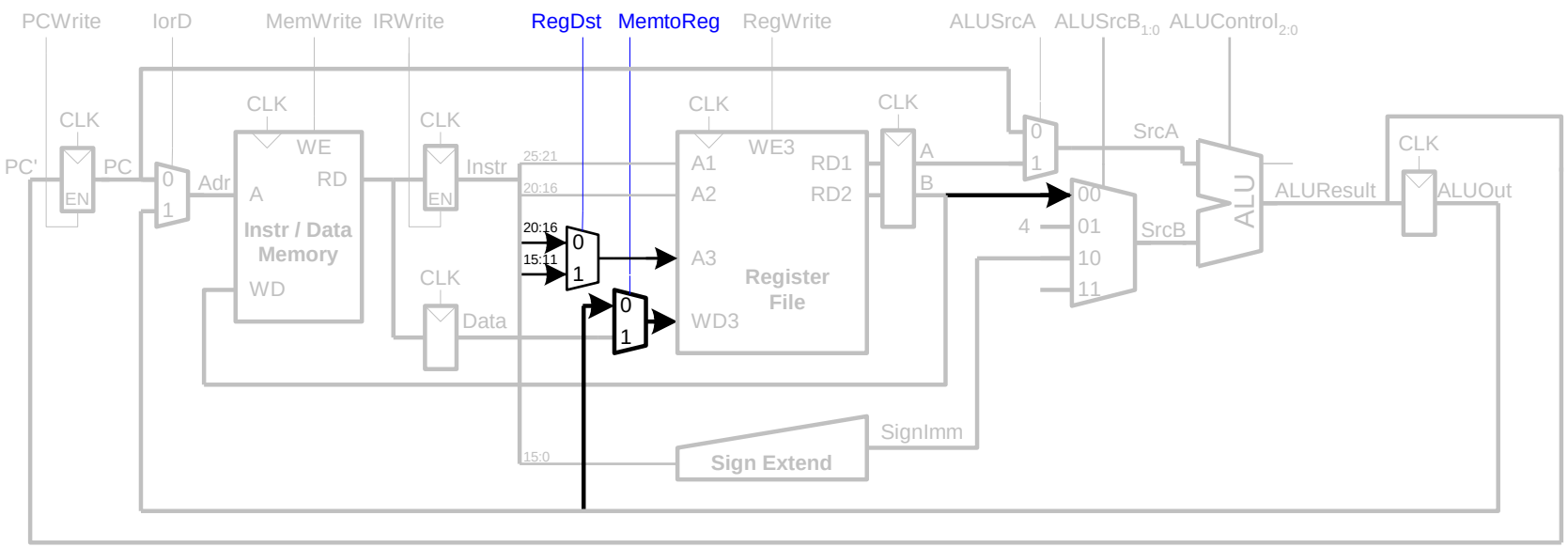
Multi-Cycle Datapath: sw

- **Write data in rt to memory**



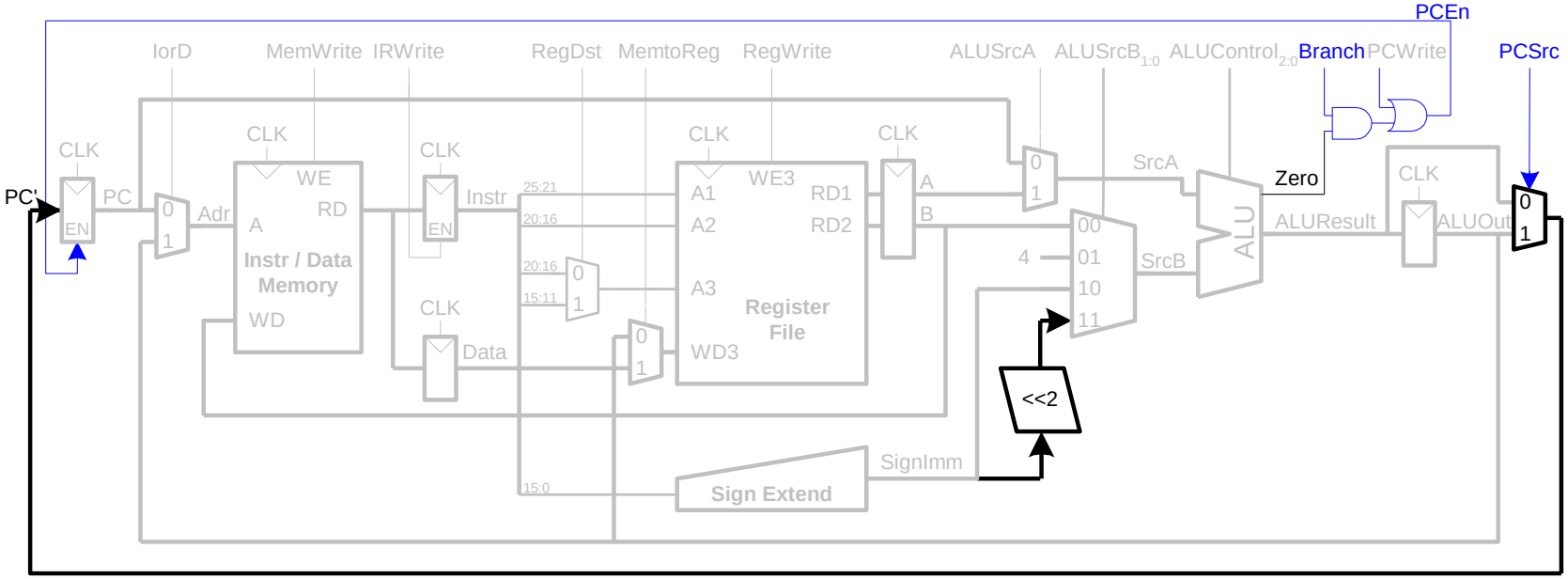
Multi-Cycle Datapath: R-type Instructions

- **Read from rs and rt**
  - Write ALUOut to register file
  - Write to rd (instead of rt)

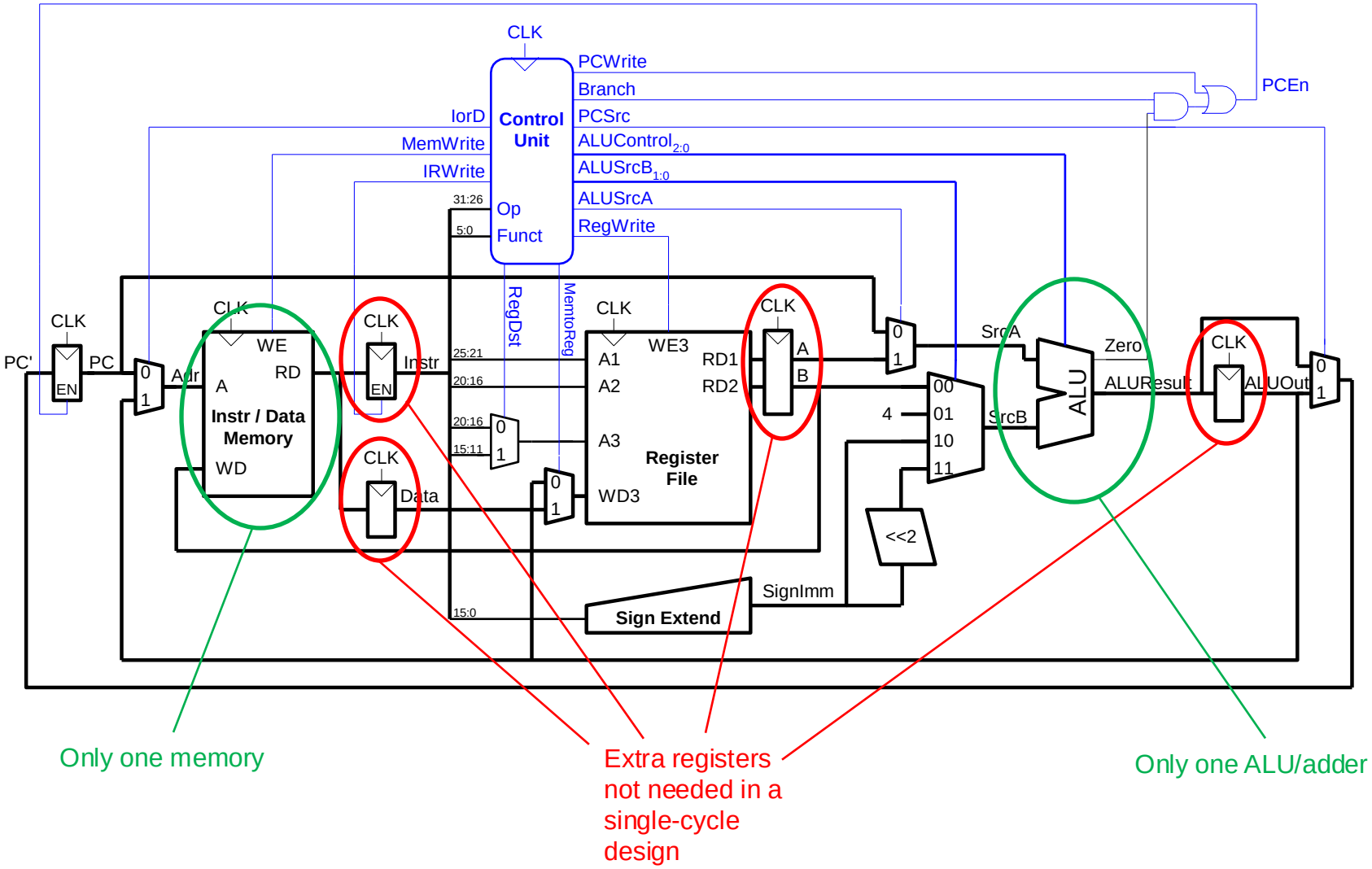


Multi-Cycle Datapath: beq

- **Determine whether values in rs and rt are equal**
  - Calculate branch target address:
    Target Address = (sign-extended immediate << 2) + (PC+4)

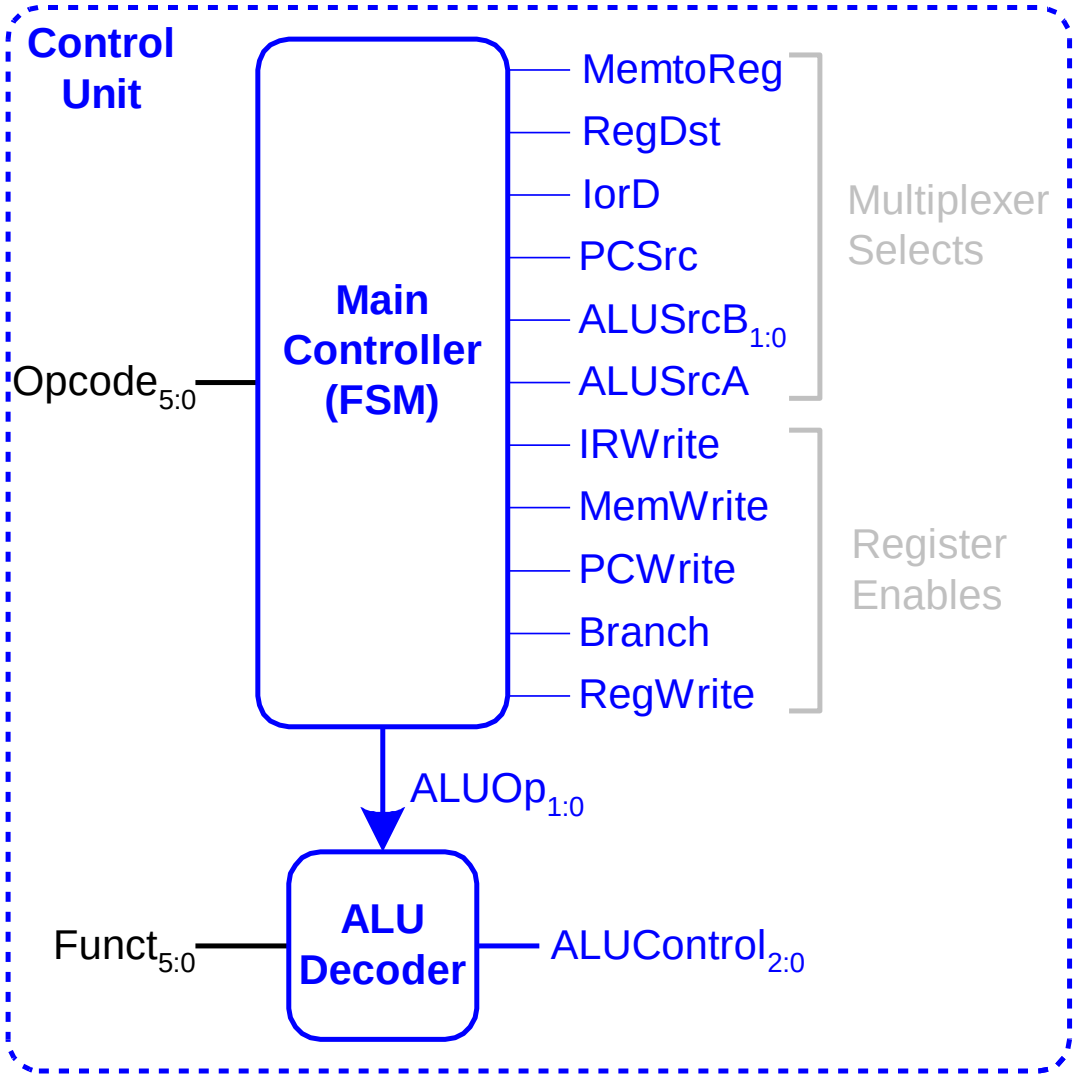


Complete Multi-Cycle Processor

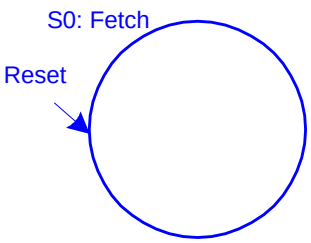


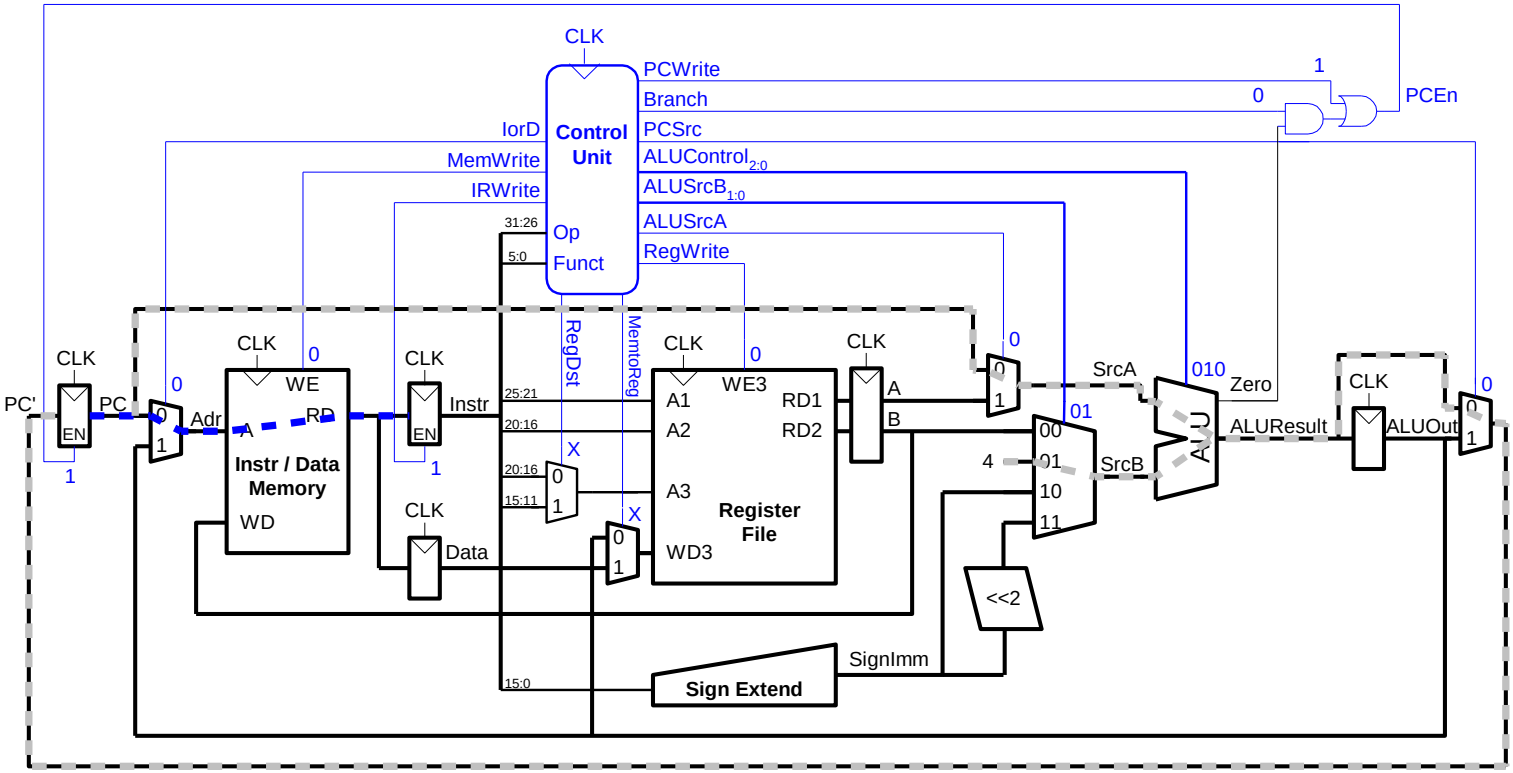
Let's Construct the Multi-Cycle Control Logic

Control Unit

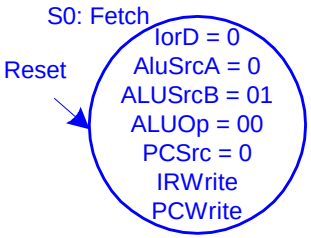


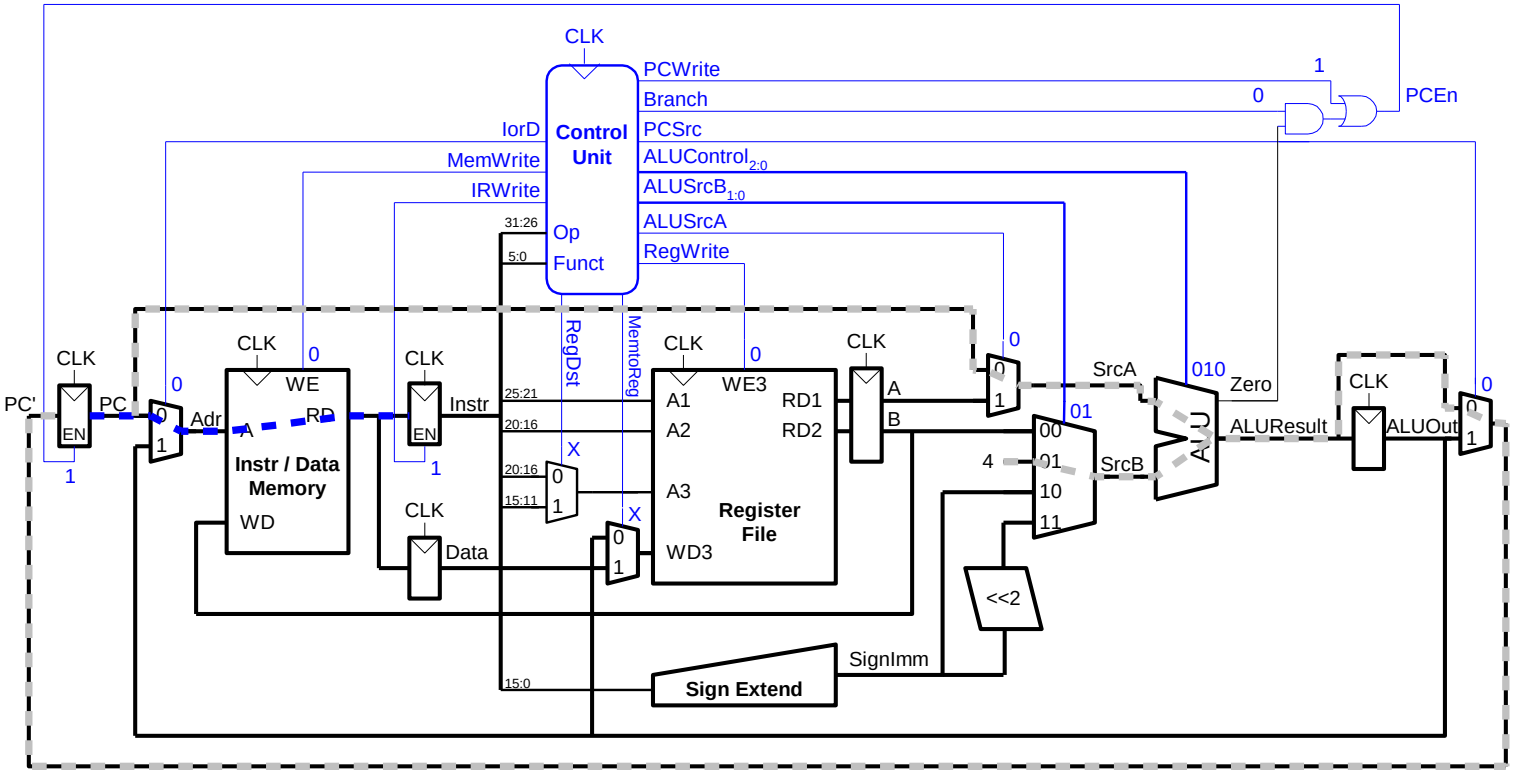
Main Controller FSM: Fetch



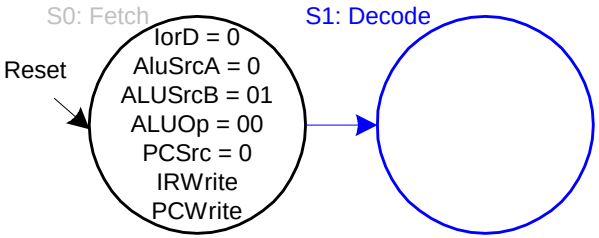


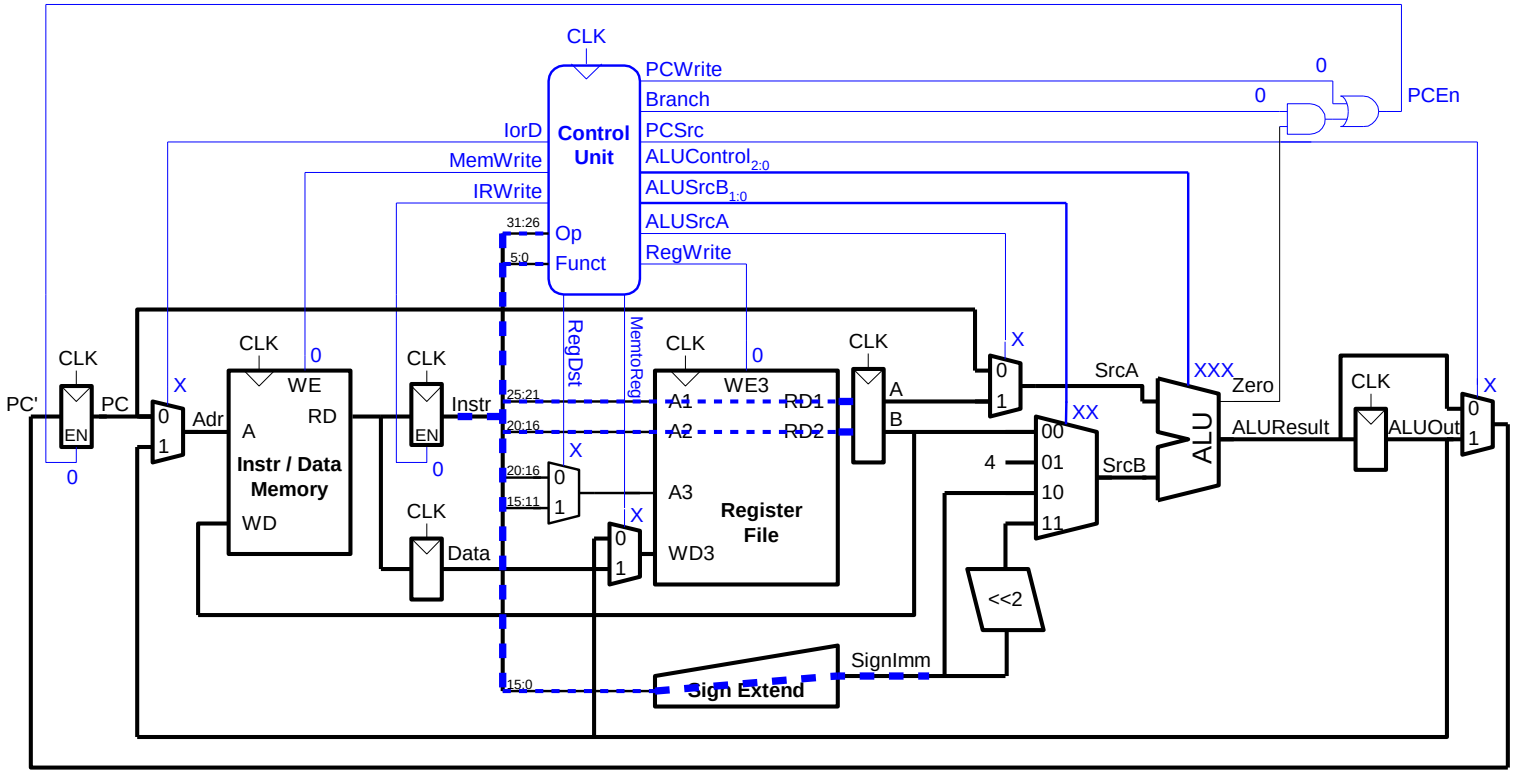
Main Controller FSM: Fetch



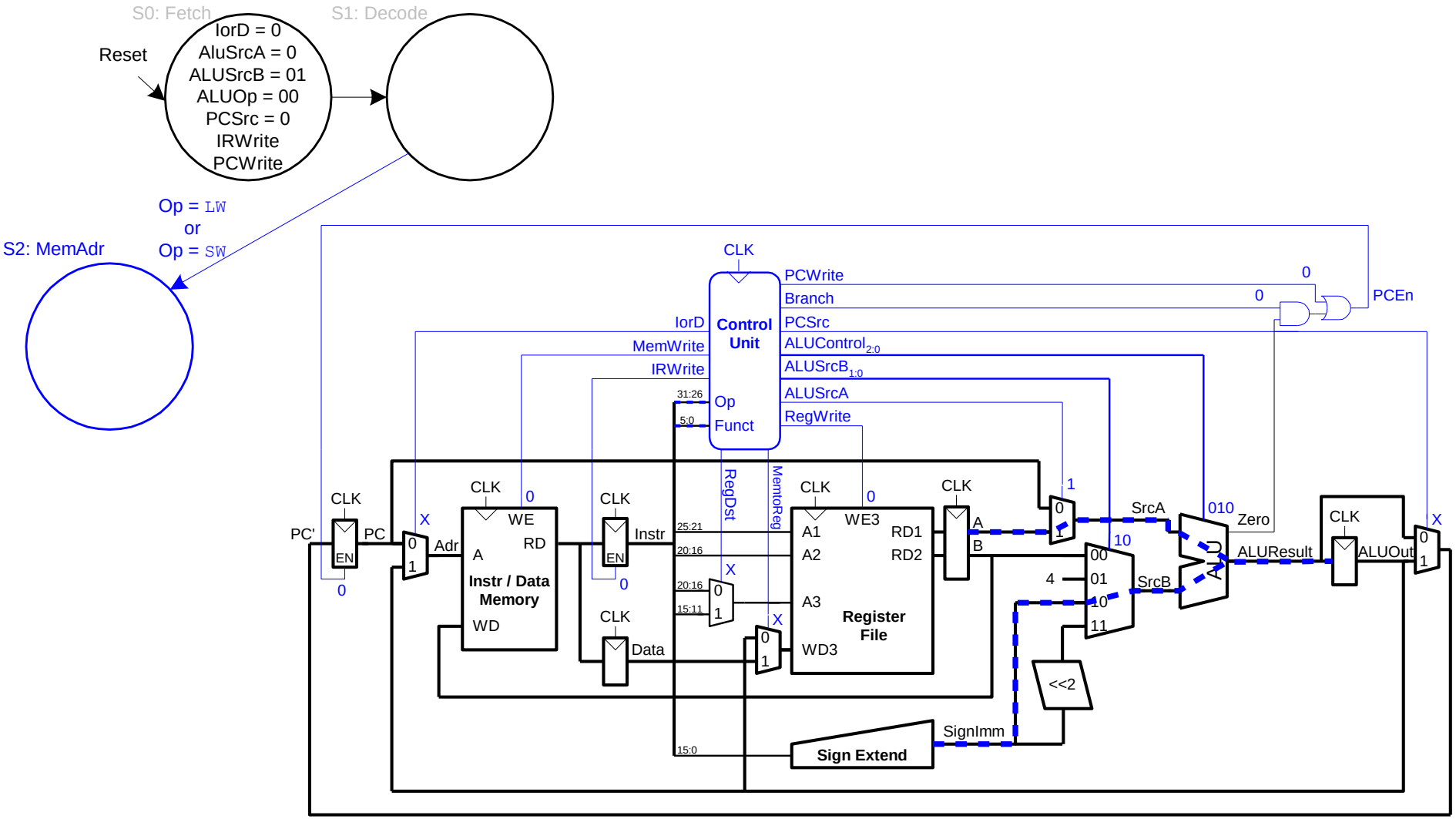


Main Controller FSM: Decode

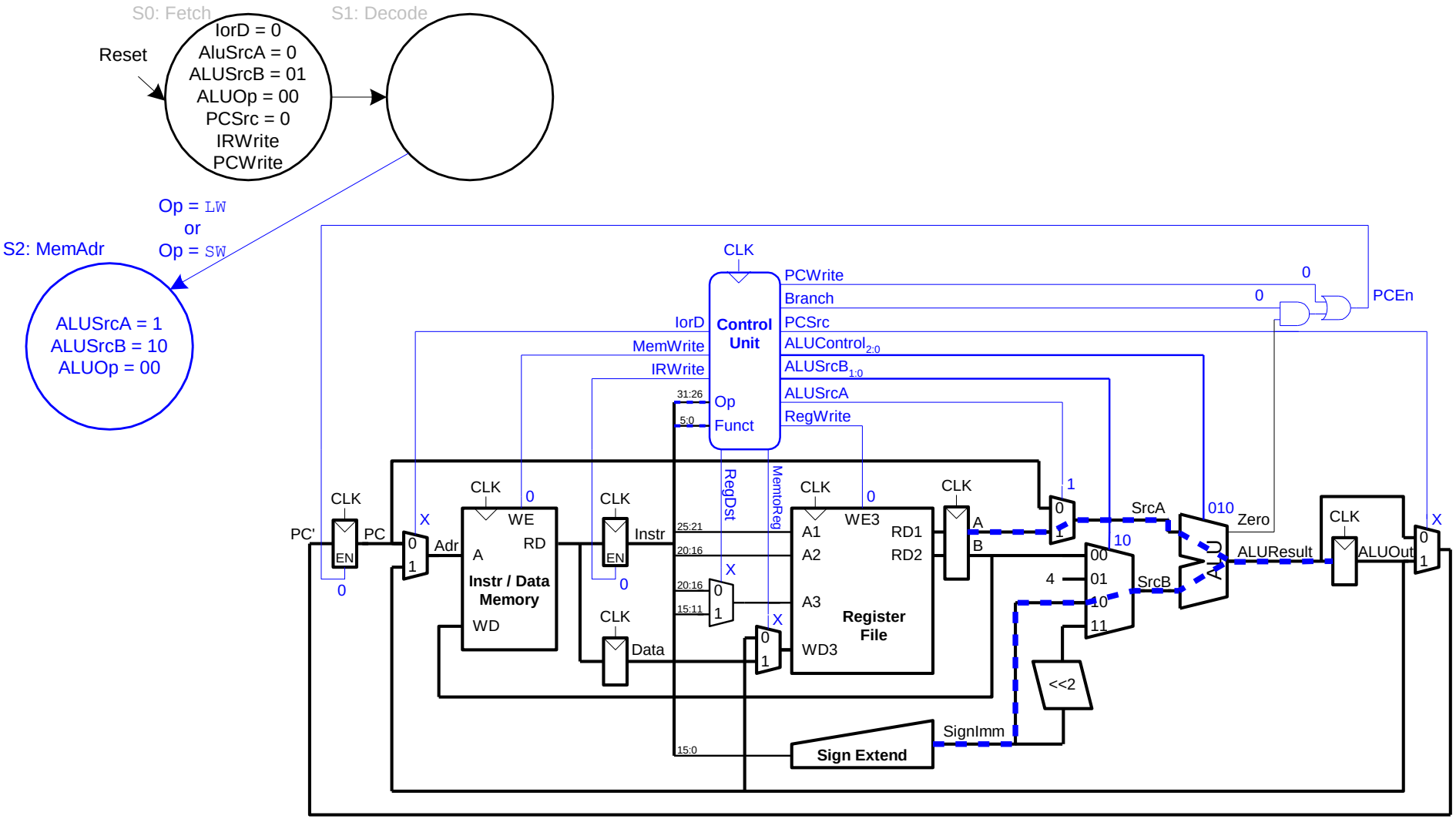




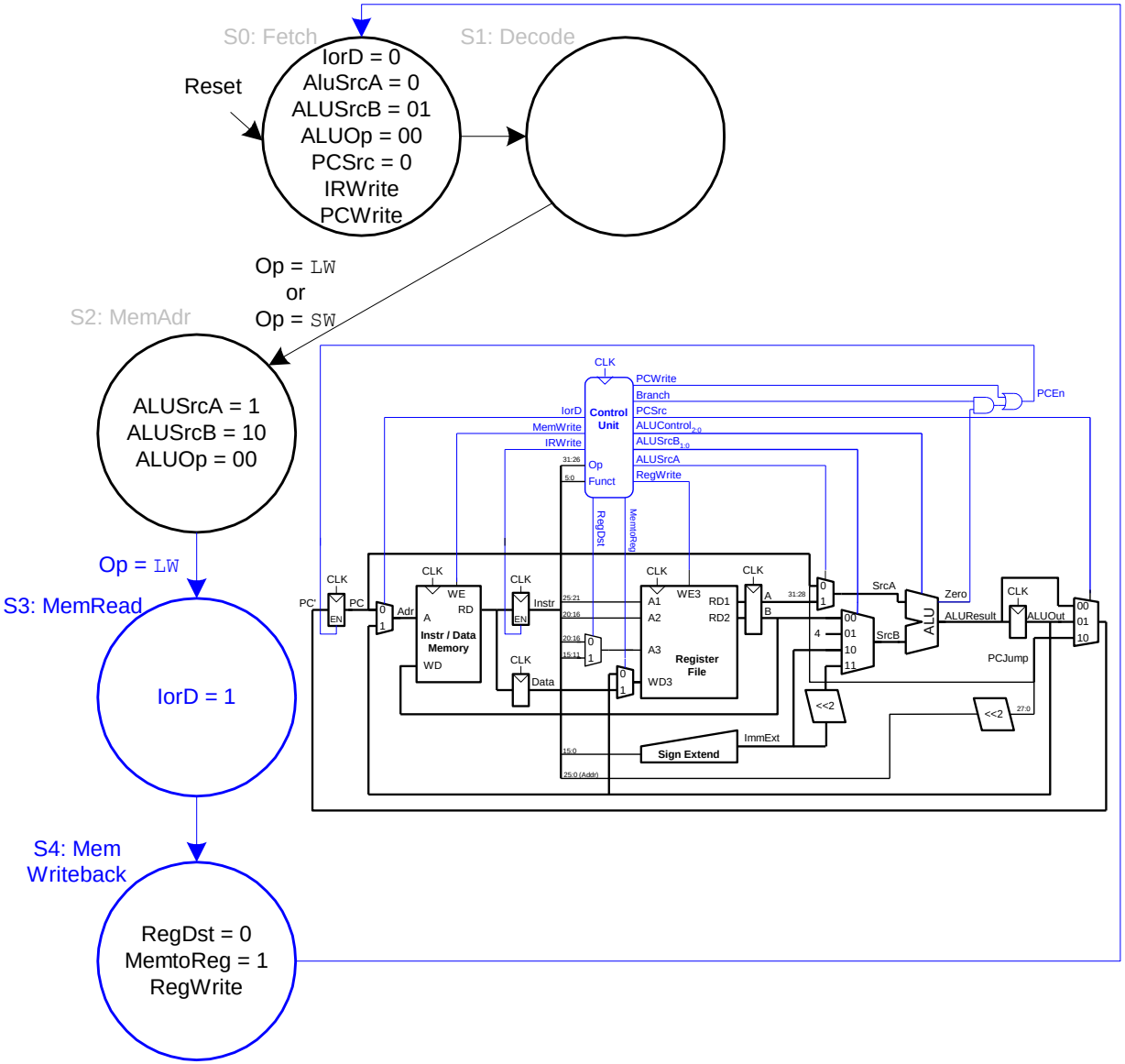
Main Controller FSM: Address Calculation



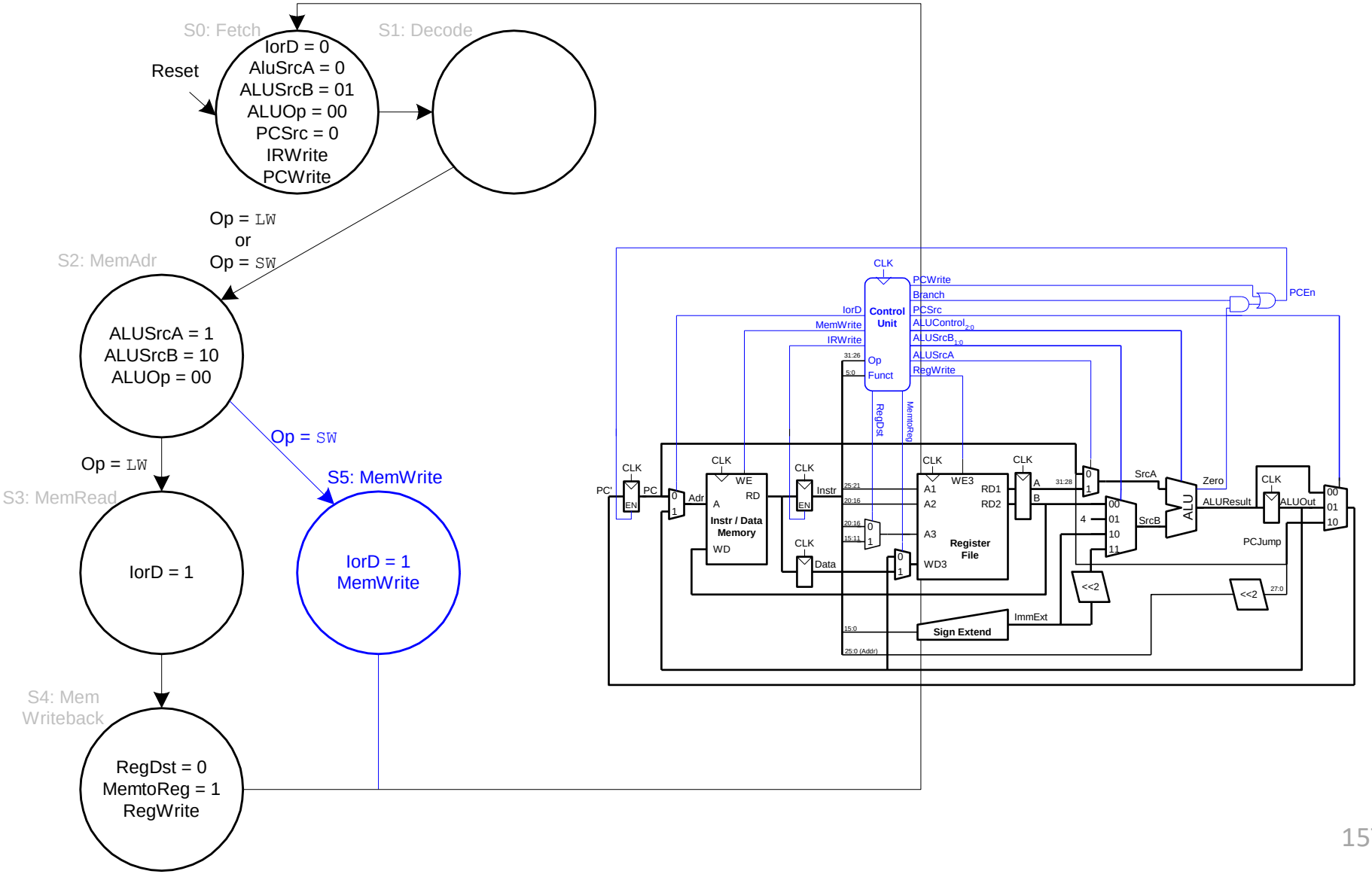
Main Controller FSM: Address Calculation



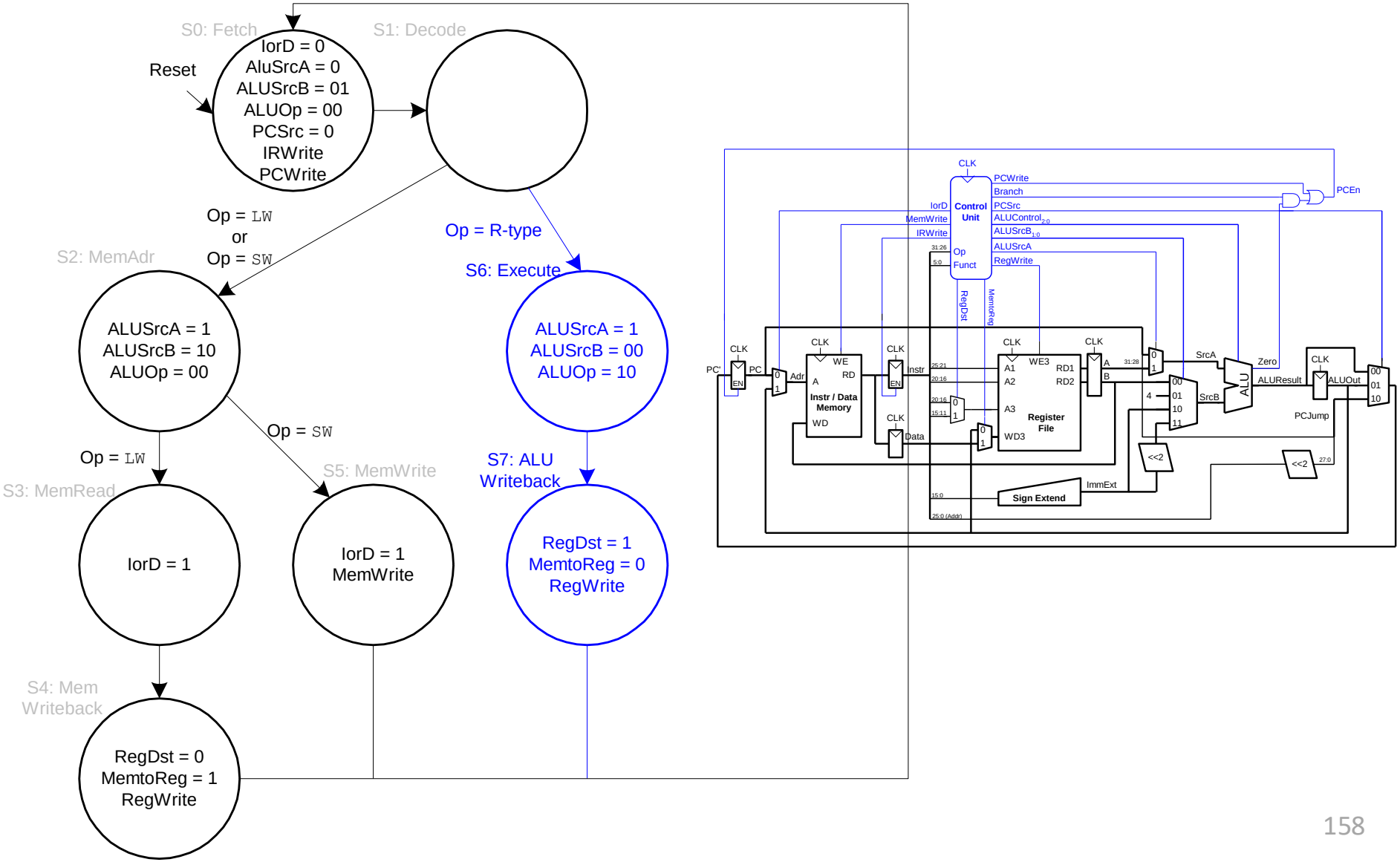
Main Controller FSM: lw



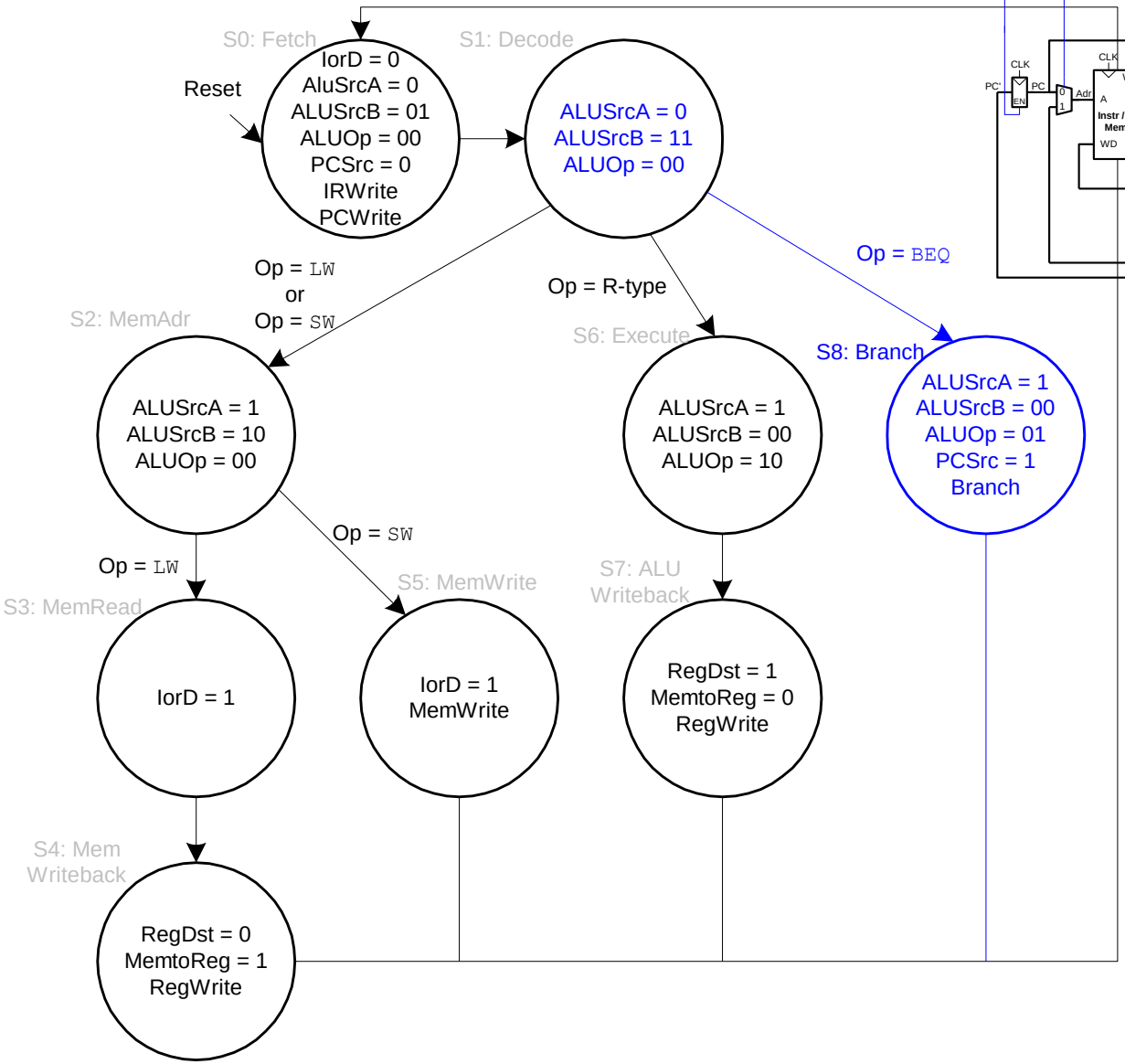
Main Controller FSM: sw

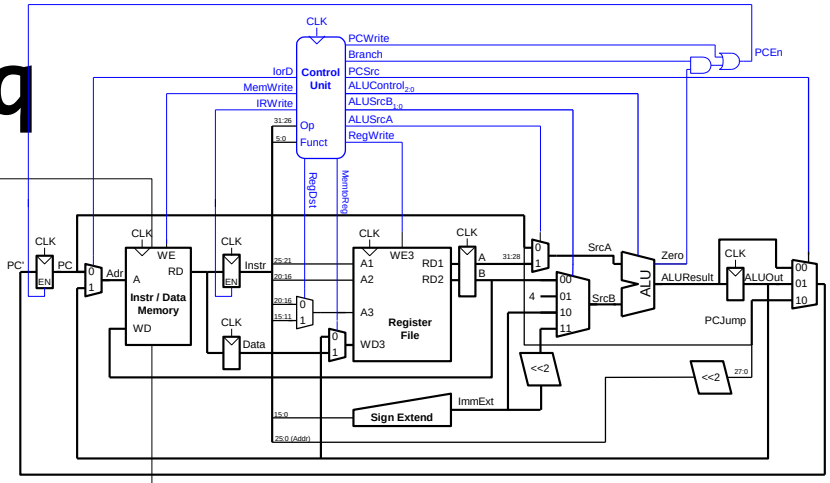


Main Controller FSM: R-Type

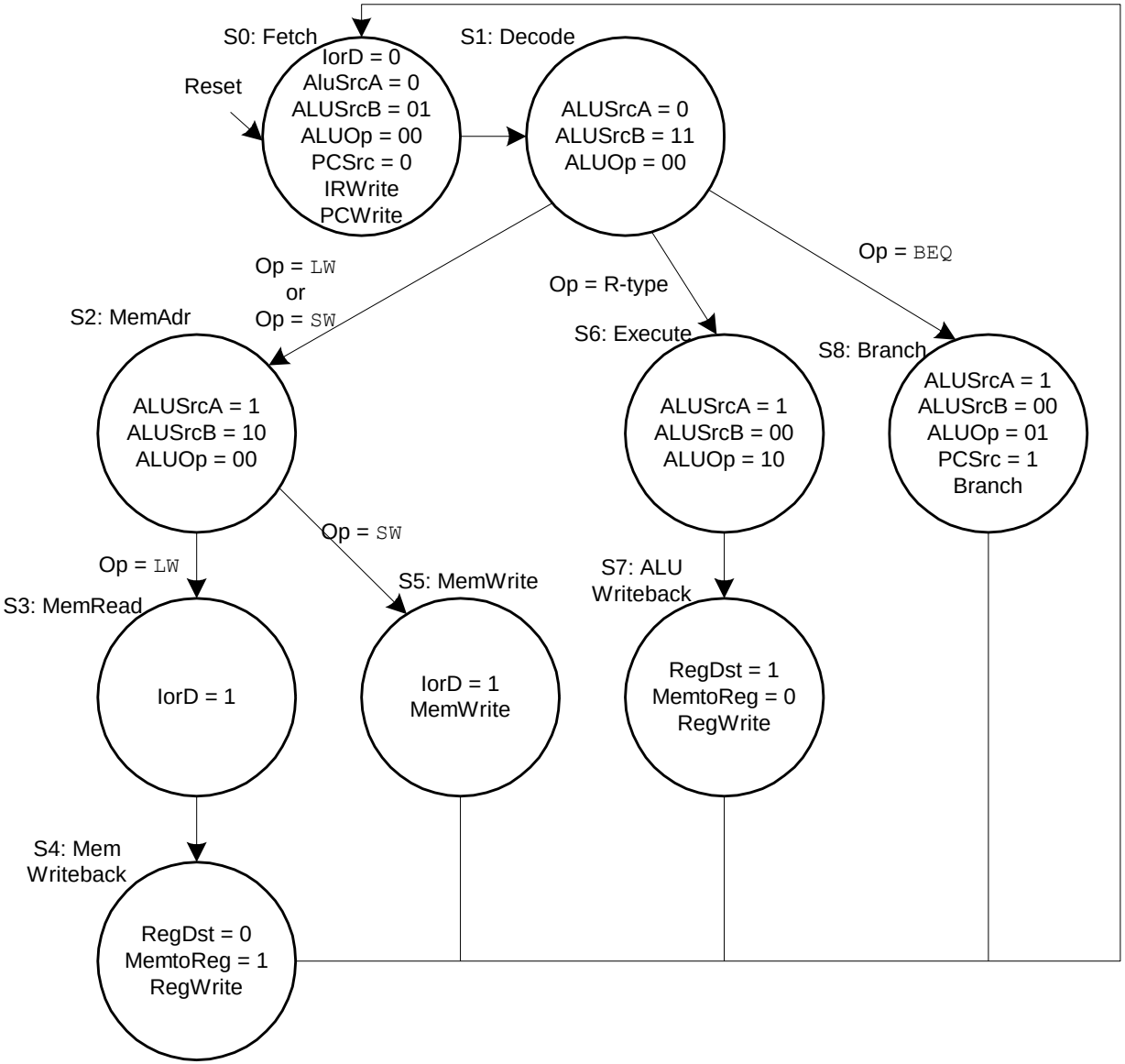


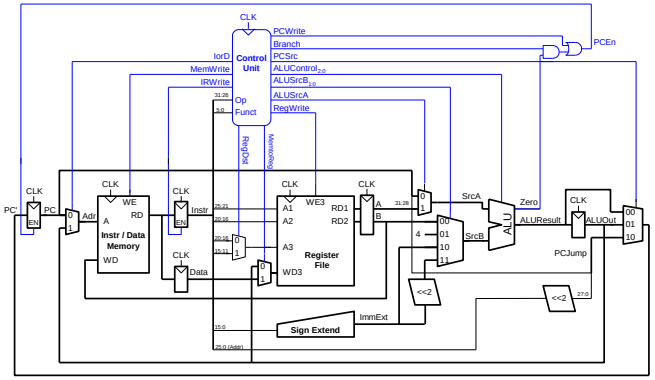
Main Controller FSM: beq



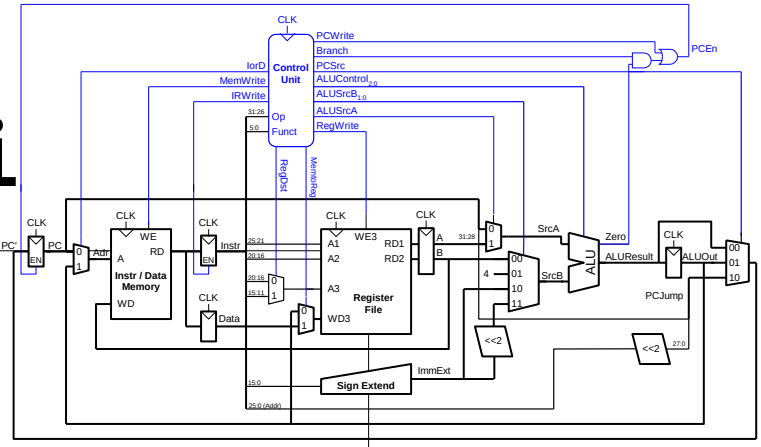


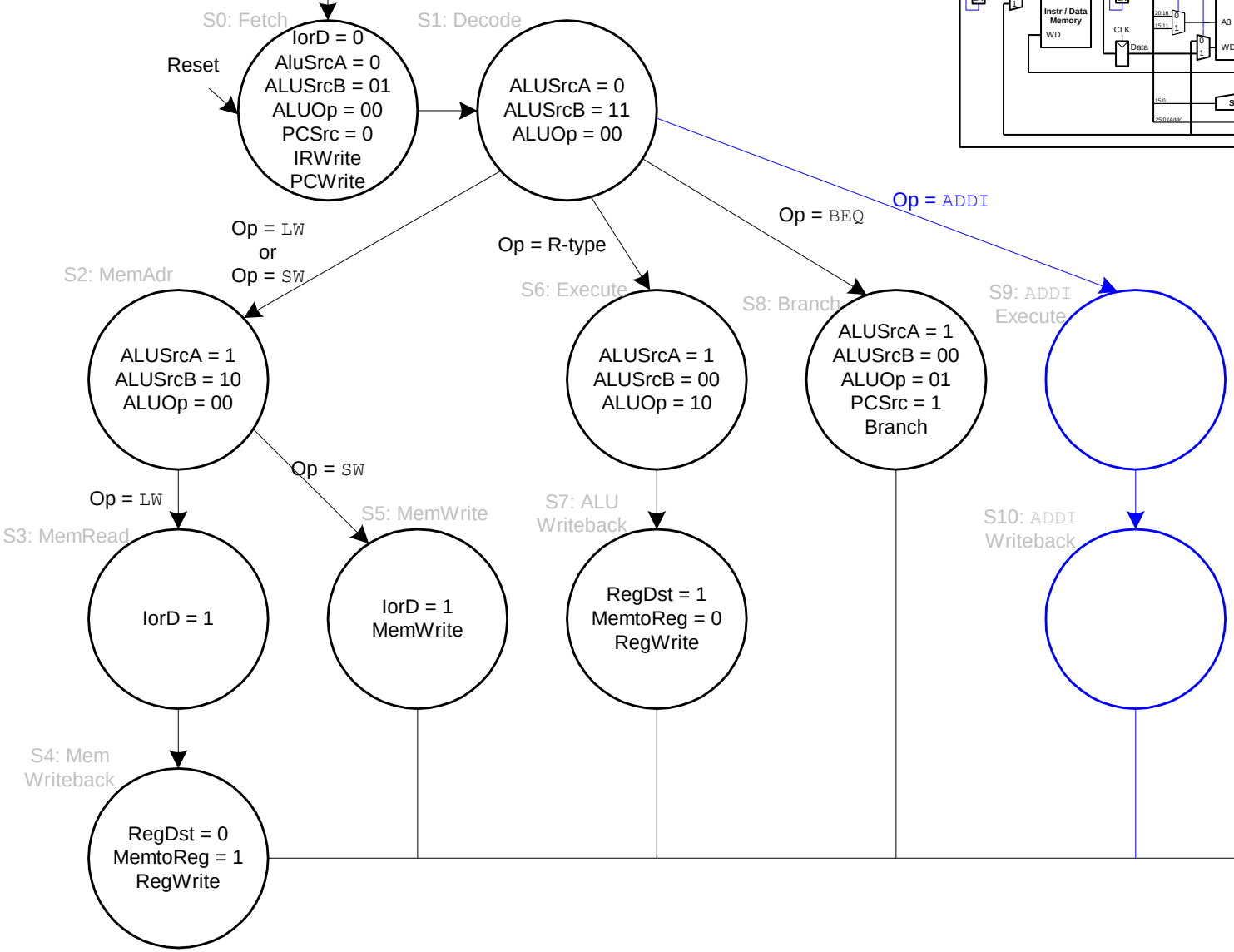
Complete Multi-Cycle Controller FSM



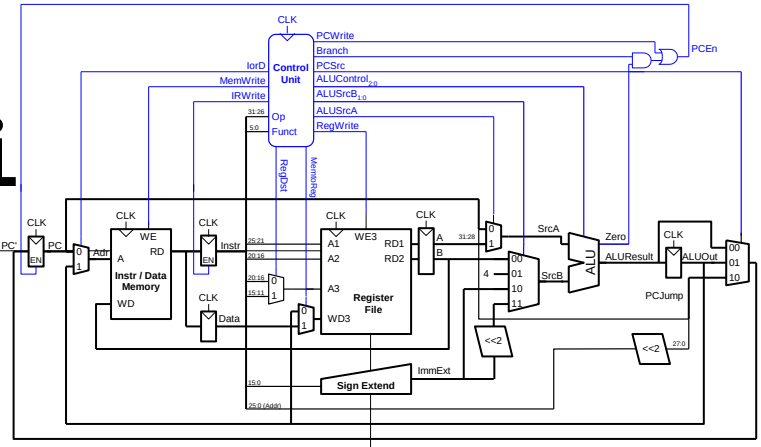


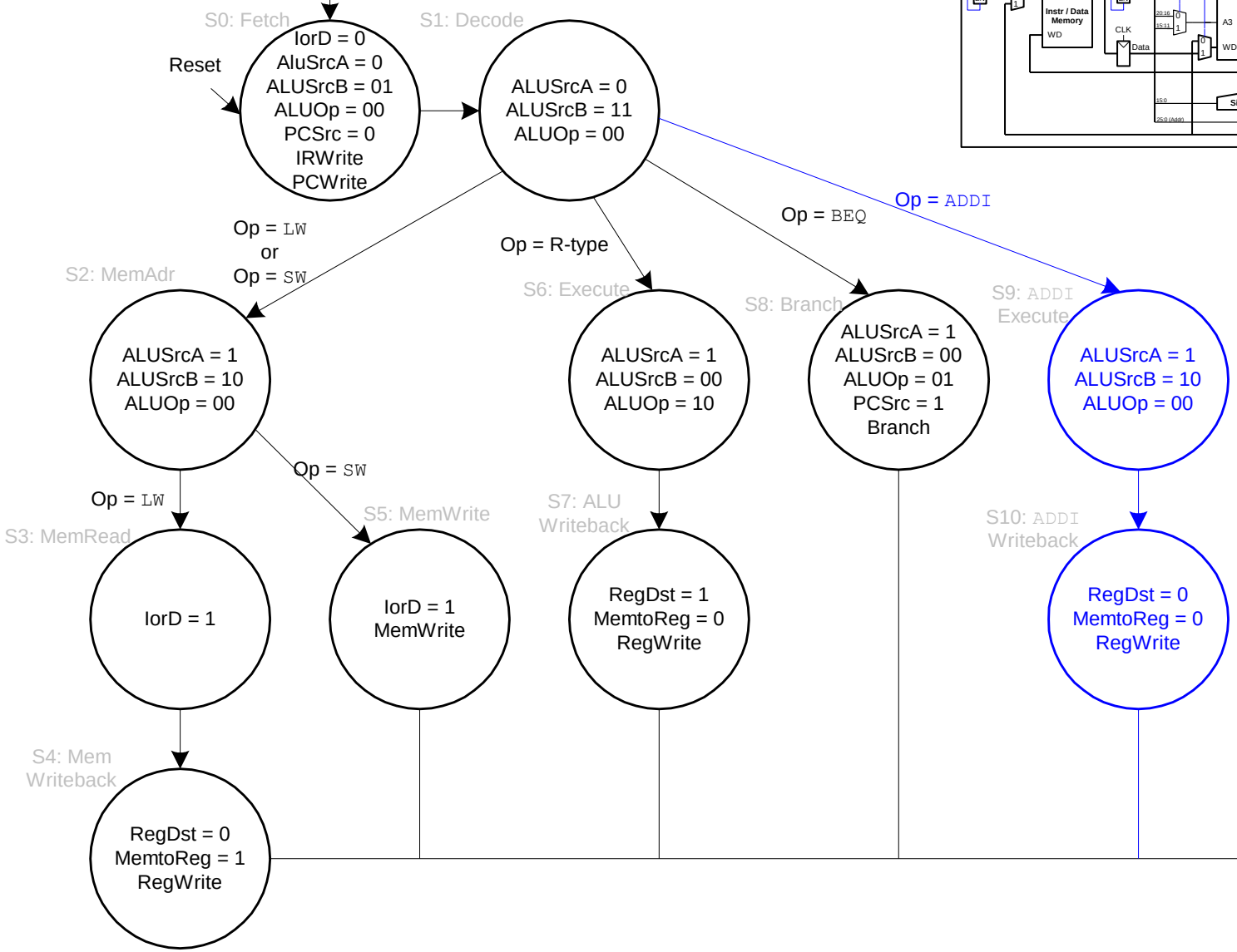
Main Controller FSM: addi



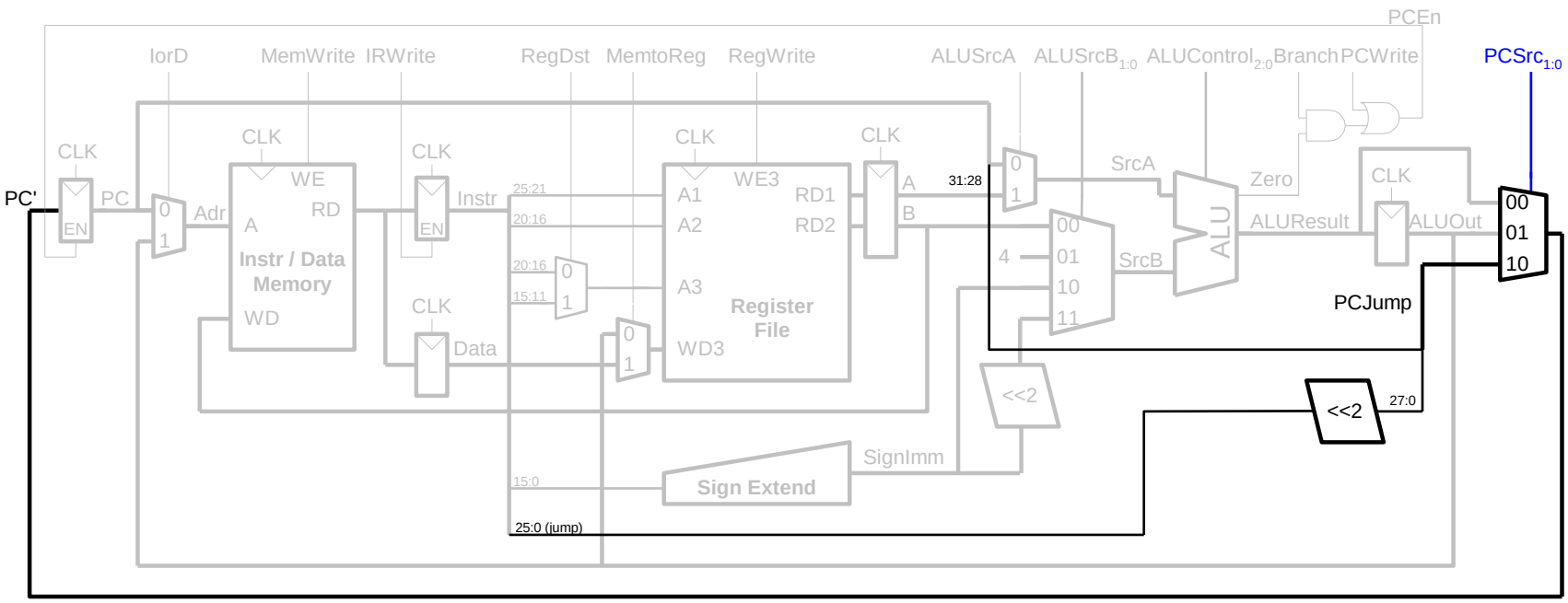


Main Controller FSM: addi

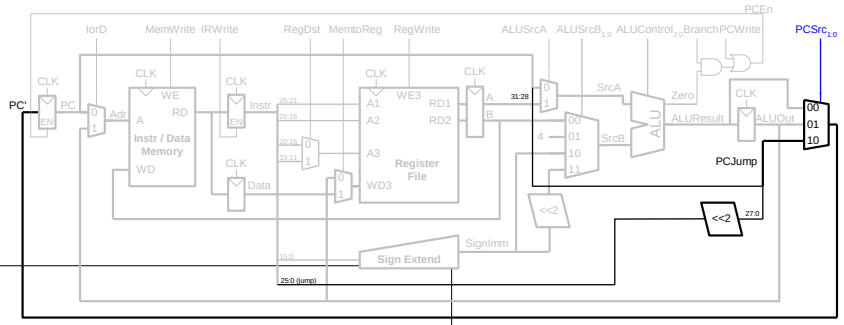


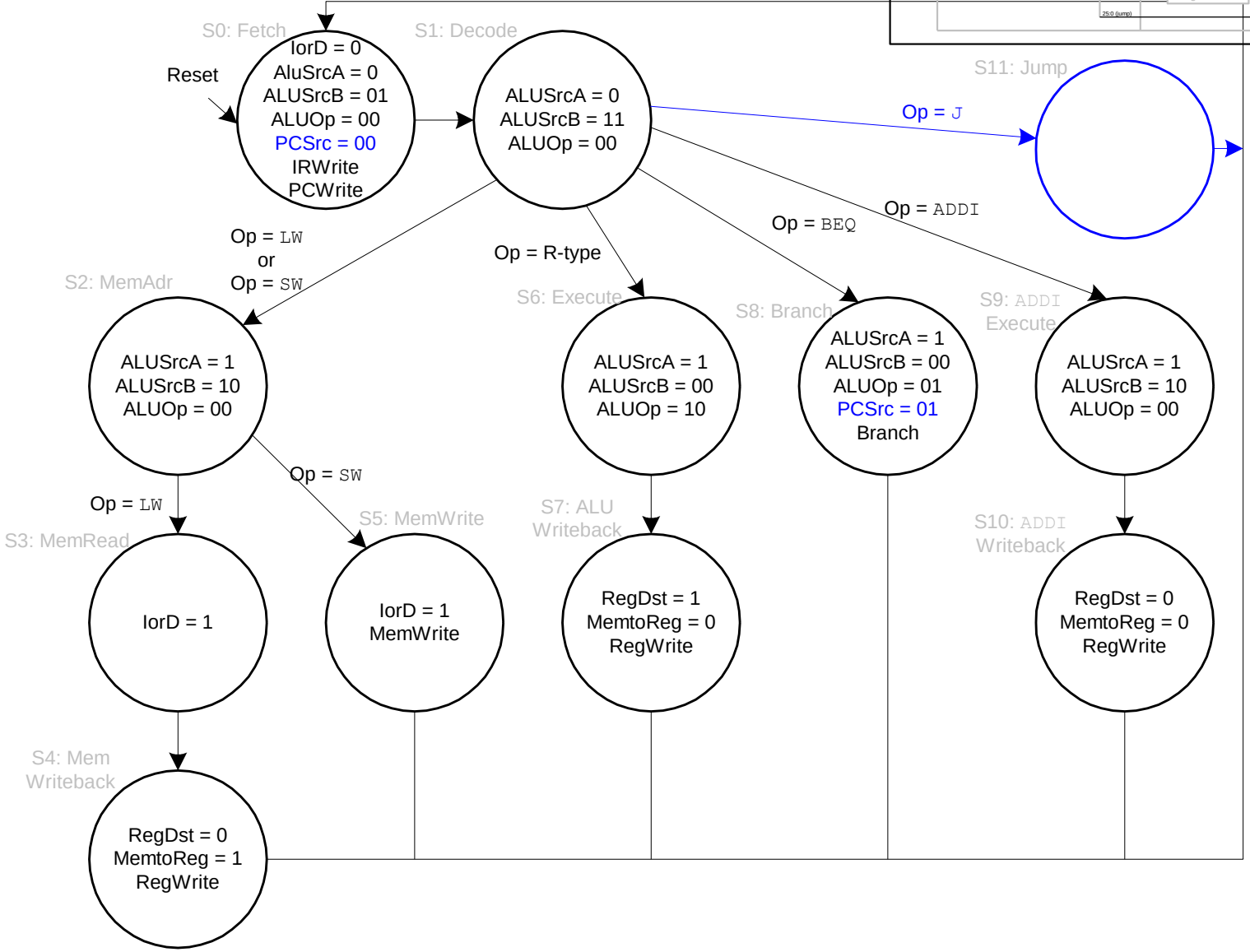


Extended Functionality: j

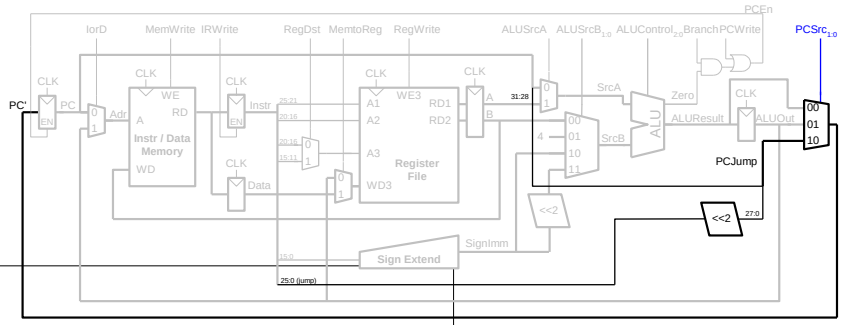


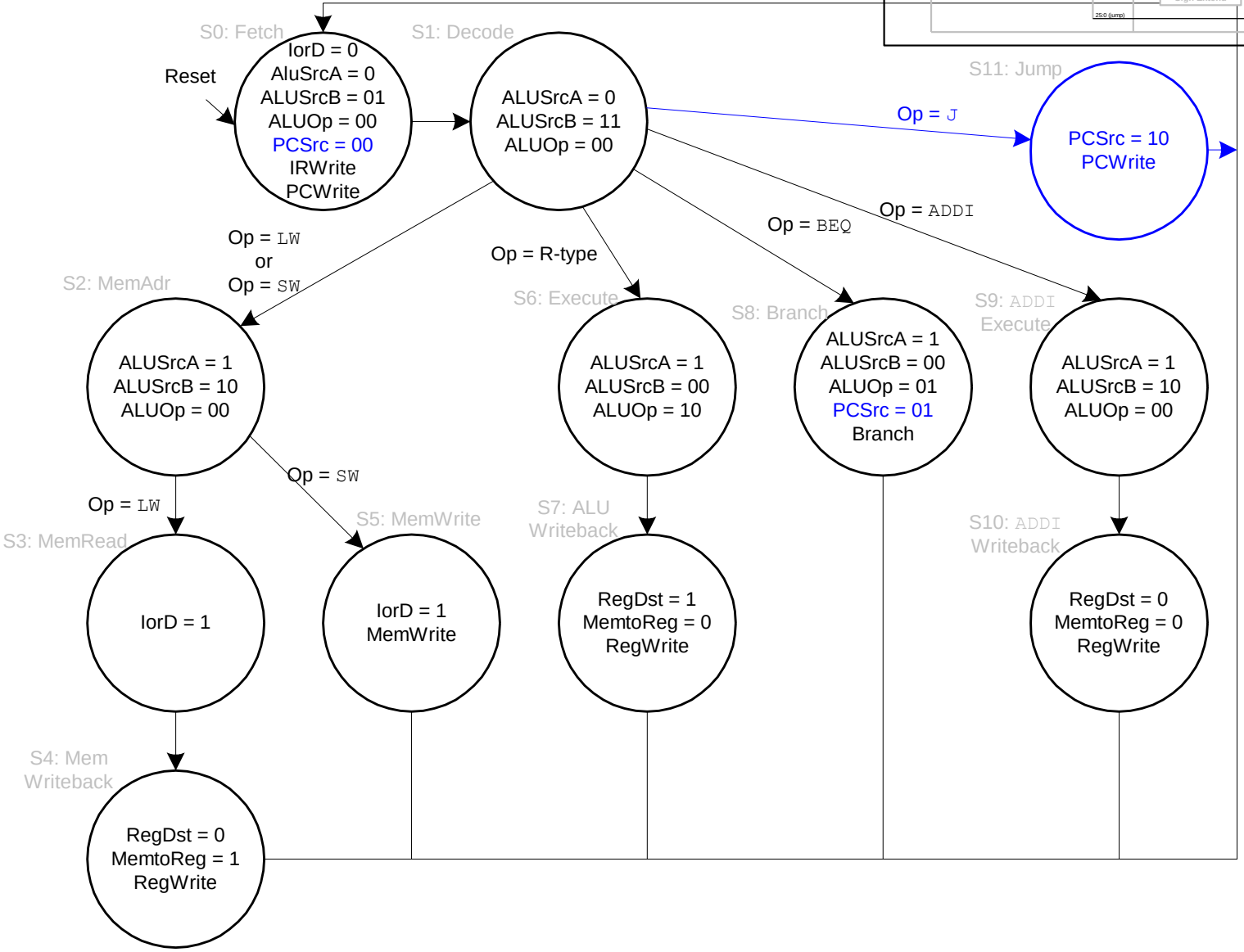
Control FSM: j





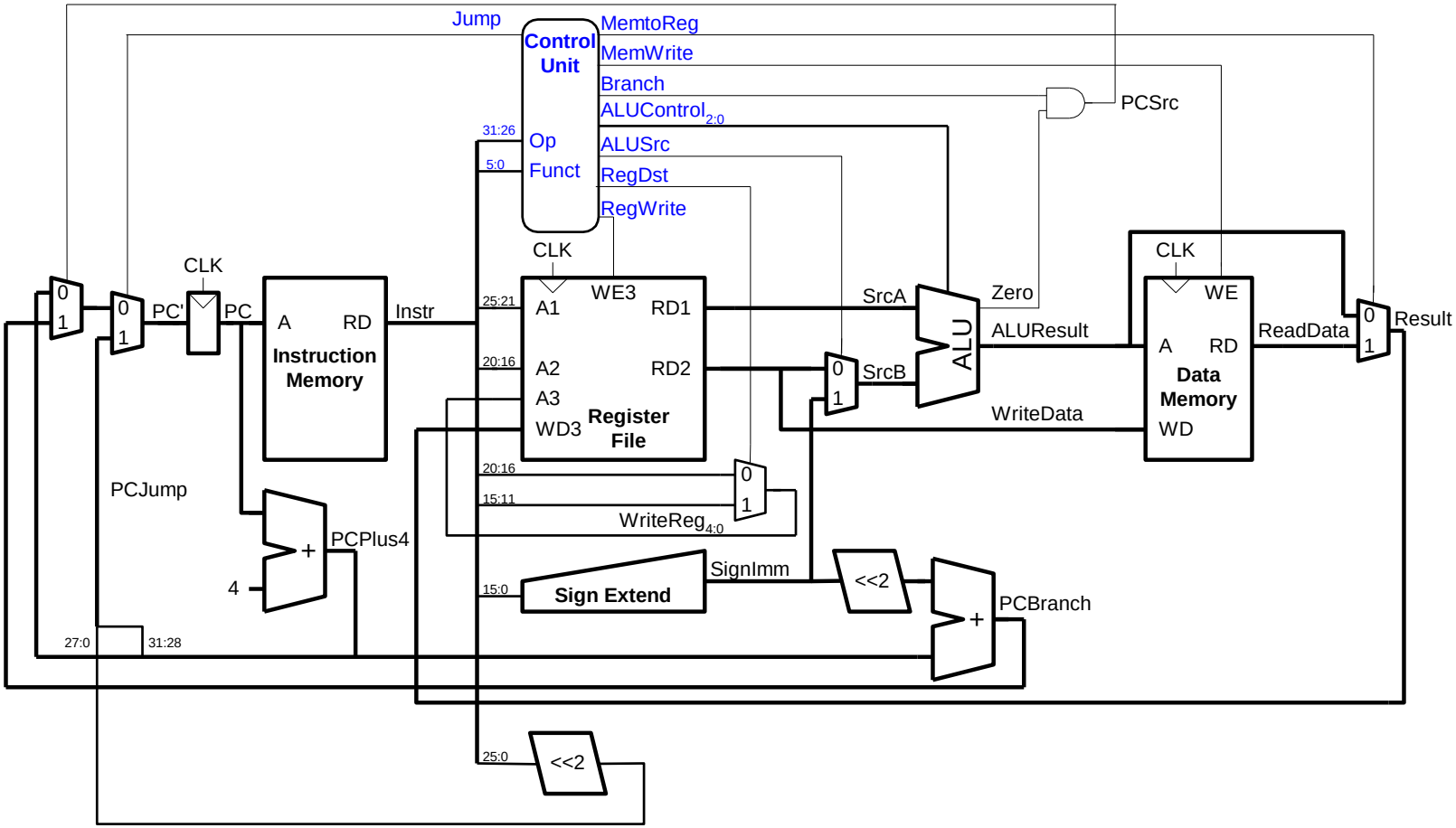
Control FSM: j





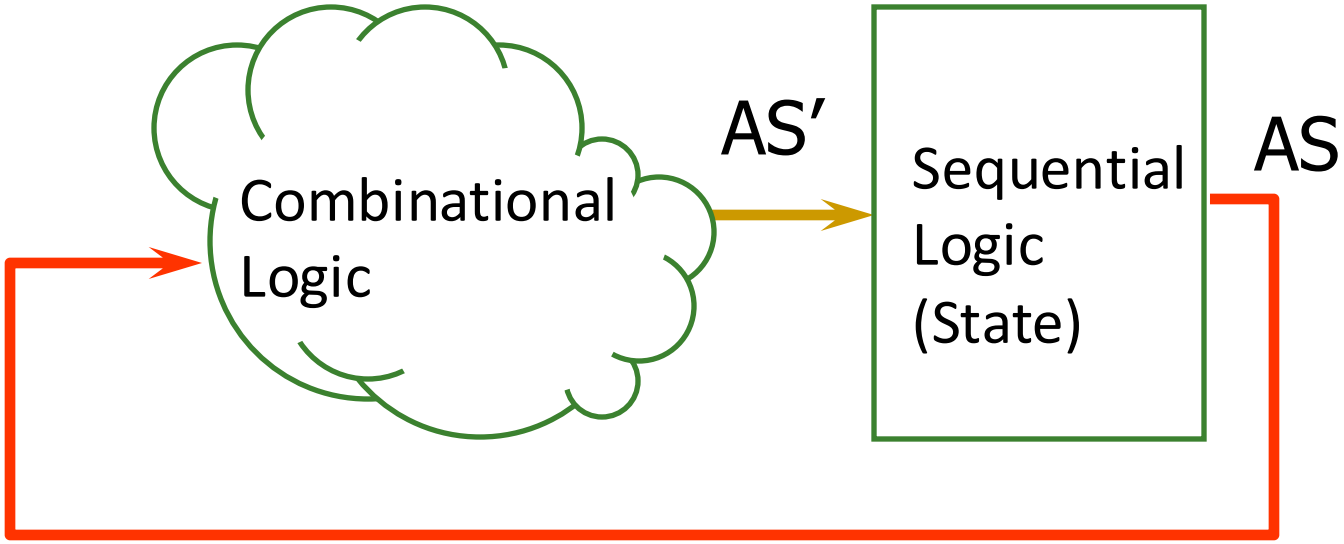
We Constructed a Multi-Cycle MIPS Microarchitecture

Review: Single-Cycle MIPS Microarchitecture



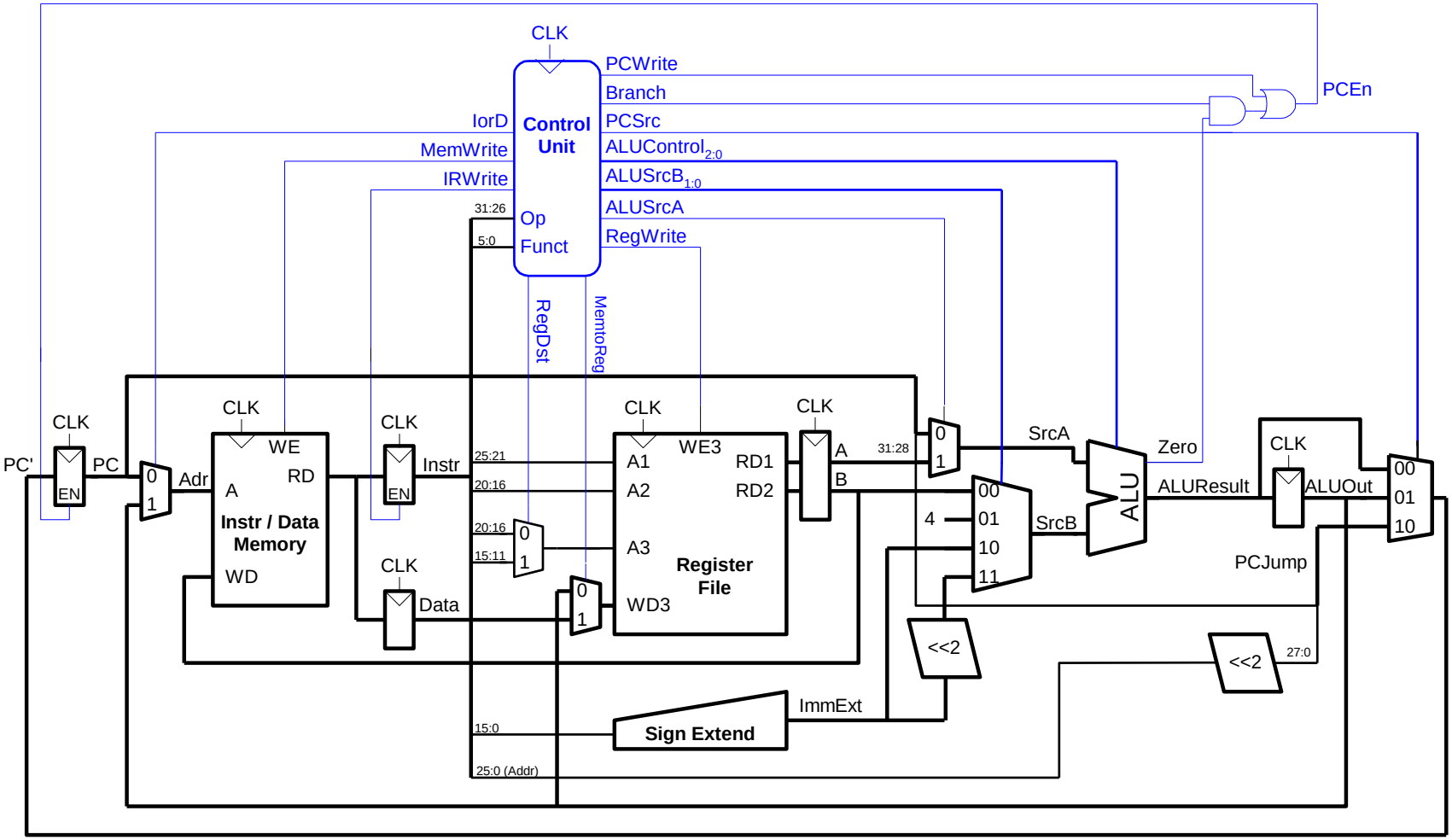
Review: Single-Cycle MIPS FSM

- Single-cycle machine



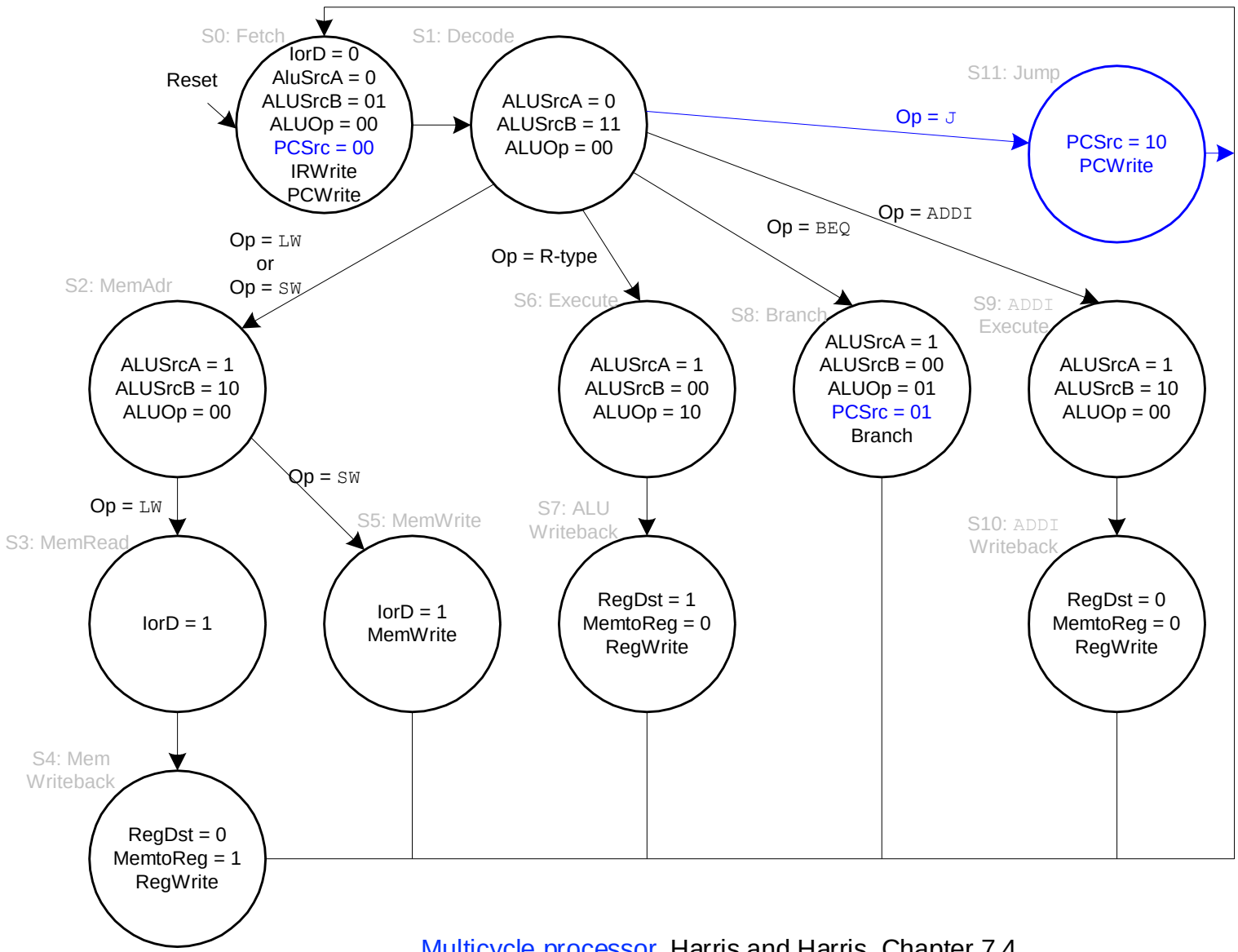
AS: Architectural State

Review: Multi-Cycle MIPS Microarchitecture



Multicycle processor. Harris and Harris, Chapter 7.4.

Review: Multi-Cycle MIPS FSM



What is the shortcoming of this design?

What does this design assume about memory?

Multicycle processor. Harris and Harris, Chapter 7.4.

What If Memory Takes > One Cycle?

- Stay in the same "memory access" state until memory returns the data
- "Memory Ready?" bit is an input to the control logic that determines the next state

Recall: Full State Machine for LC-3b

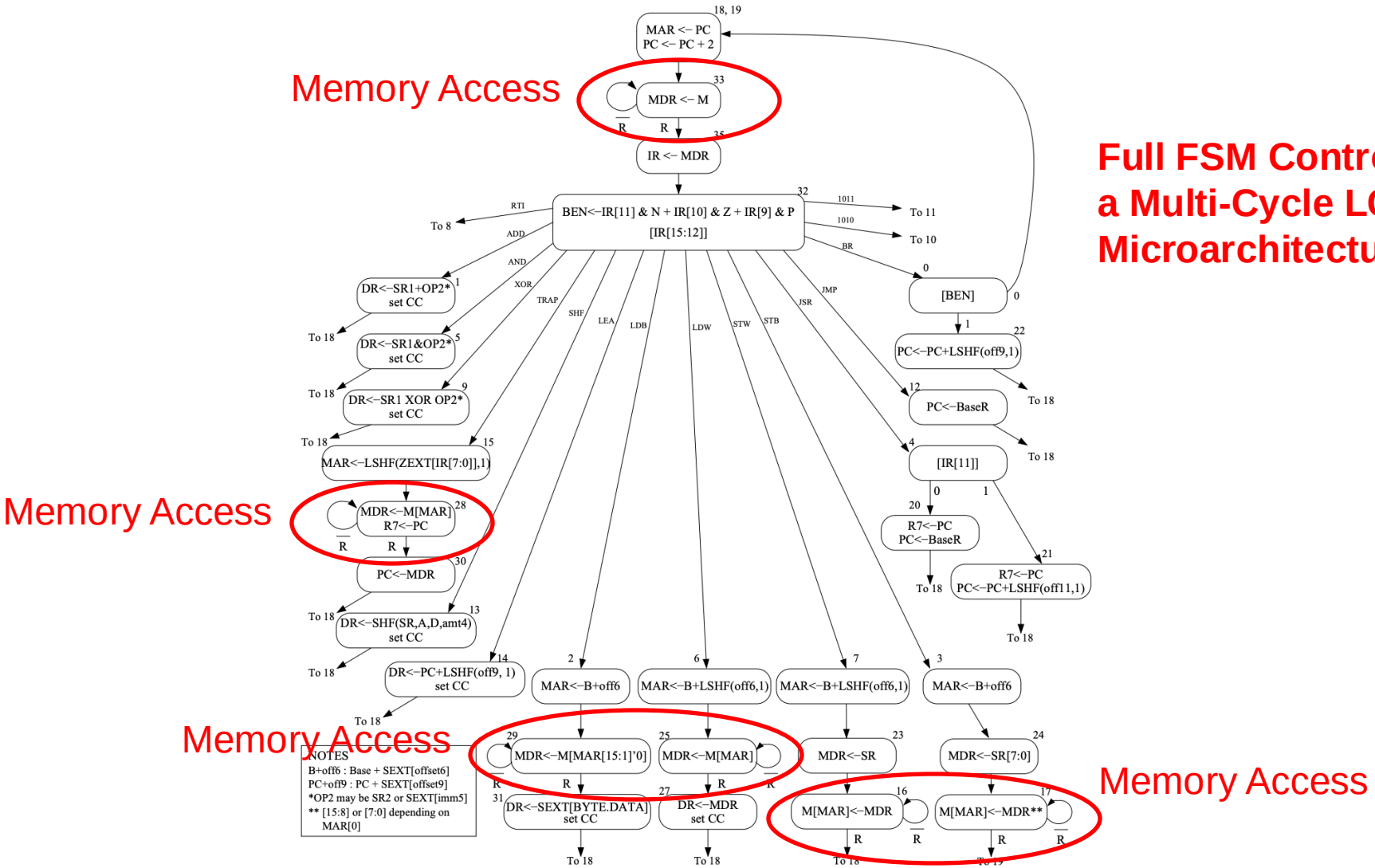


Figure C.2: A state machine for the LC-3b

Full FSM Controlling a Multi-Cycle LC-3b Microarchitecture

https://safari.ethz.ch/digitaltechnik/spring2022/lib/exe/fetch.php?media=pp-appendixc.pdf

Another Example: **Microprogrammed** Multi-Cycle Microarchitecture

Recall: An Elegant Multi-Cycle Processor Design

- Maurice Wilkes, "The Best Way to Design an Automatic Calculating Machine," Manchester Univ. Computer Inaugural Conf., 1951.

THE BEST WAY TO DESIGN AN AUTOMATIC CALCULATING MACHINE

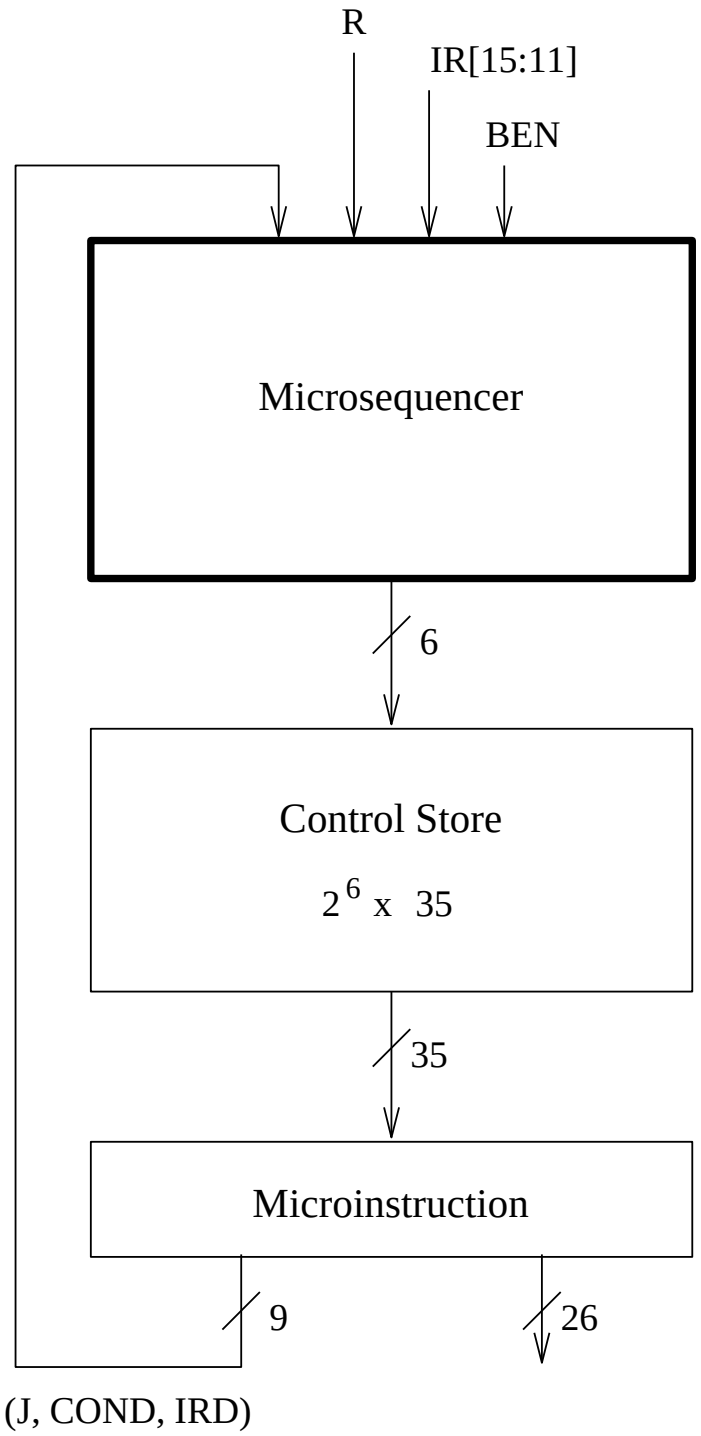
By M. V. Wilkes, M.A., Ph.D., F.R.A.S.

- An elegant implementation:
  - The concept of microcoded/microprogrammed machines

Microprogrammed Control Terminology

- Control signals associated with the current state
  - Microinstruction
- Act of transitioning from one state to another
  - Determining the next state and the microinstruction for the next state
  - Microsequencing
- Control store stores control signals for every possible state
  - Store for microinstructions for the entire FSM
- Microsequencer determines which set of control signals will be used in the next clock cycle (i.e., next state)

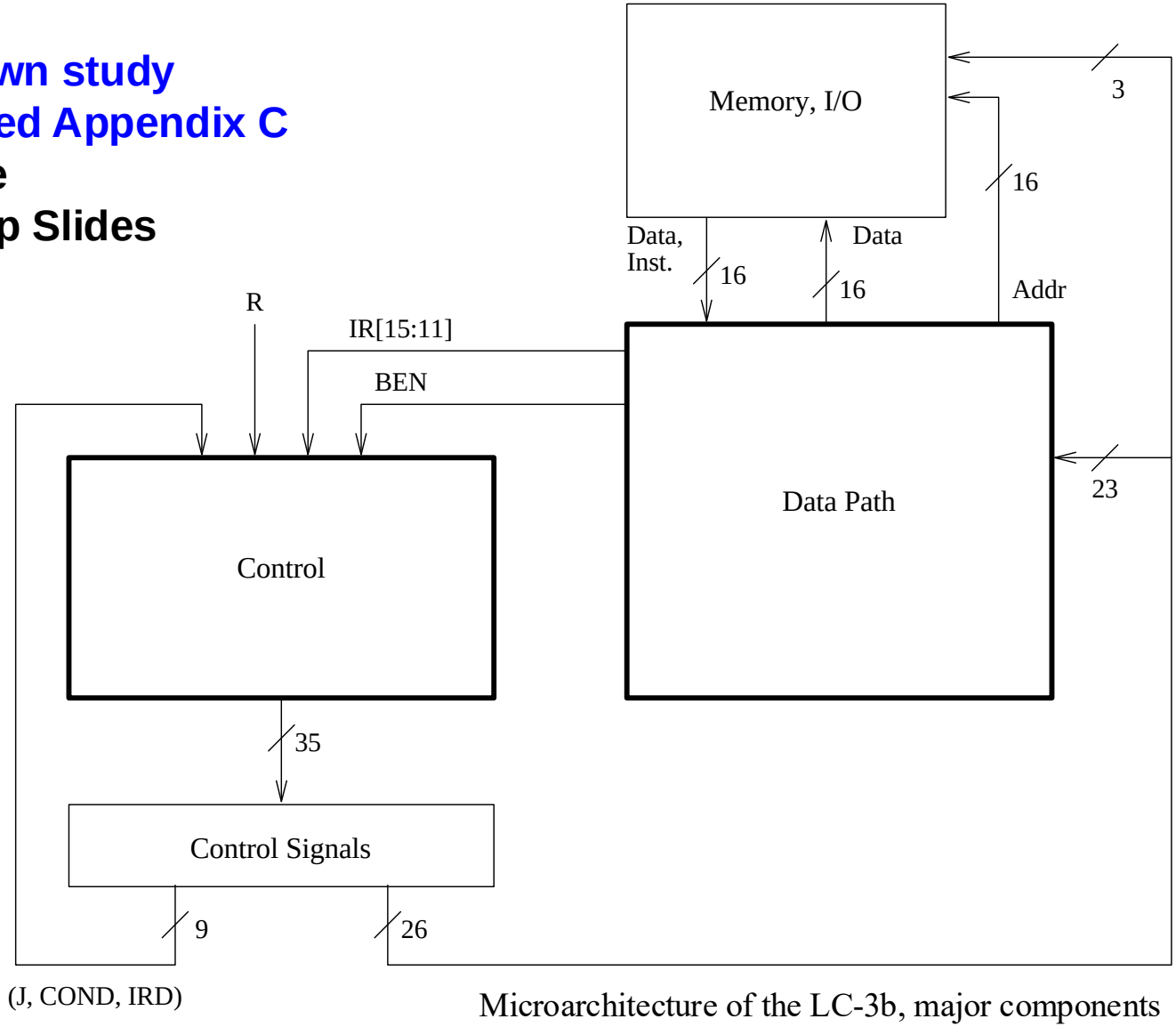
Example Control Structure



Simple Design
of the Control Structure

Example uProgrammed Control & Datapath

For your own study
P&P Revised Appendix C
On website
+ In Backup Slides



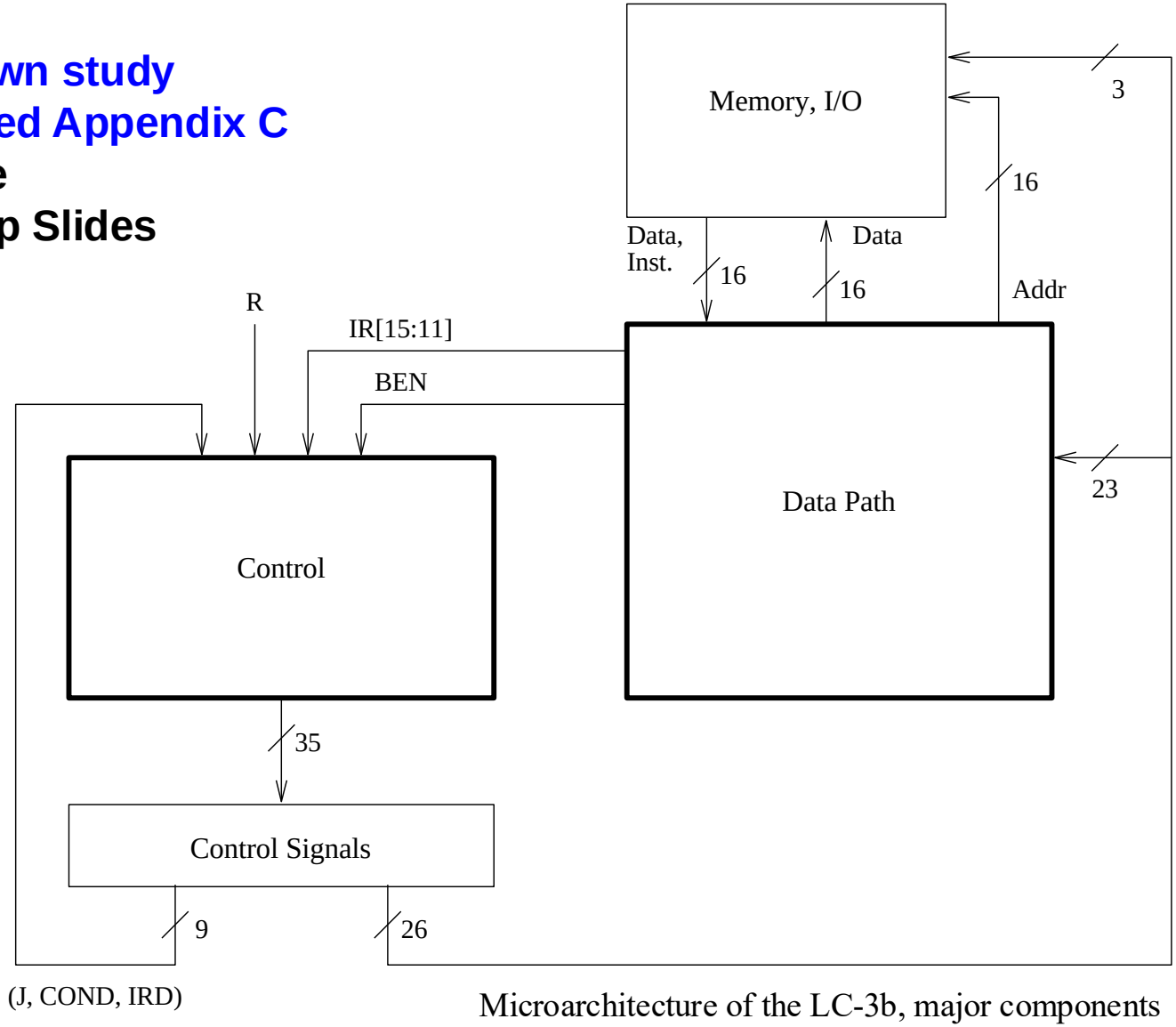
Microarchitecture of the LC-3b, major components

What Happens In A Clock Cycle?

- The control signals (microinstruction) for the current state control two things:
  - Processing in the data path
  - Generation of control signals (microinstruction) for the next cycle
- Datapath and microsequencer operate concurrently
- Question: why not generate control signals for the current cycle in the current cycle?
  - This could lengthen the clock cycle

Example uProgrammed Control & Datapath

For your own study
P&P Revised Appendix C
On website
+ In Backup Slides



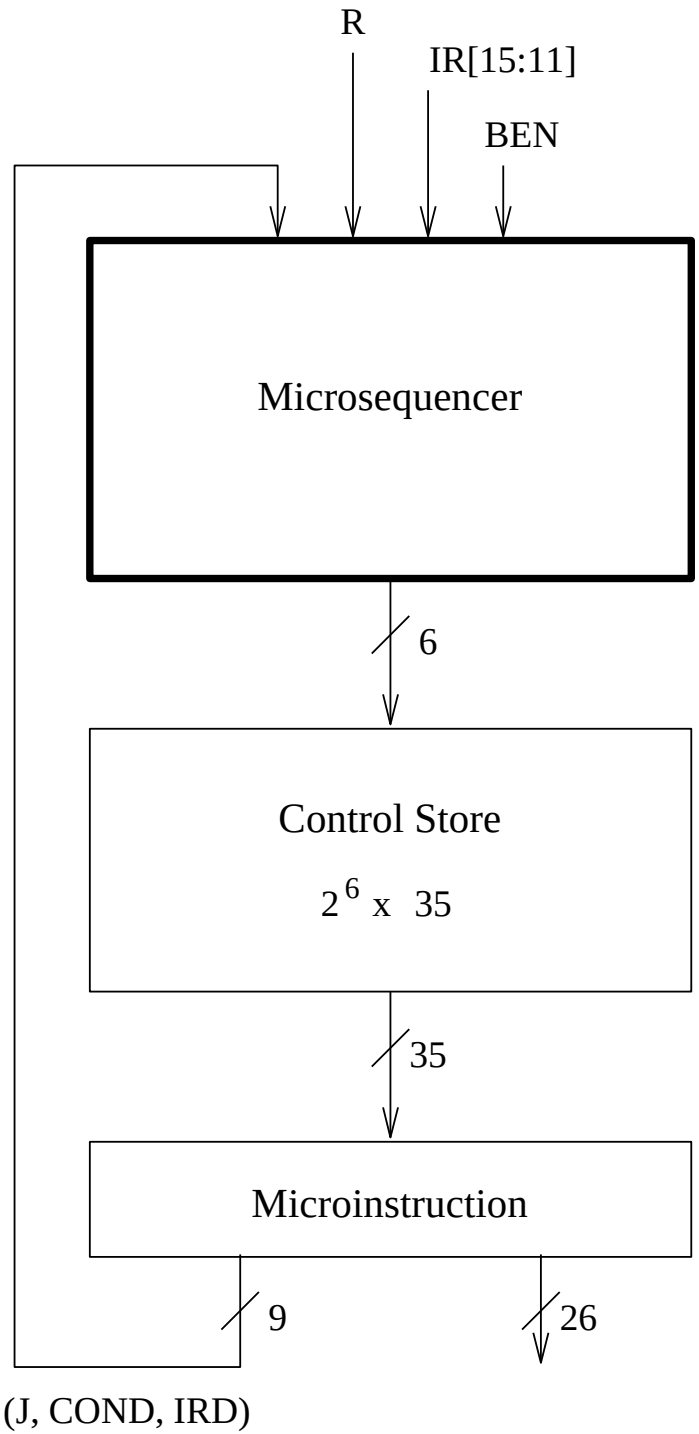
Microarchitecture of the LC-3b, major components

Microprogrammed Control Structure

- Three components: Microinstruction, Control store, Microsequencer
- Microinstruction: control signals that control the datapath (26 of them) and help determine the next state (9 of them)
- Each microinstruction is stored in a *unique location* in the control store (a special memory structure)
  - *Unique location*: address of the state corresponding to the microinstruction
  - Each state in the FSM corresponds to one microinstruction
- Microsequencer determines the address of the next microinstruction (i.e., next state)

Patt and Patel, Appendix C, Figure C.4

uProgrammed Control Structure



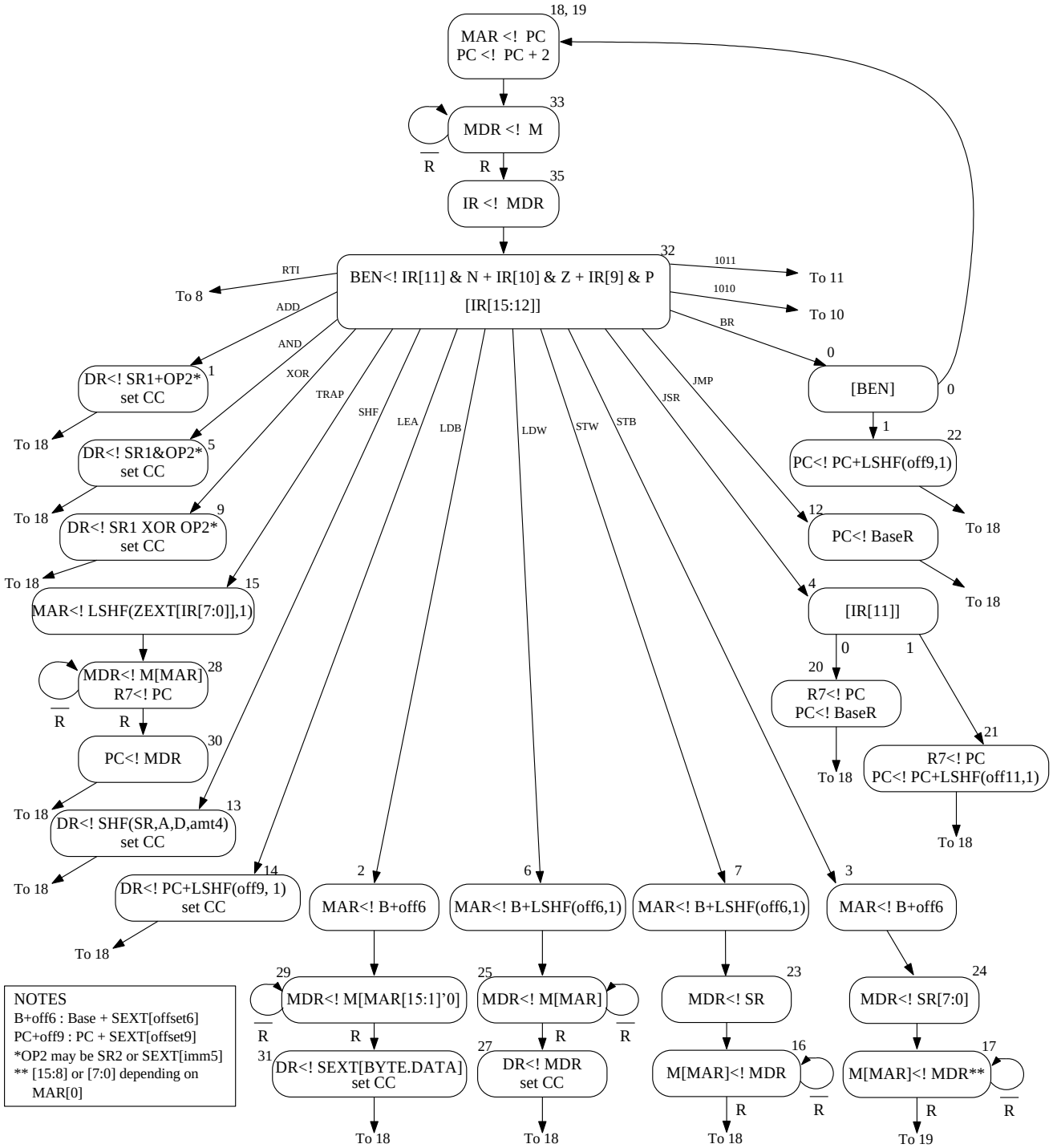
Simple Design
of the Control Structure

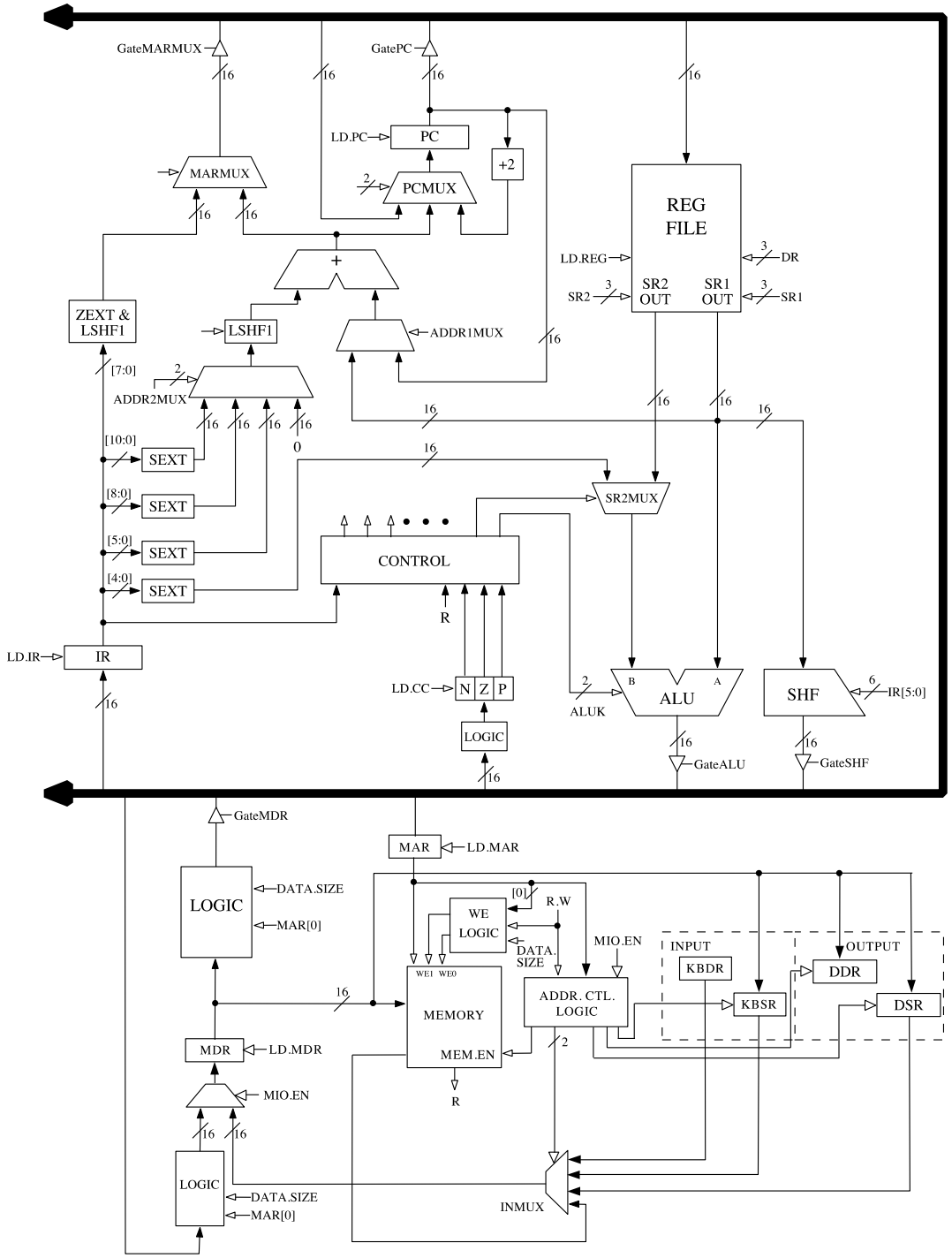
Control Store

IRD	Cond	J	LD.MAR	LD.MDR	LD.IR	LD.BEN	LD.REG	LD.CC	LD.PC	GatePC	GateMDR	GateALU	GateMARMUX	GateSHF	PCMUX	DRMUX	SR1MUX	ADDR1MUX	ADDR2MUX	MARMUX	ALUK	MIO.EN	R.W	DATA.SIZE	LSHF1	
																										000000 (State 0)
																										000001 (State 1)
																										000010 (State 2)
																										000011 (State 3)
																										000100 (State 4)
																										000101 (State 5)
																										000110 (State 6)
																										000111 (State 7)
																										001000 (State 8)
																										001001 (State 9)
																										001010 (State 10)
																										001011 (State 11)
																										001100 (State 12)
																										001101 (State 13)
																										001110 (State 14)
																										001111 (State 15)
																										010000 (State 16)
																										010001 (State 17)
																										010010 (State 18)
																										010011 (State 19)
																										010100 (State 20)
																										010101 (State 21)
																										010110 (State 22)
																										010111 (State 23)
																										011000 (State 24)
																										011001 (State 25)
																										011010 (State 26)
																										011011 (State 27)
																										011100 (State 28)
																										011101 (State 29)
																										011110 (State 30)
																										011111 (State 31)
																										100000 (State 32)
																										100001 (State 33)
																										100010 (State 34)
																										100011 (State 35)
																										100100 (State 36)
																										100101 (State 37)
																										100110 (State 38)
																										100111 (State 39)
																										101000 (State 40)
																										101001 (State 41)
																										101010 (State 42)
																										101011 (State 43)
																										101100 (State 44)
																										101101 (State 45)
																										101110 (State 46)
																										101111 (State 47)
																										110000 (State 48)
																										110001 (State 49)
																										110010 (State 50)
																										110011 (State 51)
																										110100 (State 52)
																										110101 (State 53)
																										110110 (State 54)
																										110111 (State 55)
																										111000 (State 56)
																										111001 (State 57)
																										111010 (State 58)
																										111011 (State 59)
																										111100 (State 60)
																										111101 (State 61)
																										111110 (State 62)
																										111111 (State 63)

Each entry in the control store is a microinstruction corresponding to the FSM state

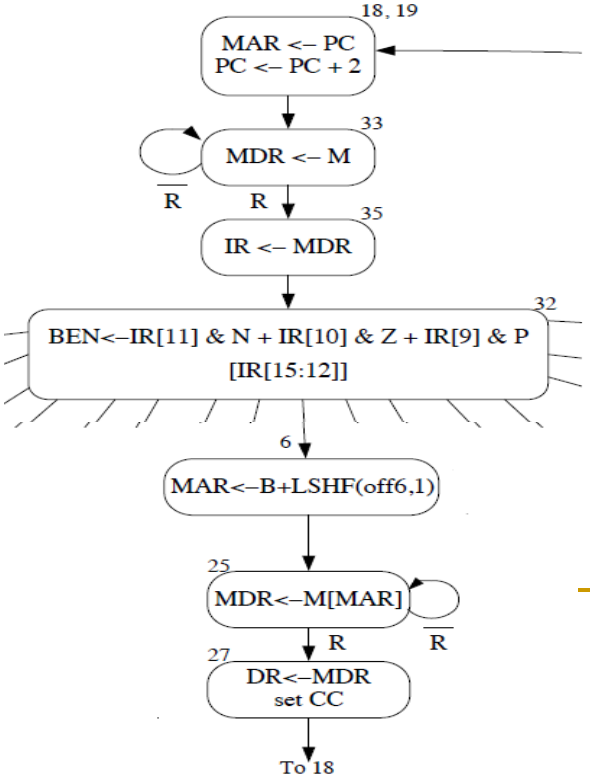
FSM state number is used to address the control store to get the relevant microinstruction


18, 19
MAR <! PC
PC <! PC + 2
33
MDR <! M
R
R
35
IR <! MDR
32
BEN<! IR[11] & N + IR[10] & Z + IR[9] & P
[IR[15:12]]
RTI
To 8
ADD
AND
XOR
TRAP
SHF
LEA
LDB
LDW
STW
STB
JSR
JMP
BR
1011
To 11
1010
To 10
1
DR<! SR1+OP2*
set CC
To 18
5
DR<! SR1&OP2*
set CC
To 18
9
DR<! SR1 XOR OP2*
set CC
To 18
15
MAR<! LSHF(ZEXT[IR[7:0]],1)
28
MDR<! M[MAR]
R7<! PC
R
R
30
PC<! MDR
To 18
13
DR<! SHF(SR,A,D,amt4)
set CC
To 18
14
DR<! PC+LSHF(off9, 1)
set CC
To 18
0
[BEN]
0
1
22
PC<! PC+LSHF(off9,1)
To 18
12
PC<! BaseR
To 18
4
[IR[11]]
0
1
20
R7<! PC
PC<! BaseR
To 18
21
R7<! PC
PC<! PC+LSHF(off11,1)
To 18
2
MAR<! B+off6
29
MDR<! M[MAR[15:1]'0]
R
R
31
DR<! SEXT[BYTE.DATA]
set CC
To 18
6
MAR<! B+LSHF(off6,1)
25
MDR<! M[MAR]
R
R
27
DR<! MDR
set CC
To 18
7
MAR<! B+LSHF(off6,1)
23
MDR<! SR
16
M[MAR]<! MDR
R
R
To 18
3
MAR<! B+off6
24
MDR<! SR[7:0]
17
M[MAR]<! MDR**
R
R
To 19
NOTES
B+off6 : Base + SEXT[offset6]
PC+off9 : PC + SEXT[offset9]
*OP2 may be SR2 or SEXT[imm5]
** [15:8] or [7:0] depending on
MAR[0]

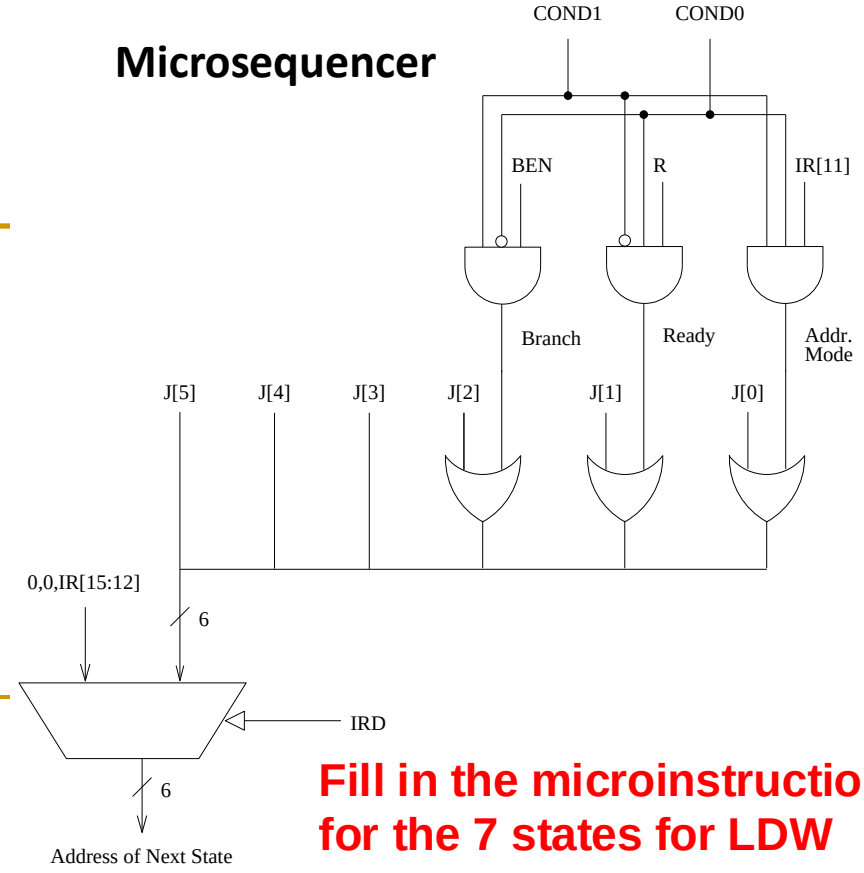


A Simple Datapath Can Become Very Powerful

State Machine for LDW



Microsequencer



Fill in the microinstructions for the 7 states for LDW

	IRD	Cond	J	LD.MAR	LD.MDR	LD.IR	LD.BEN	LD.REG	LD.CC	LD.PC	GatePC	GateMDR	GateALU	GateMARMUX	GateSHF	PCMUX	DRMUX	SR1MUX	ADDR1MUX	ADDR2MUX	MARMUX	ALUK	MIO.EN	R.W	DATA.SIZE	LSHF1
State 18 (010010)																										
State 33 (100001)																										
State 35 (100011)																										
State 32 (100000)																										
State 6 (000110)																										
State 25 (011001)																										
State 27 (011011)																										

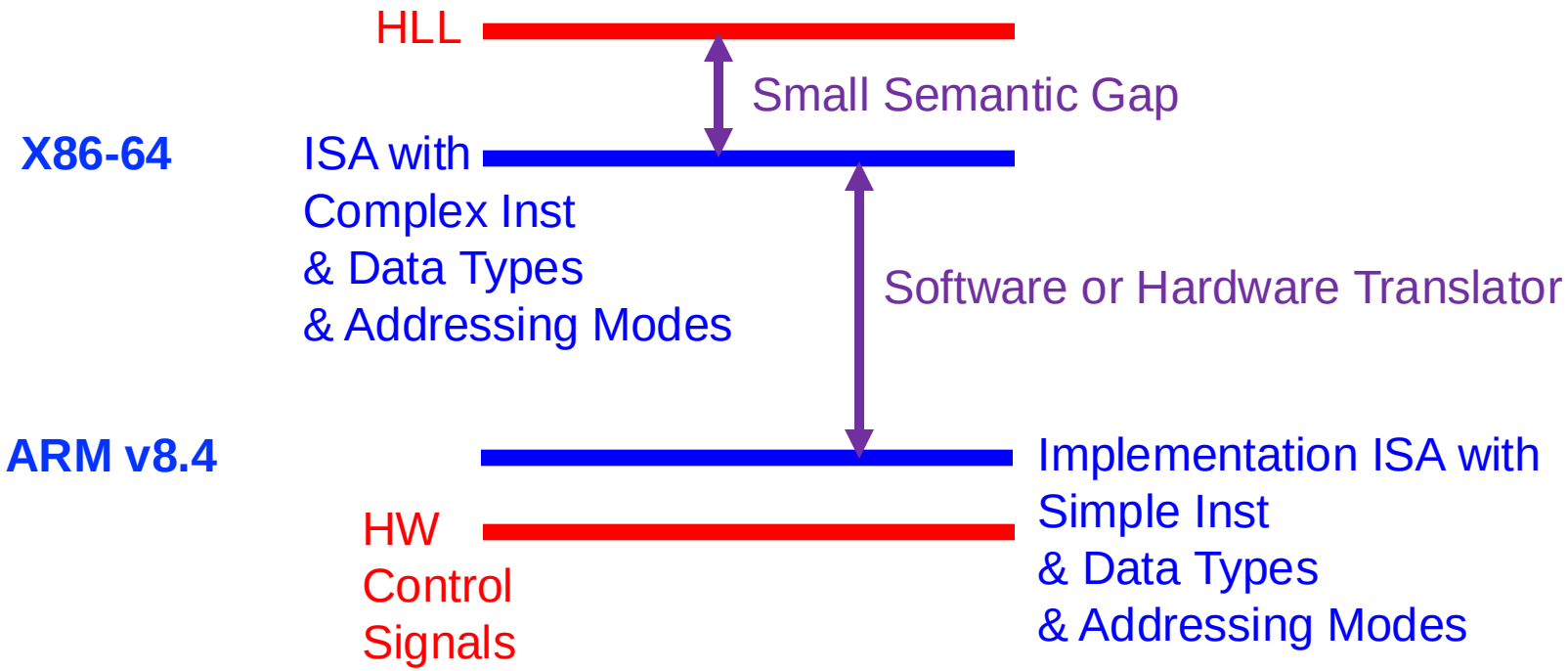
The Power of Abstraction

- The concept of a control store of microinstructions enables the hardware designer with a new abstraction: microprogramming

- The designer can translate any desired operation to a sequence of microinstructions
- All the designer needs to provide is
  - The sequence of microinstructions needed to implement the desired operation
  - The ability for the control logic to correctly sequence through the microinstructions
  - Any additional datapath elements and control signals needed (no need if the operation can be "translated" into existing control signals)

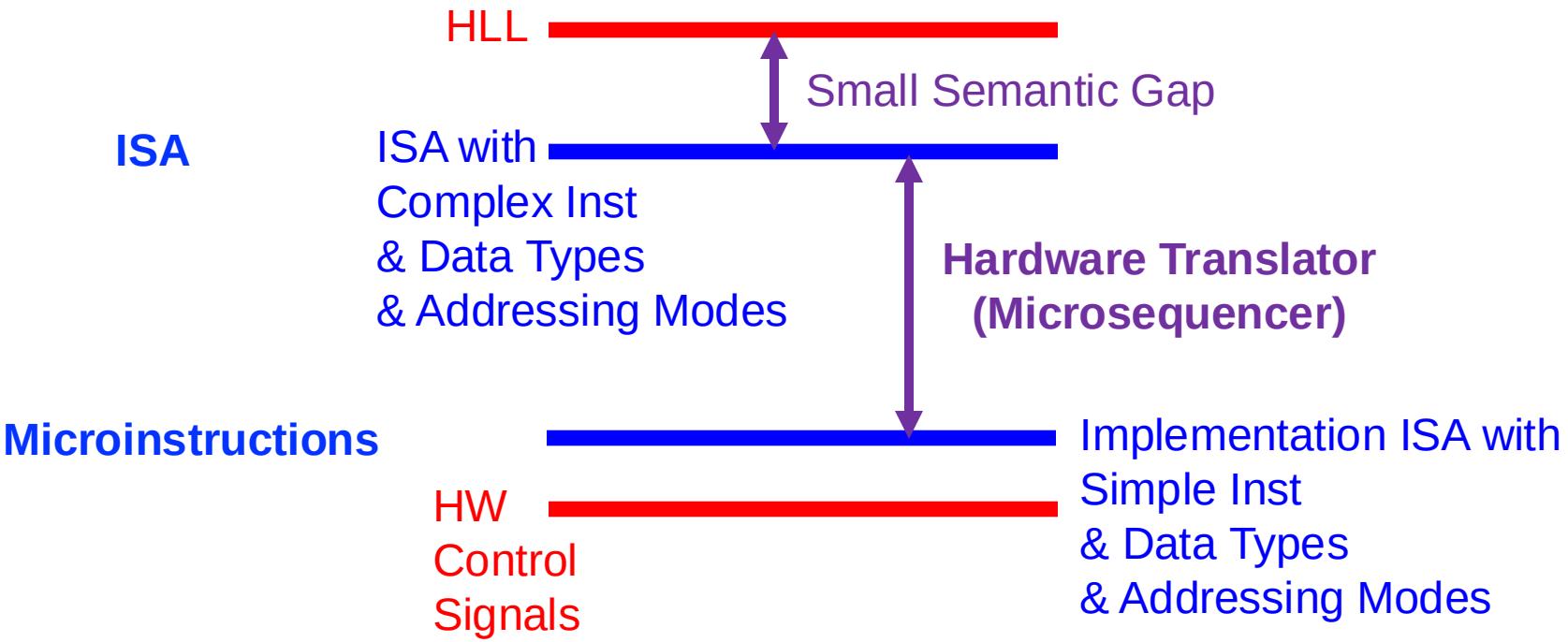
Recall: How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different "implementation" ISA



How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different "implementation" ISA



Advantages of Microprogrammed Control

- Allows a very simple design to do powerful computation by controlling the datapath (using a sequencer)
  - High-level ISA translated into microcode (sequence of u-instructions)
  - Microcode (u-code) enables a minimal datapath to emulate an ISA
  - Microinstructions can be thought of as a user-invisible ISA (u-ISA)

- Enables easy extensibility of the ISA
  - Can support a new instruction by changing the microcode
  - Can support complex instructions as a sequence of simple microinstructions (e.g., REP MOVS, MultiDimensional Array Updates)

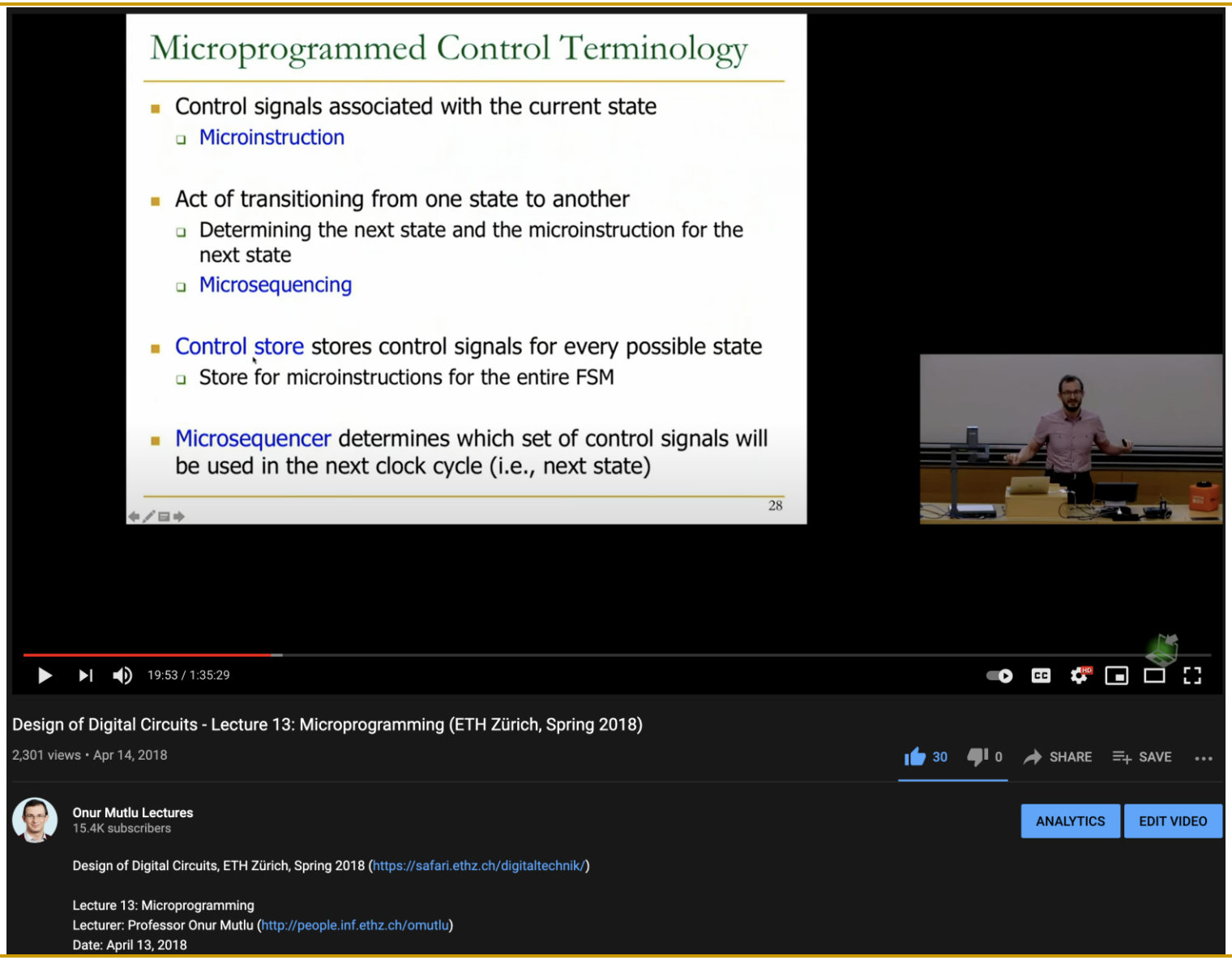
- Enables update of machine behavior
  - A buggy implementation of an instruction can be fixed by changing the microcode in the field
    - Easier if datapath provides ability to do the same thing in different ways

Update of Machine Behavior

- The ability to update/patch microcode in the field (after a processor is shipped) enables
  - Ability to add new instructions without changing the processor!
  - Ability to "fix" buggy hardware implementations

- Historical Examples
  - IBM 370 Model 145: microcode stored in main memory, can be updated after a reboot
  - IBM System z: Similar to 370/145.
    - Heller and Farrell, "Millicode in an IBM zSeries processor," IBM JR&D, May/Jul 2004.
  - B1700 microcode can be updated while the processor is running
    - User-microprogrammable machine!
    - Wilner, "Microprogramming environment on the Burroughs B1700", CompCon 1972.
  - Systems today use microcode patches to fix HW bugs/issues

For More on Microprogrammed Designs



https://www.youtube.com/onurmutlulectures

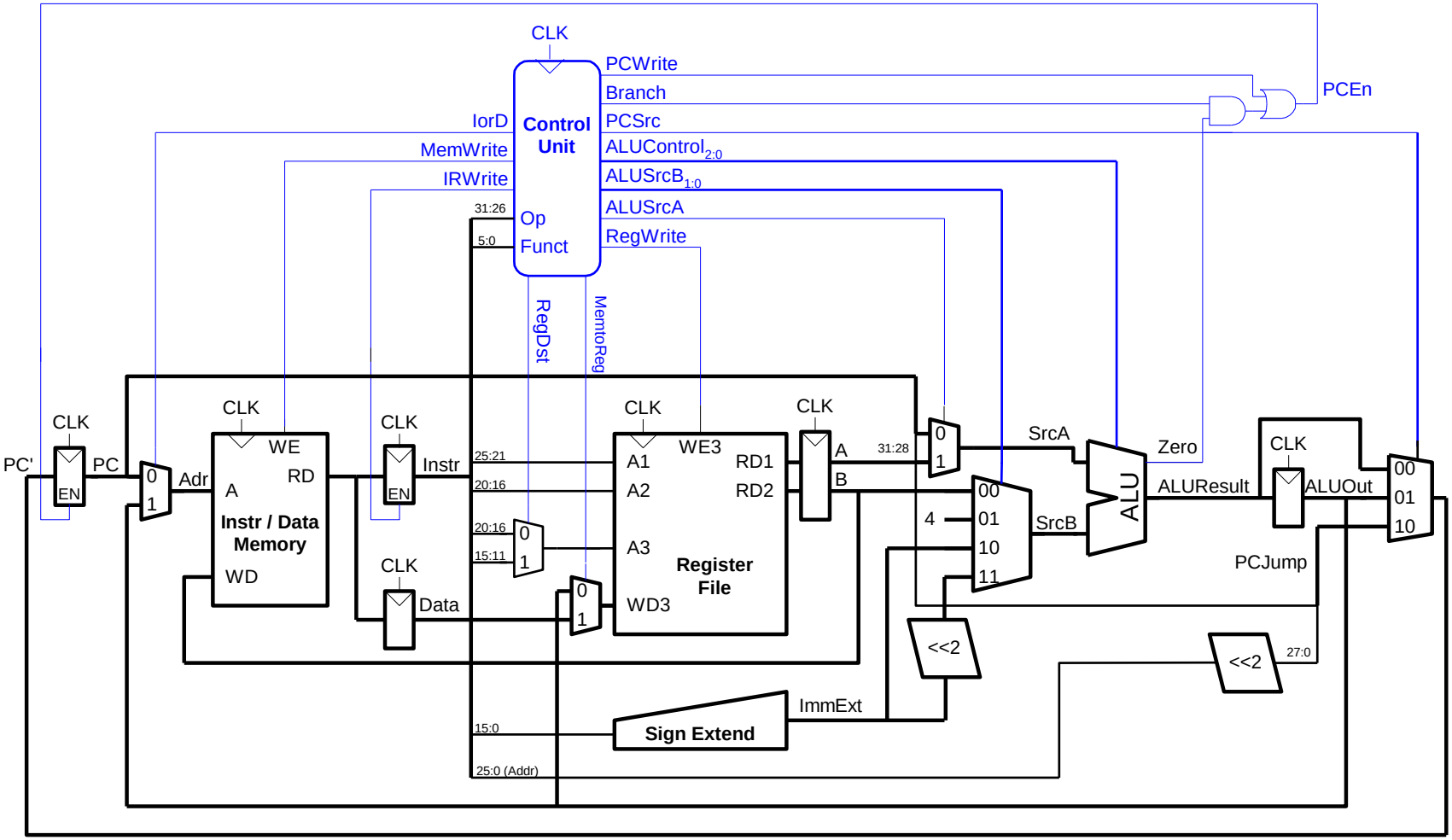
Detailed Lectures on Microprogramming

- Design of Digital Circuits, Spring 2018, Lecture 13
  - Microprogramming (ETH Zürich, Spring 2018)
  - https://www.youtube.com/watch?v=u4GhShuBP3Y&list=PL5Q2soXY2Zi_QedyPWtRmFUJ2F8DdYP7l&index=13

- Computer Architecture, Spring 2013, Lecture 7
  - Microprogramming (CMU, Spring 2013)
  - https://www.youtube.com/watch?v=_igvSl5h8cs&list=PL5PHm2jkkXmidJOd59REog9jDnPDTG6IJ&index=7

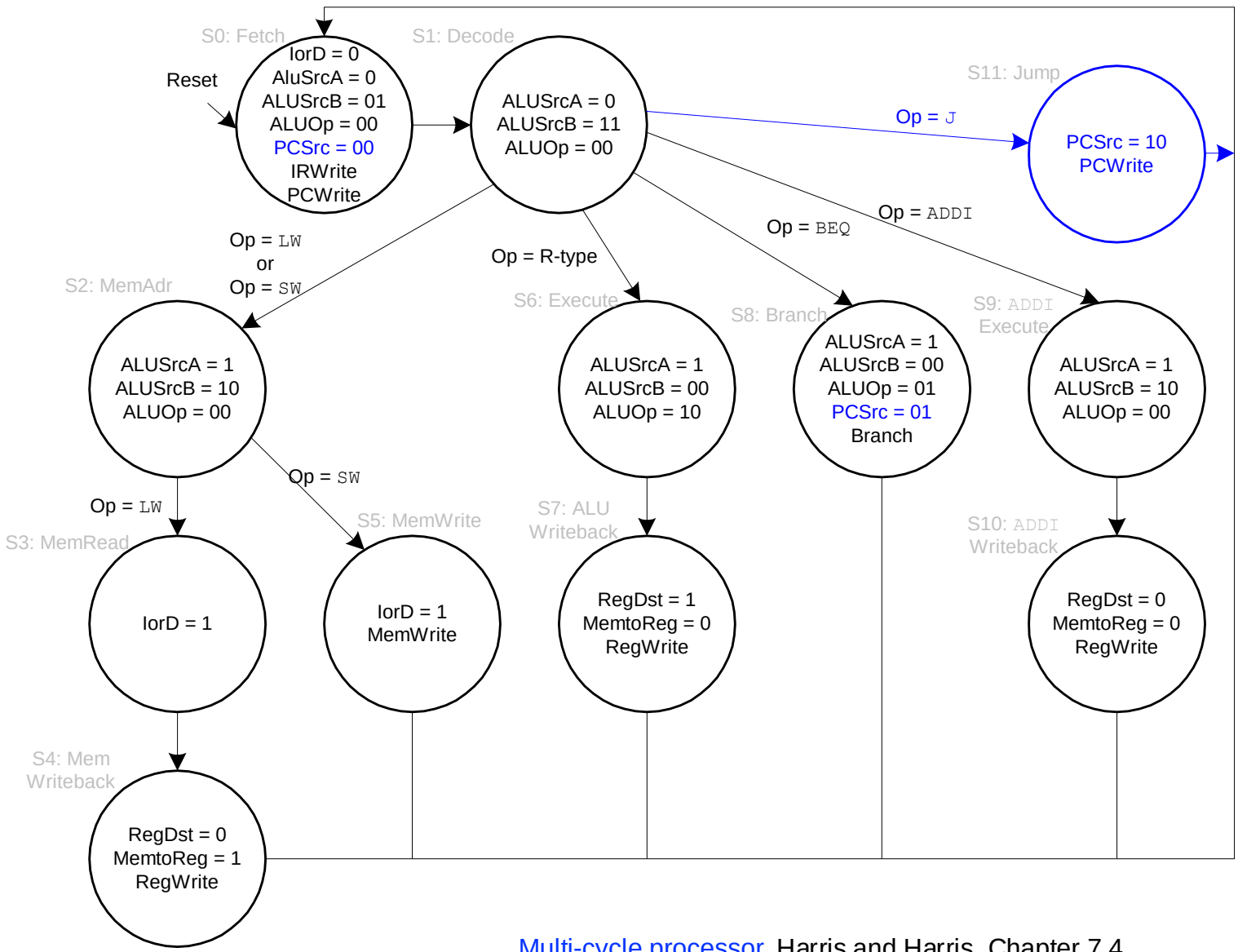
Can We Do Better?

Review: Multi-Cycle MIPS Processor



Multi-cycle processor. Harris and Harris, Chapter 7.4.

Review: Multi-Cycle MIPS FSM



We can easily handle multi-cycle memory access

Multi-cycle processor. Harris and Harris, Chapter 7.4.

Can We Do Better?

Can We Do Better?

- What limitations do you see with the multi-cycle design?

- Limited concurrency
  - Some hardware resources are idle during different phases of instruction processing cycle
  - "Fetch" logic is idle when an instruction is being "decoded" or "executed"
  - Most of the datapath is idle when a memory access is happening

Can We Use the Idle Hardware to Improve Concurrency?

- Goal: More concurrency → Higher instruction throughput (i.e., more "work" completed in one cycle)

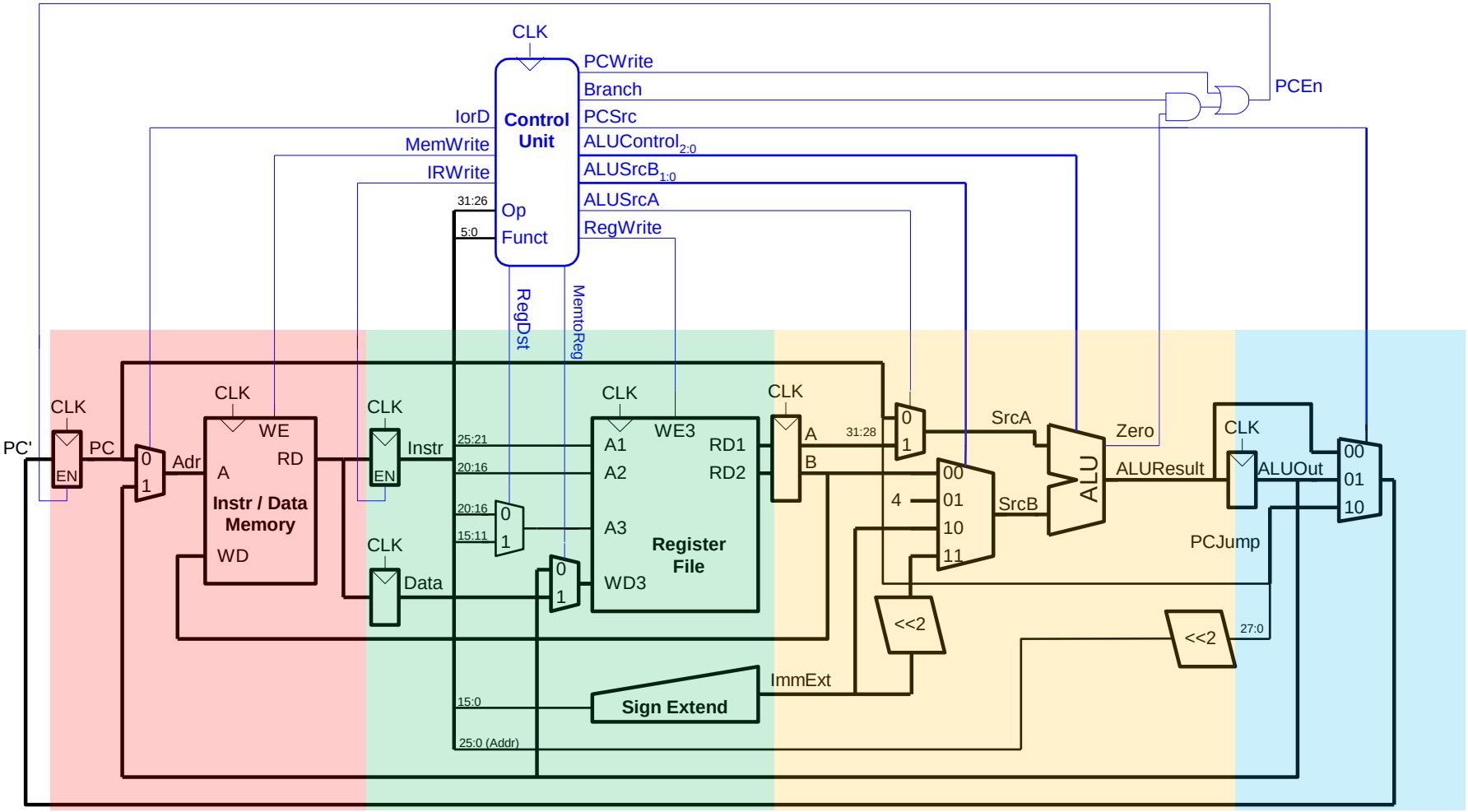
- Idea: When an instruction is using some resources in its processing phase, process **other instructions** on **idle resources** not needed by that instruction
  - E.g., when an instruction is being decoded, fetch the next instruction
  - E.g., when an instruction is being executed, decode another instruction
  - E.g., when an instruction is accessing data memory (ld/st), execute the next instruction
  - E.g., when an instruction is writing its result into the register file, access data memory for the next instruction

Can Have Different Instructions in Different Stages

- Fetch
- Decode
- Evaluate Address
- Fetch Operands
- Execute
- Store Result

1. Instruction fetch (IF)
2. Instruction decode and
   register operand fetch (ID/RF)
3. Execute/Evaluate memory address (EX/AG)
4. Memory operand fetch (MEM)
5. Store/writeback result (WB)

Can Have Different Instructions in Different Stages



Of course, we need to be more careful than this!